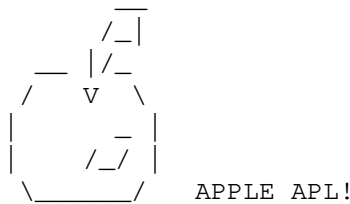


apple MANUAL

*by Antonio Maschio
programmer in Montebelluna (TV) - Italy*

ABSTRACT



This is **apple**, my APL engine. **apple**, which must be written always in lower letters, is not an official version of APL, but my personal version, basically obeying to the rules of the IBM APL\1130 version, described in the 1968 book "APL\1130 Primer" for the Students, and in the 1969 book "A Programming Language/1130", where a more complete version of the language is exposed. **apple** uses ASCII characters or combination of ASCII characters, and does not need any special keyboard to be used, nor special characters or special fonts. **apple** has also a few extensions and enhancements with respect to the original version, mostly grabbed from the ISO and the Dyalog's APL, though with some differences (because **apple** has only bidimensional matrices and works only with real numbers).

This manual adopts the 1969 manual as source, and reports and categorizes all APL's implemented features; I reversed here all the experience I gained in studying and using this wonderful language, which is not a programming language in the common sense, but rather a mathematical language, and this has some meaning to me. Read also the man page for other more general info, all options description and the installing instructions.

The project onto **apple** is based was conceived with the purpose of avoiding all causes of segmentation faults and memory errors, but I fear there could be some left. You are encouraged to report all errors and inconsistencies found (both in the program and in the manual) to

ing dot antonio dot maschio at gmail dot com

I don't ask money for this program. I only ask for a voluntary contribution, to be made through PayPal (any amount), just to support my work and let me know you like it. Donations can be directed to

tbin at libero dot it

Thanks in advance to anyone of you who will contribute. And thanks anyway to anyone of you who will download and use **apple**! Remember: An **apple** a day keeps the doctor away!

*This work is dedicated to the memory
of Kenneth E. Iverson (1920-2004)
Canadian computer scientist*

1. INTRODUCTION

Extracts from the original preface in the 1969 book "A Programming Language/1130": *APL is a conversational implementation of the Iverson notation, an extremely concise mathematical notation with simple but rigorous syntax. This concise attribute virtually eliminates the -program- step in the problem-solving chain of problem--algorithm--program--solution. All operators of the notation, editing capabilities, and the capability to save and retrieve work spaces are provided. The implementation allows data to be structured as scalars, vectors and matrices with up to 255 elements in any dimension.*

Numerical values are accurate to six decimal digits, and identifiers are up to 6 alphabetic characters. Input may come from the console typewriter, card reader or a typewriter terminal. The program is independent of the IBM monitor and requires a dedicated disk cartridge. Facilities are provided to generate the system, assign and delete workspaces, and dump/restore individual work-spaces and their functions to cards.

This brief introduction precedes a one hundred complex pages of compact text, not really easy to read but very complete and precise and, for those who dare face this text, an incredible and exciting new world unfolds before your eyes. For this, I invite you to read this book.

1.1. A necessary preamble

This work is surely my most ambitious project, and **apple** is the most important interpreter in my programming career. It's probably imperfect, and it probably needs some adjustments (it's an Alpha version, after all), but I feel that I have reached the top of my programming skills, both in understanding a language and writing some code that understands it on its turn.

Of course, I will never reach the APL programming heights of those obscure programmers who could write a matrix inversion program in just four lines¹, nor will I fully understand the entire project behind the mathematical rules. Anyway, I think that this is the right place to thank a friend and former colleague who took the trouble to read the manual and who suggested some fundamental corrections: thanks Luigi! And I also want to thank Ian, an old English friend of mine, who had helped me with my other past interpreters, and who found many errors here and there, especially in the initial phase, when I was a bit confused by this new programming language that I was about to study.

I hope you'll like it. And if so, let me know, please.

1.2. What is this manual?

This manual illustrates **apple**², my APL interpreter. **apple** must be written always in lower letters (so no Apple, AP-LE or the like) and is principally based on the IBM version built for the 1130 machines and described in the 1968 book "APL\1130 Primer" and also, more completely, in the 1969 book "A Programming Language/1130".

Indeed, APL\1130 was the first publicly available APL system, released by IBM in 1968 for the IBM 1130 computer. It became the most popular IBM Type-III Library software released by IBM, contributing to the success of the first APL conference.

APL\1130 was a complete APL system, supporting all the language's features, including:

- APL operators for matrix and vector manipulation

¹ See page B.10 of the book "APL\360 User's Manual" (1970), by DEC.

² I know that this name, which is the name of an innocent fruit, has already been used by a much more powerful computer industry. The reference that I use here is of course the name of the fruit (written with a lower starting character), and I hope that the owners of the Other Company won't force me to change it. After all, and until a strong disproof, an apple is only a fruit.

- APL functions for mathematics, statistics, and data processing
- An integrated development environment (IDE) that included a text editor, a debugger, and an APL terminal

APL\1130 was used by a wide range of users, including students, researchers, and professionals. It was used for a variety of applications, including:

- Scientific, mathematical and engineering computing
- Data processing
- Data visualisation
- Functional programming

APL\1130 was an important starting point in the development of the further versions of the APL language. It helped to make APL a popular language and helped to spread it around the world.

Here are some technical details about the original APL\1130:

- Operating system: IBM OS/1100
- Memory: 8K to 64K 16-bit words
- Hard disk: 1 megabyte
- Graphical display: IBM Grafa
- User interface: APL terminal

APL is still in use by some users today, and is considered a classic piece of software.

Here is the Dyalog APL introduction to the TryAPL site: "*APL is an array-oriented programming language that will change the way you think about problems and data. With a powerful, concise syntax, it lets you develop shorter programs that enable you to think more about the problem you're trying to solve than how to express it to a computer.*"³.

I believe this is true, and I fully agree with it!

apple tries to emulate all the main features of the terminal commands and of the APL\1130 language, with the main difference that it uses commands typed as common ASCII characters (rather than those obtained from special character combinations that perform the transformation into the formal APL characters set), and possibly with some limitations in the program editor and in executing programs. However, it's not an ISO version (I am referring to the ISO 8485:1989 standard), released 20 years after the release of APL\1130, which is perhaps simpler, but at the same time it's a powerful, version. We may agree on the fact that the APL\1130, that was created with the limitation of arrays confined to 255 values for each dimension, does not meet the current requirements (which involve computing matrices with millions of values), but **apple** was built for historical reasons, to start appreciating this mathematical notation language, and follows the 1968-1969 IBM protocol, with few differences (e.g. , it has removed the restriction of 255 values for each dimension and the limit's only in the memory of the host computer).

1.3. Sparse bibliography

Some texts made a crucial role during the developing of **apple**; they are listed here:

- 1 In 1968 IBM published the book 'APL1130 primer' (from now on APL68), dedicated to students, focused on vectors, and excluding most of the matrices functions not having a direct use on vectors. It can be downloaded here:
http://bitsavers.org/pdf/ibm/1130/lang/C20-1697-0_APL_1130_Primer_1968.pdf

³ See <https://tryapl.org/>

- 2 in 1969 IBM published the book 'A Programming Language/1130' (from now on APL69), with all the references to the language features implemented for the 1130 machines. It can be found here:
http://bitsavers.org/pdf/ibm/1130/lang/1130-03.3.001_APL_1130_May69.pdf
- 3 in 1970 IBM published a revised and integrated version of APL69, titled 'APL360 User's Manual' (from now on APL70), with all the references to the language features implemented for the 360 machines. It can be found here:
http://bitsavers.org/pdf/ibm/apl/GH20-0683-1_APL_360_Users_Manual_Mar70.pdf
- 4 Another important text, the IBM APL manual, for the IBM 5110 system (from now on APL77) was useful for debugging apple; it adopted an expanded version of APL, but including all the features of the 1130 version. It can be found from bitsavers.org:
/pdf/ibm/5110/SA21-9303-0_IBM_5110_APL_Reference_Manual_Dec1977.pdf
- 5 The advanced features of apple were inspired (if not directly copied) from the DEC APL AA-5076B-TC manual targeted for the PDP11 computers (January 1980, from now on DEC80). As always, DEC shows a great ability to write complete manuals, full of examples and clear. It can be found from bitsavers.org:
/pdf/dec/pdp11/lang/apl/AA-5076B-TC_APL-11_Programmers_Reference_Manual_Jan80.pdf
- 6 I have discovered another ASCII APL interpreter called APL11, which can be downloaded here:
<http://www.catb.org/retro/apl11.tar.gz>
but I warn you that this software, dating back to 1992, is not compilable under the modern compiler systems, so this version is interesting only in virtue of its manual, from which I indeed grabbed some ideas. Anyway, this is said to contain substantial amounts of code written by Ken Thompson, one of UNIX's co-inventors, code that was taken then by more programmers that extended the properties of APL11, the most recent programmer who delivered the software being Michael Cain (November 1992).
- 7 By the way, the GNU APL can be found here:
<https://www.gnu.org/software/apl/>
- 8 A historical article by Leonard J. Shustek, with lots of news about the birth of APL is here:
<https://computerhistory.org/blog/the-apl-programming-language-source-code/>
- 9 Last, but not least, here is a historical article by Larry Breed, which illustrates the birth of the IBM APL\1130, available from web.archive.org:
/web/20080512031437/http://www.vector.org.uk/archive/v223/APL_1130.htm
- 10 Historical and practical relevance have the Modifications to the APL1130 System (parts 1, 2 and 3), that prove the continuous development behind the APL\1130, and that can be downloaded here from the ACM Digital Library site:
Part 1: <https://dl.acm.org/doi/pdf/10.1145/1316871.1317467>
Part 2: <https://dl.acm.org/doi/pdf/10.1145/1316873.1317458>
Part 3: <https://dl.acm.org/doi/pdf/10.1145/1115910.1115915>
- 11 Historical relevance has also the Summary of the presentation at APL user conference workshop 3, that can be found here:
<https://dl.acm.org/doi/pdf/10.1145/1316871.1317464>

2. THE APL LANGUAGE

2.1. A bit of history

The APL language is not a real programming language, but an environment dedicated to math. It lacks all instructions to make cycles, that must be achieved through go-tos and labels (the good old way, I'd say) but not so much surprisingly, it's got many capabilities for programming and variables-management, and it has one great property: a native array-driven calculus, that makes APL unique (at least in 1969) among programming languages. Variables are of two types, numeric or string, and a variable can indifferently host string or numerical values, without any special declaration.

APL was born after the studies of Kenneth E. Iverson for mathematical notation applied to computing, starting in 1957 at the Harvard University. In 1960, he began work for IBM, where he could formalize his theories, and published his book *A Programming Language* in 1962 in collaboration with Adin Falkoff.

After the publication, Iverson tried to implement a complete computer system, with the help of Falkoff and also of William C. Carter, for the set of the machines that later became the IBM System/360 family.

Wikipedia reports: "*In 1963, Falkoff, Iverson, and Edward H. Sussenguth Jr., all working at IBM, used the notation for a formal description of the IBM System/360 series machine architecture and functionality, which resulted in a paper published in IBM Systems Journal in 1964. After this was published, the team turned their attention to an implementation of the notation on a computer system. One of the motivations for this focus of implementation was the interest of John L. Lawrence who had new duties with Science Research Associates, an educational company bought by IBM in 1964. Lawrence asked Iverson and his group to help use the language as a tool to develop and use computers in education.*

After Lawrence M. Breed and Philip S. Abrams of Stanford University joined the team at IBM Research, they continued their prior work on an implementation programmed in FORTRAN IV for a part of the notation which had been done for the IBM 7090 computer running on the IBSYS operating system. This work was finished in late 1965 and later named IVSYS (for Iverson system). The basis of this implementation was described in detail by Abrams in a Stanford University Technical Report, "An Interpreter for Iverson Notation" in 1966, the academic aspect of this was formally supervised by Niklaus Wirth. Like Hellerman's PAT system earlier, this implementation did not include the APL character set but used special English reserved words for functions and operators. The system was later adapted for a time-sharing system and, by November 1966, it had been reprogrammed for the IBM System/360 Model 50 computer running in a time-sharing mode and was used internally at IBM."

This manual explains some APL features in detail, with many examples for each function or operator, so I guess it may be considered a primer, but since the descriptions of the APL functions are brief and perhaps not exhaustive, the APL68, the APL69 and the APL70, and also the APL77 and the DEC80, should be consulted for a deep knowledge of the required features.

Anyway, it's beautiful thinking that the Beatles were assembling *Abbey Road* when the APL\1130 was being developed...

2.2. The algebraic principle

The algebraic principle on which APL is based is peculiar: it's not the classic algebraic mode we are used to, the one in use in the majority of algebraic calculators, and not even similar to the RPN mode of the classic HP calculators.

The calculation proceeds always from right. Take for instance a simple math expression like this:

$2 \times 4 + 7$

My mental representation for this operation (I bet it's the same for you) is: first multiply 2 by 4, and then sum the

result to 7; result is 15. APL works differently: proceeding from right, it finds that 4 and 7 can be summed, giving 13, and then the result is multiplied by 2, yielding 26.

I report here what in turn is reported in APL68:

"...This is the rule: An operator operates on everything to the right of it. Stating it more formally, any operator takes as its right argument everything to the right of it.

This order is the same of the usual order of English speech. You may not have thought of English speech in that way, but it will be obvious in a brief example. Take the following English sentence:

"They oppose a rise in the price of products from farms."

To examine the meaning of this sentence, let's pose some questions about what applies to what.

Oppose what? A rise in the price of products from farms.

What rise? One in the price of products from farms.

Rise in what price? The one of products from farms.

Price of what products? The ones from farms.

Here each of the key elements in the sentence refers to all the rest of the sentence after it.

The same structure occurs in APL. Consider this example:

$$A+B \times C - D \div E$$

what is A added to? $B \times C - D \% E$

What is B multiplied by? $C-D\%E$

What is subtracted from C? D%E

What is D divided by? E

[...] when the computer executes an instruction such as

$$A+B \times C - D \div E$$

it starts by looking for the value associated with the name E. Then it looks up the value of D. That gives it enough information to evaluate D divided by E. Next it looks up the value of C, which gives enough information to evaluate the difference between C and D-divided-by-E..."

* * * * *

If you prefer (these are words of mine), you can imagine that anything is an operator, from the assignment operator to the round or square brackets, and that all the calculations are performed from right. The judicious use of parenthesis can solve any of the practical doubts for beginners, but you'll soon find that this mental model is very very useful. **apple** follows strictly the APL algebraic model.

2.3. The concept of workspace

A *workspace*, in APL, is the common organizational unit, capable of holding together all the information that the user has written. A workspace is said to be *active* when in use, *inactive* when not in use. There may be only one workspace active at a time.

An active workspace is always present when an **apple** session starts. If this is the first time, or if there is no saved workspaces, or again) CLEAN has been used, the current active workspace will be empty.

All jobs are accomplished within the workspace. The name of variables (data items) and defined functions (programs) used in calculations, always referring to objects of the current active workspace, and the information data (system flags) of the active workspace are all maintained by **apple** in the current workspace configuration. Each workspace can hold up to 512 programs (each with 512 lines at most, each line up to 512 characters long) at most, and can create up to 32768 variables. A clean workspace is quite compact and occupies only 4.27 kB (the system flags and the empty programs structures).

The system commands provide means to save and load workspaces, to copy objects from a saved workspace, to list workspaces (see the chapter "System Commands") and constitute a sort of primitive operating system.

If the user logs off the session with) CONTINUE, the current workspace is saved to a hidden file (overwriting any preceding state); a subsequent run will retrieve this workspace and continue the previous session. If the user logs off with) OFF, the current workspace is not saved nor erased (so you can restore the precedent state at the next run). To erase the hidden file, type from inside **apple** the system commands) CLEAN and then) CONTINUE.

<i>What follows is an extract from the APL69, from page 57 to page 92, reported here and adapted for apple, in the hope that original authors won't claim for its removal, because in no way a better text could replace it.</i>
--

2.4. The Terminal System

The IBM APL\1130 Terminal System executes system commands or mathematical statements entered on a terminal keyboard. The system commands are treated next in this manual; the mathematical statements will be treated here.

Acceptable statements may employ either primitive functions (e.g. + - %) which are provided by the system, or defined functions, which the user provides by entering their definitions on the terminal (or on external editors and then loaded back in the workspace).

If system commands are not used, the worst that can possibly result from erroneous use of the keyboard is the printing of an error report. It's therefore advantageous to experiment freely and to use the system itself for settling any doubts about its behavior⁴. For example, to find what happens in an attempted division by zero, simply enter the expression 4%0.

For what about the characters that form the operators, the original IBM computers onto which the IBM APL ran (1960-1970) could use a specific complex keyboard that worked in conjunction with a printer. The APL alphabet could use many specific characters and, by over-striking some of them, they could obtain more combinations for added mathematical issues.

Needless to say, these tools are no more available today; moreover, the modern ASCII keyboards (which are more or less standardized, at least for the classic 0..127 ASCII range) lack all the specific fonts that can be used on the terminal, unless complicated characters combinations are typed.

So I had to study a possible substitution protocol for all unavailable characters, adopting a simple rule: use only punctuation characters and lower letters. This leaves latitude for variable and procedure names to be written in capital letters.

⁴ I hope that all possible wrong statements are caught, and if not so, please write to <ing dot antonio dot maschio at gmail dot com> and report the issue.

The operators scheme is summarized in the following table 2.1.

IBM operator	ASCII substitute	Notes	Conventional Name
$_$	$_$	the underscore	
$\times$	x	the letter 'x'	times
$\div$	%	the percent sign	divide
$\rfloor$	}	the curly closed bracket	
$\lceil$	{	the curly open bracket	
$\otimes$	&	the ampersand	
$\wedge$	^	the caret sign	
$\vee$	v	the letter 'v'	
$\neq$	#	the grid sign	
ρ	p	the letter 'p'	rho
ι	i	the letter 'i'	iota
$\in$	e	the letter 'e'	
$\circ$	@	the 'at' sign	
Δ	/\	slash/backslash	Delta (aka <i>slash up</i>)
∇	\	backslash/slash	Del (aka <i>slash down</i>)
$\leq$	<=	less then plus equal	
$\geq$	>=	greater then plus equal	
$\wedge\sim$	^~	caret plus tilde	
$\vee\sim$	v~	the letter 'v' plus tilde	
ϕ	o	the letter 'o' plus bar	
$\ominus$	o-	the letter 'o' plus dash	
$\oslash$	\o	backslash plus the letter 'o'	
$\uparrow$	/^	caret amid slash up	
∇	\v/	letter 'v' amid slash down	
f	/-	slash plus dash	
$\backslash$	\-	backslash plus dash	
T	' '	bar amid inverse ticks	encode
$\perp$	_ _	bar amid underscores	decode
$\circ.$	@.	the 'at' sign plus dot	
$\uparrow$	/	bar amid slash up	
$\downarrow$	\ /	bar amid slash down	
$\leftarrow$	<-	lesser than plus dash	equals
$\rightarrow$	->	dash plus greater than	go to
$\square$	_	bar, underscore, bar	quad
$\square$	'	bar, tick, bar	
$\equiv$	=_	equal plus underscore	

Table 2.1: ASCII rendition

The adoption of the ASCII protocol is not smooth; the ASCII multi-character operators require a little care in writing programs.

For instance, the dash for setting the negation must be used with care, since it's also part of other combination of characters. If you want to use the / operator with a negation, you write:

1 0 1/-M

but the system will see this as the operation of /- with the vector 1 0 1 and the matrix M; to force the desired interpretation, put a blank between the symbols:

1 0 1/ -M

What is said is valid for <-, ->, o- and also for \- and in general in every expression where you use a dash.

Now suppose that you want to apply the monadic operators |, % and again | to a quantity A; so you write:

| % | A

but the system will see this as a request to invert the matrix A (with unpredictable results). The solution is of course to separate at least two monadic operators with a space:

| % | A
| % | A

These are general warnings, that you should keep in mind when you use single-character operators occurring in multi-character operators; always ask yourself if this can be misinterpreted. This is the only care to adopt when using ASCII combinations in **apple**, not so difficult to maintain, since the general rule is to add a space to separate them when in doubt.

2.5. Language fundamentals

Statements. Statements are of two main types, the branch (denoted by -> and treated in the section on Defined Functions), and the specification. A typical specification statement is of the form

X<-3x4

This statement assigns to the variable X the value resulting from the expression to the right of the specification arrow.

If the variable name and arrow are omitted, the resulting value is printed. For example:

3x4
12

Results typed by the system begin at the left margin whereas entries from the keyboard are automatically indented six spaces to the right. The keyboard arrangement does not need any special figure, because a common ASCII keyboard is sufficient.

Scalar and vector constants. All scalar numbers entered via the keyboard or typed out by the system are in decimal⁵, either in conventional form (including a decimal point if appropriate) or in exponential form. The exponential form consists of an integer or decimal fraction followed immediately by the symbol E followed immediately by an integer (the symbol is the upper E, not the lower e). The integer following E specifies the power of ten by which the part preceding E is to be multiplied. Thus 1.44E2 is equivalent to 144.

Negative numbers are represented by a negative sign _ immediately preceding the number, e.g., _1.44 and _144E_2 are equivalent negative numbers. The negative sign _ can be used only as part of a constant and is to be distinguished from the negation function which is denoted, as usual, by the minus sign -. The negative sign alone is equivalent to the scalar zero and, in some expressions, this can return an error.

apple supports the hexadecimal numbers; they start with \$, immediately followed by uppercase or lowercase

⁵ The numeric storing is the one of the C-double, which respects the IEEE-754 format in 64 bits, with one bit used for the sign, 11 bits for the exponent and 52 bits for the mantissa. The 11 bits of the exponent allows the representation of numbers between 10⁻³⁰⁸ and 10³⁰⁸, with full 16 decimal digits precision.

)DIGI 52
DIGIT IS 52
1.2
1.199999999999999955591079014993738383054733276367188
That's why **apple** limits this value to 16; a greater value is not correctly representable.

characters in the range 0 . . 9A . . F; the result is returned in form of an integer scalar. For example, \$24 is the decimal 36, and the sequence \$2A \$FF 30 is the (decimal) vector 42 255 30. Internally, the number is always stored in decimal.

The symbol \$ is not an operator; it's like the minus sign _ or the dot, which are parts of a literal number, and don't extend beyond it. For instance:

```
$45-5
13
```

This operation is not to be read as the impossible \$(45-56)⁶ (which would return _17), but rather (\$45)-56, that returns 13. Like _, using a single \$ character is equivalent to the scalar zero and in some expressions, this also can return an error.

A constant numeric vector is entered by typing the constant components in order, separated by one or more spaces. Constant components means that only literal numbers can be used in a vector or matrix input.

Consider now the following statement, where PI is assigned the value 3.141592 for π :

```
V<-PI PI%2 PI%3 PI%4 PI%5 PI%6 PI%8
```

The previous statement is accepted in the modern APL engines, capable of using variables names, math expressions and also vectors of vectors, but not accepted in the IBM APL\1130 (and thus not in apple), where an error is issued because, as stated earlier, numeric vectors and matrices can be input using only literal numbers: variables and mathematical expressions cannot be used.

This problem can be solved by the combined usage of the comma operator and parenthesis (see ahead); for the moment, it suffices knowing that the previous failing statement is perfectly valid in IBM APL\1130 (and thus in apple) if written in this way:

```
V<-PI, (PI%2), (PI%3), (PI%4), (PI%5), (PI%6), PI%8
```

where all math expressions (but last), due to the algebraic APL algorithm, is enclosed by parentheses. This technique is so simple and sophisticated, at the same time, to make superfluous the implementation of more modern techniques.

A characters constant (a string vector) is entered by typing the characters between single or double⁷ quotation marks, and a sequence of characters entered in quotes represents a vector whose successive components are the ASCII characters themselves. Such a vector is printed by the system as a sequence of characters, with no enclosing quotes and with no separation of the successive elements.

In case of single-quoted strings, the quote character itself must be typed in as a pair of quotes. Thus, the abbreviation of CANNOT is entered as 'CAN' 'T', and prints as CAN' T.

Single-quoted strings properties:

- A standard single-quoted string of length one is classified as a scalar. This may be a problem when the analysis is performed on strings with any length (and thus with length 1 included, which is not a string).
- Single-quoted strings cannot parse any escape sequences.

In case of double-quoted strings, the double quote character itself must be typed in as a pair of double quotes. Thus,

⁶ This is forbidden, of course...

⁷ A pure theft from GNU APL...

the string SAY "YES" AND GO is entered as "SAY ""YES"" AND GO".

Double-quoted strings properties:

- A double-quoted string of length one is always a vector. This may be useful when the analysis is performed on strings with any length (and thus with length 1 included).
- A double-quoted string can parse the following escape sequences:

✓	alarm beep	\a
✓	backspace	\b
✓	form feed	\f
✓	new line	\n
✓	carriage return	\r
✓	horizontal tab	\t
✓	vertical tab	\v
✓	void character	\0
✓	backslash	\\

The \0 sequence can be used to temporarily shorten strings by putting a zero in the middle, like in A←"TEMP\0ERATURE", which is saved as complete but, after the assignment, A will hold only TEMP.

In the case of a series of single characters, similarly to the series of scalars, a string vector is assembled, and the single characters do not need to be consistent:

```
'A' 'B' 'C'
ABC
'A' "B" 'C'
ABC
"A" "B" "C"
ABC
"A" "B" 'C'
ABC
```

A constant matrix of any type is entered via the p operator, as will be seen in further chapters, with values listed by rows in a constant vector, both for strings and numeric matrices, with the properties that were previously exposed for the constants.

Names and Spaces. The name of a variable or defined function may be any sequence of 15 characters⁸ beginning with an uppercase letter, followed by uppercase letters or digits, and not containing any blanks, lower letters or punctuation characters.

Spaces are not required between primitive functions and constants or variables, or between a succession of primitive functions, but they may be used, if desired. Spaces are needed to separate names of adjacent defined functions, constants, and variables. For example, the expression 3+4 may be entered with no spaces, but if F is a defined function that needs two arguments, then the expression 3F 4 must be entered with the indicated spaces⁹ (that is, the function name must be followed by one space). The exact number of spaces used in succession is of no importance and extra spaces may be used freely.

Variable names precede function names; this means that if a variable and a program have the same name, the name

⁸ They were six characters for the original IBM APL\1130.

⁹ The original IBM APL\1130 required one space before (if preceded by a digit or a letter) and one space after the defined function name, while **apple** requires always one space after, but a blank before causes no harm.

used in a console or program request identifies the variable, not the program, which causes the function to be unreachable at all effects; on the other hand, this implies that the command)ERASE <name> acts on the variable, not on the function so that, after such command, the program gets finally available.

Error reports. The attempt to execute an invalid statement will cause one of the error reports of Table 2.2 to be typed out. The error report will be followed by the offending statement with a caret typed under the point in the statement where the error was detected.

If an invalid statement is encountered during the execution of a defined function, the error report includes the function name and the line number of the invalid statement. The recommended procedure at this point is to enter a right arrow followed by a zero →0 (that is equivalent to ending the program) and then retry with an amended statement. The matter is treated more thoroughly in the following and in the APL69.

Type	Causes	Corrective actions
CHARACTER ERROR	Illegitimate lower letter or unrecognized operator sequence	Retype
VALUE ERROR	Use of a name that has not been assigned a value or that corresponds to no defined functions	
DOMAIN ERROR	The mathematical function does not exist in $\mathfrak{R}$ for the given argument	Check the line and the loop limits
SYNTAX ERROR	Syntax not conforming	
RANK ERROR	Rank not conformable	
LENGTH ERROR	Shape not conformable	
DEFN ERROR	1. Editing requested while the function is pendant 2. Use of more of the header than \ / F to reopen the definition F 3. Misuse of the \ / operator 4. Assignment invoked from console with an active stop control	1. Display the State Indicator)SI and clear with →0 2 and 3. Review the typed text (lower letters used into the expression?) 4. Emend the program, reset the stopping control
LABEL ERROR	Duplicate label in program, or label with a name of an existing variable (including labels in calling functions)	Rename some label or variable
SYSTEM ERROR	A memory fault in internal system of apple	Please write to <ing dot antonio dot maschio at gmail dot com>
FN SPACE FULL ERROR	Too many user functions in this workspace	Clean the workspace and remove the unneeded user functions
INDEX ERROR	Index value out of range	Check the line and the loop limits.
WS FULL ERROR	Too many variables in this workspace	Clean the workspace and remove the unneeded variables
ID ERROR	Identifier longer than 15 characters	
NOT WITH OPEN DEFINITION	usage of illegal commands in a program line	(see the system command chapter)
FN RECURSION LIMIT ERROR	Too many recursive calls to a function that seems not to converge	Assure the algorithm converges for the input values.
NON-STANDARD ASCII CHARACTER ERROR	A character beyond the Standard 0-127 range was used in the expression	Remove the alien ASCII character
AXIS ERROR	The index specified for a lamination is not a scalar decimal in the range [0.001, 0.999] or [1.001, 1.999]	Review the index

Type	Causes	Corrective actions
EDITING SYNTAX ERROR	The editing command (internal editor) enclosed in brackets contains a syntax error	Review the command
QUAD PRINT ERROR	Too much quad printing required (there is a maximum of 100 per line)	
UNBALANCED BRACKETS ERROR	The expression is not balanced for (...) or [...]	
NO PENDENT PROGRAM ERROR	The console command -> is used to continue a program which is not executing	
WARNING: CANNOT CALCULATE THIS EIGENSPACE	A warning that appears when the algorithm for the eigenspace solution cannot find any result	
WARNING: THIS EIGENSPACE HAS COMPLEX RESULTS	A warning that appears when the algorithm for the eigenspace solution determine the presence of complex values	Note: this algorithm is heuristic and, in the case of almost-zero values, it can be fooled
NON-SQUARE MATRIX ERROR	The object matrix is not square.	

Table 2.2: Error reports

Other messages, not in the errors list, are printed during console statements, when the user types something that the parser cannot understand or to confirm that some action took place. In the following table 2.2B, you can read about them, with the note that X represent a variable content, that may change from message to message:

Type	Causes	Corrective actions
DESCRIPTION ERROR	This message appears when)LOAD is called and no description string could be created in the memory space	The memory is full. Reduce the workspace
NOT FOUND X	Printed by)LIST,)TYPE,)EXPORT and)RENAME when the object file could not be found, and by the routines used by)LOAD,)SAVE,)ERASE and the internal editor in case of elimination of a variable or a program instance.	
X {NOT} IMPORTED	Printed by)IMPORT; the NOT part is printed if the file could not be imported.	
X ALREADY IMPORTED	Printed by)IMPORT in case the program is already in memory,	
X {NOT} EXPORTED	Printed by)EXPORT; the NOT part is printed if the file could not be exported	
X NOT RENAMED	Printed by)RENAME; the NOT part is printed if the file could not be imported.	
VALUE ERROR	Printed by the tracing and stopping features if a wrong line number is required	
WAS X	Printed by)EXP,)DIGIT,)ORIG,)DIV,)COMP,)FUZZ,)INV,)WIDTH,)SEED and)WSID to inform that the value has changed.	
IS X	Printed by)EXP,)DIGIT,)ORIG,)DIV,)COMP,)FUZZ,)WIDTH,)SEED and)WSID to inform that the value has not changed.	If it was not an argument-less request, probably the argument value is out of range. Review and retype
NO DESCRIPTION	Printed by)DESCRIBE if no description was found in the current workspace	

Type	Causes	Corrective actions
CHANNEL X IS OPENED IN X MODE TO FILE 'X'	Printed by) FILES to inform about the opened files	
CHANNEL X IS CLOSED.	Printed by) FILES to inform about the closed files	
NAMELESS WSID IS NEW.	Printed by) WSID in case no programs or variables are present and no name was given to the current workspace	
NAMELESS WSID IS USED.	Printed by) WSID in case programs or variables are present but no name was given to the current workspace	
COULDN'T LOAD X: CONTINUING WITH AN EMPTY WORKSPACE:	Printed by) LOAD if a problem was found during the loading of the required workspace, and to signal that the current workspace is empty; it can also appear at the start of a session if the current saved workspace is corrupted	
UNRECOGNIZED SYSTEM COMMAND	Something weird was typed, the system cannot understand	Review the syntax. Consult the manual, in case
MALFORMED TRACE	There is a mistake in the tracing command	Review the command; consult the manual
MALFORMED STOP	There is a mistake in the stopping command	Review the command; consult the manual
CANNOT OPEN FILE FOR EDITING FUNCTION X	The command) EDIT cannot open the indicated file	Check if the file exists or is in an inaccessible position.
ACTIVATING SCRIPTING ON FILE X	The command) SCRIPT started a scripting session.	
COULDN'T OPEN FILE X FOR SCRIPTING	The command) SCRIPT cannot open the scripting file	Check if the file exists or is in an inaccessible position.
X NOT A WORKSPACE	The workspace is not a regular workspace file	Check the file name.
X (NOT) LOADED	The information that the workspace could be loaded or not	In case of NOT LOADED message, the workspace was found but it's probably corrupted.
X SAVED	The information that the workspace was saved to disk	
MISSING FUNCTION NAME	Printed by) LIST if typed without argument	

Table 2.2B: Console messages

There are more error messages, due to the addition of the file input/output commands, that were not in the original APL\1130; they were conceived to be clear and self-explaining, and you can find all of them in Table 2.11 ahead in this manual.

Names of primitive functions. The primitive functions of the language are summarized in Tables 2.3A, 2.3B, 2.8A and 2.8C, and will be discussed individually in subsequent sections. The tables show one suggested name for each function. This is not intended to discourage the common mathematical practice of vocalizing a function in a variety of ways (for example, ' $X\%Y$ ' may be expressed as *I divided by Y*, or *X over Y*). Thus, the expression pM yields the dimension of the array M , but the terms *size* or *shape* may be preferred both for their brevity and for the fact that they avoid potential confusion with the dimensionality or rank of the array.

The importance of such names and synonyms diminishes with familiarity. The usual tendency is toward the use of the name of the symbol itself (e.g., *rho* (ρ) for *size*, and *iota* (ι) for *index generator*), probably to avoid unwanted connotations of any of the chosen names.

2.6. Scalar functions

Each of the primitive functions is classified as either scalar or mixed. Scalar functions are defined on scalar i.e. individual) arguments and are extended to arrays in four ways: element-by-element, reduction, inner product, and outer product, as described in the section on Functions on Arrays. Mixed functions are discussed in a later section.

IBM operator	apple operator	Purpose	Notes
+	+	set positive (optional)	
-	-	set negative	
×	x	signum function	the letter 'x'
÷	%	reciprocal	percent
⌈	}	ceiling	
⌊	{	floor	
*	*	exponential, i.e. e^x	
⊗	&	natural logarithm	ampersand
		absolute value	bar
!	!	factorial (Gamma function)	
?	?	random number generation	
○	o	π times	the letter 'o'
~	~	logical not	tilde

Table 2.3A: Monadic scalar operators

IBM operator	apple operator	Purpose	Notes
+	+	addition	
-	-	subtraction	the minus sign
×	x	multiplication	the letter 'x'
÷	%	division	percent
⌈	}	maximum	
⌊	{	minimum	
*	*	power	
⊗	&	general logarithm	ampersand
		residue	bar
!	!	binomial function	
?	?	random selection	
^	^	lcm/logical and	caret
∨	∨	gcd/logical or	the letter 'v'
^~	^~	logical not-and	caret plus tilde
∨~	∨~	logical not-or	the letter 'v' plus tilde
<	<	lesser	
≤	<=	lesser or equal	
=	=	equal	
≥	>=	greater or equal	
>	>	greater	
≠	#	not equal	grid
○	o	circular trigonometry	the letter 'o'

Table 2.3B: dyadic scalar operators

$(-A) \circ B$	A	$A \circ B$
$(1-B*2) * 0.5$	0	$(1-B*2) * 0.5$
Arc Sin B	1	Sin B
Arc Cos B	2	Cos B
Arc Tan B	3	Tan B
$(-1+B*2) * 0.5$	4	$(1+B*2) * 0.5$
Arc Sinh B	5	Sinh B
Arc Cosh B	6	Cosh B
Arc Tanh B	7	Tanh B

Table 2.3C: Circular functions

The scalar functions are summarized in Tables 2.3A and 2.3B; the circular functions in Table 2.3C. Each is defined on real numbers or, as in the case of the logical functions *and* and *or*, on some subset of them (integers). The distinction between "fixed point" and "floating point" numbers is of lesser importance, this being primarily a matter of the representation in a particular medium; the commands)DIGIT,)FUZZ and)COMP or the I-Beam variables I_5, I_2 and I_1 may be used to seize the output printing.

For functions such as *floor* and *ceiling*, and in comparisons, the system command)FUZZ can be used to apply a "fuzz" value in order to avoid anomalous results that might otherwise be engendered by doing decimal arithmetic on a binary machine. Besides, these functions are defined on characters as well as on numbers, but not mixed (i.e. you cannot 'sum' a number and a string).

The logical operators $\wedge$ and $\vee$, if used with the logical values 1 and 0 (zero), operate according to their respective truth-table; if used with any other value (integer or real) they work respectively as the Least Common Multiple function ($\wedge$) and the Greatest Common Divisor ($\vee$).¹⁰ The decimal values that can be used are those in the unsigned range $1 \times 10^{-8} \div 1 \times 10^8$; **apple** follows the Dyalog APL rules for these, returning gcm as an unsigned real, regardless of the sign of the operands, and lcm as a signed real.

The logical co-operators $\wedge\sim$ (nand) and $\vee\sim$ (nor), instead, work exclusively with the logical values 1 and 0; any other integer or real number will cause an error.

Monadic and dyadic functions. Each of the functions defined in Tables 2.3A, 2.3B and 2.3C may be used in the same manner as the familiar arithmetic functions $-$ $\times$ and $+$. Most of the symbols employed may denote either a monadic function (which takes one argument) or a dyadic function (which takes two arguments). For example, }Y denotes the monadic function ceiling applied to the single argument Y, and X}Y denotes the dyadic function maximum applied to the two arguments X and Y. A monadic symbol is expected whenever a number is expected in a given expression, while a dyadic symbol is expected whenever an operator, and not a number, is expected. For example:

23×45
1035

In this tiny expression, 23 starts the expression, and a number is expected here. Then an operator is expected, thus $\times$ is used in dyadic sense. Then the last number is in a place where a number is expected, and the expression can be calculated. In the following example, instead:

23××45
23

23 starts the expression (as in the previous case); then the first $\times$ operator is in a place where an operator is expected, so it's used in dyadic sense. Then the second $\times$ is not in a place where an operator is expected, but a number, so it's used in monadic sense, associated with the number that follows. In practice it acts as:

¹⁰ This characteristic is also formalized in the ISO standard; now, I don't know if it was present - but not discussed - in the original APL1130 version... but I think not.

23 x (x45)
23

At this point it may be helpful to scrutinize each of the functions of Tables 2.3A, 2.3B and 2.3C and to work out some examples of each, either by hand or on a terminal. However, it's not essential to grasp all of the more advanced mathematical functions in order to proceed. Treatments of these functions are readily available in standard texts.

Certain of the scalar functions deserve brief comment. The residue function¹¹ $A|B$ has the usual definition of residue used in number theory. For positive integer arguments this is equivalent to the remainder obtained on dividing B by A, and may be stated more generally as the smallest non-negative member of the set $B - N \times A$, where N is any integer. This formulation covers the case of a zero left argument. The conventional definition is extended in two further respects:

- 1 The left argument A need not be positive; however, the value of the result depends only on the magnitude of A. Anyway, an error message is raised in case A is null.
- 2 The arguments need not be integral. For example, $1|2.6$ is 0.6 and $1.5|8$ is 0.5.

In the IBM APL\1130, the domain of the ? function is limited to positive integer arguments less than 32768. **apple** gently uses any integer value, whose module 32768 is taken, without errors. Moreover if the argument is zero, a real number in the range]0.0, 1.0[is returned.

The factorial function !N is defined in the usual way as the product of the first N positive integers. The function A!B (pronounced *A out of B*) is defined as $(!B) \% (!A) \times !B - A$ and is the number of combinations of B things taken A at a time.

The symbols < <= = >= > and # denote the relations less than, less than or equal, etc., in the usual manner. However, an expression of the form A<B is treated not as an assertion, but as a function which yields a 1 if the proposition is true, and 0 (zero) if it's false. For example:

3<=7
1
7<=3
0

When applied to logical arguments (i.e., arguments whose values are limited to 0 and 1), the six relations are equivalent to six of the logical functions of two arguments. For example, <= is equivalent to material implication, and # is equivalent to exclusive-or. These six functions together with the *and*, *or*, *nand*, and *nor* shown in Tables 2.3A, 2.3B and 2.3C exhaust the nontrivial logical functions of two logical arguments, when they are used with zero as a false value, and 1 as a true value.

Vectors. Each of the monadic functions of Tables 2.3A, 2.3B and 2.3C applies to a vector, element by element. Each of the dyadic functions applies element by element to a pair of vectors of equal dimension or to one scalar (or a single element vector or matrix) and a vector of any dimension, the scalar being used with each component of the vector. For example:

1 2 3 4x4 3 2 1
4 6 6 4
2+1 2 3 4
3 4 5 6
1 2 3 4}2
2 2 3 4

¹¹ In the original APL\1130 this was not available when the left argument is zero and the right argument is lesser than zero. **apple** follows the most modern APL evolution (also in the Dyalog APL, for instance), that accepts even this case and treats all the numerical cases.

Index generator. If N is a non-negative integer, then iN denotes a vector of the first N integers. The dimension of the vector iN is therefore N ; in particular, $i1$ is a vector of length one which has the value 1, and $i0$ is a vector of length zero, also called an *empty vector*. The empty vector prints as a blank. For example:

```

      i4
1  2  3  4
      i5
1  2  3  4  5
      i0
                                Empty vector prints as a blank
      6-i6
5  4  3  2  1  0
      2xi0                      Scalar applies to all (i.e., 0) elements
                                of i0, resulting in an empty vector
      2xi6
2  4  6  3  10  12

```

The index generator is one of the class of mixed functions to be treated in detail later; it's included here because it's useful in examples.

2.7. Rectangular arrays

Introduction. A single element of a rectangular array can be selected by specifying its indices; the number of indices required is called the dimensionality or rank of the array. Thus a vector is of rank 1, a matrix (in which the first index selects a row and the second a column) is of rank 2, and a scalar (since it permits no selection by indices) is an array of rank 0. In the IBM APL1130, arrays of rank greater than rank 2 couldn't be used and no dimension of an array could exceed 256; thus, a vector could have no more than 256 elements and a matrix no more than 256 rows or columns. **apple** has maintained the limit of rank 2 for matrices (that is no matrices of order greater than 2 can be created) but has removed the limit of 256 objects for vectors and matrices. The limit's your computer memory and your computation speed, but remember that to elaborate the data for huge matrices or vectors, you'd need also a fast computer and much much RAM.

Vectors dimension. If X is a vector, then pX denotes its dimension. For example, if $X \leftarrow -2 \ 3 \ 5 \ 7 \ 11$, then pX is 5, and if $Y \leftarrow 'ABC'$ then pY is 3. A single character entered in quotes or in response to a `|'` input is a scalar, not a vector of dimension 1; this parallels the case of a single number, which is also a scalar.

Catenation. Catenation chains two vectors (or scalars) together to form a vector along an existing axis; it's denoted by a comma. For example:

```

      X ← -2 3 5 7 11
      X, X
2  3  5  7  11  2  3  5  7  11

```

In general, the dimension of X, Y is equal to the total number of elements in X and Y .

apple has extended the possibility to catenate matrices as well, provided they have the same dimension on the chosen direction (that is selectable with $[D]$, where $D=1$ means row catenation, and $D=2$ means column catenation). For example:

```

      X ← -2 3pi6
      Y ← -2 4 p12-i12
      Y
11  10  9  8
 7  6  5  4

```

```

      X
1  2  3
4  5  6
      X, Y
1  2  3  11  10  9  8
4  5  6  7   6  5  4
      X, [2] Y
1  2  3  11  10  9  8
4  5  6  7   6  5  4

```

Conformability requires that in case of column catenation, the number of rows is the same for all matrices, and in case of row catenation, the number of columns is the same for all matrices.

Note: the index is ignored when the operation of catenation does not involve matrices.

Catenation of strings and values was forbidden in the IBM APL\1130 (see for instance the example at page A.6 of the DEC70), and returned a domain error; **apple** accepts instead these catenation types, with the law that the final type of the catenation is the type of the last added item (which is the first to be interpreted); for instance:

```

      'BE', 65, 'TLES'
BEATLES
      'BE', 65, 'TLE', 83
66  69  65  84  76  69  83

```

Lamination. Differently from the concatenation performed by the comma operator, the lamination, using the same comma operator, is a procedure that glues together two items forming an object along a new axis, specified by a decimal index¹² in the range [0.001, 0.999] for the row axis (and the two arguments are piled, each on its own row), in the range [1.001, 1.999] for the columns axis (and the two arguments are lined one beside the other, each in its own column).¹³ Typical values, avoiding cumbersome and weird decimals like 0.269 or 1.478 decimals, even if legal, are of course 0.5 and 1.5, which are also used in the following examples.

For instance, the comma operator with two vectors creates a vector which is the sum of the two (maintaining the axis):

```

      1 2 3 4, 5 6 7 8
1  2  3  4  5  6  7  8

```

The same procedure with the laminate index is, in case of the row axis:

```

      1 2 3 4, [0.5] 5 6 7 8
1  2  3  4
5  6  7  8

```

or, in case of the column axis:

```

      1 2 3 4, [1.5] 5 6 7 8
1  5
2  6
3  7
4  8

```

¹² Not in the original APL\1130.

¹³ It does not seem useless underlying the fact that without the decimal index the catenation is performed, and not the lamination.

In both cases, two vectors originate a matrix. The more advanced APL engines, which can deal with undetermined limits in the dimension number of an array, can manage lamination by generating an object which is the sum of the arguments dimensions, but this is not possible for **apple**. That's why **apple** cannot use matrices here.

The case of laminating two scalars extends the concept of the comma operator¹⁴; for instance, the classic comma behaviour is:

```

      2, 4
2    4
      2, [1] 4
2    4
      1, [2] 4
2    4

```

because the index is ignored for scalars and vectors. The lamination, instead, leads to

```

      2, [0.5] 4
2
4
      1, [1.5] 4
2    4

```

in perfect accordance with the vector behaviour. Thus, two scalars originate a vector or a one-column matrix.

The conformability of the lamination introduced by **apple** requires that in case of scalars to vectors, scalars are extended to vectors, in case of vectors to vectors, they must have the same dimension, or a length error is generated. In case a matrix is used, in any association, a rank error is generated.

Matrices dimension. The monadic function **p** applied to an array **A** yields the size of **A**, that is, a vector whose components are the dimensions of **A**. For example, if **A** is the matrix

```

1    2    3    4
5    6    7    8
9    10   11   12

```

of three rows and four columns, then **pA** is the vector 3 4; consequently, a fast way to calculate the dimension of a matrix is by using the multiplication reduction operator over **pA**, so that **x/pA** is 12.

Since **pA** contains one component for each coordinate of **A**, the expression **ppA** is the rank of **A**. Table 2.4 illustrates the values of **pA** and **ppA** for arrays of rank (scalars) up to rank 2. In particular, the function **p** applied to a scalar yields an empty vector, and applied to a vector yields a one-value vector, not a scalar.

Type of Array	pA	ppA	pppA
Scalar		0	1
Vector	N	1	1
Matrix	M N	2	1

Table 2.4: Dimension and rank vectors

Ravel. The monadic function **ravel** is denoted by a comma; when applied to any array **A** it produces a vector whose elements are the elements of **A** in row order. For example, if **A** is the matrix

¹⁴ The GNU APL does not interpret scalars in lamination, while the Dyalog APL accepts only the index [0.5] and returns a vector.

```

      2   4   6   8
    10  12  14  16
    18  20  22  24

```

and if $V < -$, A then V is a vector of dimension 12 whose elements are the integers 2 4 6 8 10 12 ... 24. If A is a vector, then $, A$ is equivalent to A ; if A is a scalar, then $, A$ is a vector of dimension 1.

Reshape. The dyadic function p reshapes its right argument to the dimension specified by its left argument. If $M < -DpV$, then M is an array of dimension pD whose elements are the elements of V . For example, $2\ 3p1\ 2\ 3\ 4\ 5\ 6$ is the matrix

```

  1  2  3
  4  5  6

```

If N , the total number of elements required in the array DpV , is equal to the dimension of the vector V , then the ravel of DpV is equal to V . If N is less than pV , then only the first N elements of V are used; if N is greater than pV , then the elements of V are repeated cyclically. For example, $2\ 3p1\ 2$ is the matrix

```

  1  2  1
  2  1  2

```

and $3\ 3p1\ 0\ 0\ 0$ is the identity matrix

```

  1  0  0
  0  1  0
  0  0  1

```

More generally, if A is any array, then DpA is equivalent to Dp, A . For example, if A is the matrix

```

  1  2  3
  4  5  6

```

then $3\ 5pA$ is the matrix

```

  1  2  3  4  5
  6  1  2  3  4
  5  6  1  2  3

```

The expressions $0pX$ and $0\ 3pX$ and $3\ 0pX$ and $0\ 0pX$ are all valid; any one or more of the dimensions of an array may be zero.

Note: if the left argument is the empty array, then this forces the reshape operator to return as a scalar the first element of the right argument; if both the left and right argument are the empty array, the scalar zero is returned¹⁵.

Uses of empty arrays. A vector of dimension zero contains no components and is called an *empty vector*. Three expressions which yield empty vectors are $\mathbf{i}0$ and $\mathbf{'}$ and p applied to any scalar. An empty vector prints as a blank line. It's illegal using selectors with empty vectors.

One important use of the empty vector is that when one occurs as the argument of a branch, the effect is to continue the normal sequence.

¹⁵ These properties are not described in the APL1130 or APL360 manuals, but a hint to solve this issue is in the APL3000 Reference manual, at page 3-32, where it's stated: "If A is the numeric empty vector, then ApB is a scalar containing the first element of ravel B "; this can be justified by the fact that the request $p4$ returns the empty vector, that is to say that the empty vector *defines* the scalar entity, taken from the first item of the right element's ravel list. When also this element is an empty vector, it seems reasonable to adopt zero as the scalar to be used.

Empty arrays of higher rank can be useful in analogous ways in conjunction with the expansion function described in the section on Mixed Functions.

If A is a 3-by-0 matrix and B is a 0-by-3 matrix, then $A \cdot B$ ($A+.xB$ in APL coding) is the 3-by-3 zero matrix corresponding to the null map from a 3-dimensional space V to itself, while $B \cdot A$ ($B+.xB$ in APL coding) is a 0-by-0 matrix (the *empty matrix*).

A matrix with one or both of the dimensions zero (e.g. $3 \ 0pX$) is an *empty matrix*. As with empty vectors, empty matrices help in dealing with maps involving the zero vector space¹⁶. Developing the former examples in an *apple* session:

<pre>A<-3 0p19 A</pre>	<pre>An empty matrix prints as a void line</pre>
<pre>B<-0 3p19 B</pre>	<pre>Another empty matrix</pre>
<pre>C<-A+.xB C</pre>	<pre>Pre-multiplying B with A returns a 3x3 regular matrix</pre>
<pre>0 0 0 0 0 0 0 0 0</pre>	<pre>C is the null matrix</pre>
<pre>D<-B+.xA D</pre>	<pre>Post-multiplying B with A returns a 0x0 empty matrix which prints as a void line</pre>

As with empty vectors, when an empty matrix occurs as the argument of a branch, the effect is to continue the normal sequence.

Indexing. The `[]` square brackets constitute one special operator that acts on the left argument only, leaving in its place the sub-array specified by what is inside.

Thus, if X is a vector and I is a scalar, then $X[I]$ denotes the I -th element of X . For example, if $X \leftarrow -2 \ 3 \ 5 \ 7 \ 11$ then $X[2]$ is 3.

If the index I is a vector, then $X[I]$ is the vector obtained by selecting from X the elements indicated by successive components of I . For example, $X[1 \ 3 \ 5]$ is $2 \ 5 \ 11$ and $X[5 \ 4 \ 3 \ 2 \ 1]$ is $11 \ 7 \ 5 \ 3 \ 2$ and $X[i3]$ is $2 \ 3 \ 5$. If at least one among the elements of I does not belong to the set of indices of X , then the expression $X[I]$ induces an index error report.

In general, $pX[I]$ is equal to pI . In particular, if I is a scalar, then $X[I]$ is a scalar, and if I is a matrix, then $X[I]$ is a matrix. For example:

```
A<-'ABCDEFGG'
M<-4 3p3 1 4 2 1 4 4 1 2 4 1 4
M
3 1 4
2 1 4
4 1 2
4 1 4
A[M]
```

¹⁶ See also [https://en.wikipedia.org/wiki/Matrix_\(mathematics\)#Empty_matrix](https://en.wikipedia.org/wiki/Matrix_(mathematics)#Empty_matrix) for some more advanced information.

CAD
BAD
DAB
DAD

Each element of M identify a position in the string A ; if a value beyond the string width is used, an index error is printed.

If M is a matrix, then M is indexed by a two-part list of the form $I; J$ where I selects the row (or rows) and J selects the column (or columns). For example, if M is the matrix

1	2	3	4
5	6	7	8
9	10	11	12

then $M[2; 3]$ is the element 7 and $M[1\ 3; 2\ 3\ 4]$ is the matrix

2	3	4
10	11	12

In general, $pM[I; J]$ is equal to $(pI), pJ$. Hence if I and J are both vectors, then $M[I; J]$ is a matrix; if both I and J are scalars, $M[J]$ is a scalar; if I is a vector and J is a scalar (or vice versa), $M[I; J]$ is a vector, and if I is a matrix and J is a scalar (or vice versa), $M[I; J]$ is a matrix.

The form $M[I;]$ indicates that all columns are selected, and the form $M[; J]$ indicates that all rows are selected. For example, $M[2;]$ is 5 6 7 8 and $M[; 2\ 1]$ is

2	1
6	5
10	9

If M is a matrix and the index inside $[]$ is a simple scalar or a simple vector (indicating respectively one item and multiple items on one dimension), then an index error is generated because it's impossible to know which dimension to take into account.

The various possible indexing cases are exposed in the following table 2.5, where I is the left index and J is the right index (where present); the cases that lead to the rank error are also exposed:

Type of Array	I	J	Result
Scalar	<i>not empty</i>	<i>not empty</i>	rank error
Vector	Scalar		Scalar
	Vector		Vector
	Matrix		Matrix
	Any	<i>not empty</i>	rank error
Matrix	Scalar	Scalar	Scalar
	Vector	Scalar	Vector
	Scalar	Vector	Vector
	Vector	Vector	Matrix
	Matrix	Scalar	Matrix
	Scalar	Matrix	Matrix
	Matrix	Matrix	rank error
	Any	<i>empty</i>	rank error

Table 2.5: Indexing cases for arrays with $[]$

Permutations. Permutations are an interesting use of indexing. A vector P whose elements are some permutation of its own indices is called a permutation of order pP . For example, 3 1 4 2 is a permutation of order 4. If X is any vector of the same dimension as P , then $X[P]$ produces a permutation of X . Moreover, if pP is equal to $(pM)[1]$, then $M[P;]$ interchanges the rows of M and is called a rows permutation. Similarly, if pP equals $(pM)[2]$, then $M[:,P]$ is a columns permutation of M .

Indexing on the left. An array appearing to the left of a specification arrow may be indexed, in which case only the selected positions are affected by the specification. For example:

```
X<-2 3 5 7 11
X[1 3]<-6 8
X
6 3 8 7 11
```

Matrix single elements may be assigned if in $M[I;J]$, both I and J are scalars:

```
M<-3 3pi9
M
1 2 3
4 5 6
7 8 9
M[2;2]<-0
M
1 2 3
4 0 6
7 8 9
```

If I is a scalar and J is a vector, then the assign value must be a vector with the same dimensions of J and the correspondent values of the row indicated by I will be re-assigned. If I is a vector and J is a scalar, then the assign value must be a vector with the same dimensions of I and the correspondent values of the column indicated by J will be re-assigned. All other cases return a rank error. Continuing the previous example:

```
M[2;3 1]<-4 6
M
1 2 3
6 0 4
7 8 9
M[1 3;3]<-_1 _2
M
1 2 _1
6 0 4
7 8 _2
```

If both I and J are vectors, then the assign value must be a matrix with pI rows and pJ columns, whose values will be changed accordingly. For example:

```
M<-3 3pi9
M
1 2 3
```

```

4 5 6
7 8 9
  M[1 2;2 3]<-2 2p(i4)x18
  M

1 18 36
4 54 72
7 8 9

```

The normal restrictions on indexing apply in all cases; in particular, a variable which has not already been assigned a value cannot be indexed, and an out-of-range index value cannot be used.

In particular, it's understood that an empty value instructs APL to return all the values in the specific axis; so, if V is any vector, the figure $V[]$ is equivalent to the whole vector V , regardless of its dimension. If M is any matrix, the same figure $M[]$ returns a rank error, because one empty value is not assignable to two dimensions, and to return the whole matrix, the figure $M[;]$ must be used, where an empty value is assigned to each dimension.

Index origin¹⁷. In 1-origin indexing, $X[1]$ is the leading element of the vector X and $X[pX]$ is the last element. In 0-origin indexing, $X[0]$ is the leading element and $X[_1+pX]$ is the last. 0-origin indexing is instituted by the command `)ORIGIN 0` (zero). The command `)ORIGIN 1` restores the 1-origin indexing. The index origin in effect applies to all coordinates of the all rectangular arrays.

In certain expressions like $+/[J]M$ or $K\circ|[J]M$, the value of J determines the coordinate (direction) of the array M along which the function is to be applied; so that the range in 1-origin is $[1, 2]$, and in 0-origin is $[0, 1]$.

The index origin affects the following functions: the monadic and the dyadic forms of $?$ (roll and deal) and i (index generation and index of), the monadic and dyadic forms of $/^{\wedge}\backslash$ and $\backslash v/$ (grade up and down), the monadic and dyadic forms of $\circ|$, the dyadic compression and expand operators $/$, $/-$, $\backslash$ and $\backslash-$, the reduction and scan operators $f/$ and $f\backslash$ (where f is a scalar operator).

A different remark is reserved for the comma operator, because it has also decimal numbers as axis for the laminate case. Even in such cases, the indices are influenced by the origin, so that in 1-origin the limits for catenation are $[1,2]$ and in 0-origin are $[0,1]$, and the limits for lamination are $[0.001,1.999]$ for 1-origin, and $[-0.999,0.999]$ for 0-origin.

Some commands use the $[. .]$ figure as selector, and are not influenced by `)ORIGIN`; they are: the file open/close operator $|v|$, the write to file operator $|>|$ and the ranking/echelon operator r .

The index origin remains associated with the workspace and, if saved, is maintained in the next session. A clean workspace provided on sign-on or via the `)CLEAR` command is in 1-origin.

All examples in this manual are in 1-origin, and only when preceded by the command `)ORIGIN 0` they are intended to work in 0-origin.

Array output. Character arrays print with no spaces between components in each row; numeric arrays print numbers with two spaces in between. If a vector or a row of a matrix requires more than one line, succeeding lines are indented.

A matrix prints with all columns aligned to the right and with a blank line before the first row. A matrix of dimension $1, N$ (1 row and N columns) prints as a vector, but retains its state of matrix. A matrix of dimension $N, 1$ (N rows and one column) prints as a vertical vector; in both cases, a blank is printed before. Examples:

```
M<-5 1pi5
```

¹⁷ This command in effect was not on the APL69, but was present in the APL70, and here replicated, with minor differences.

```

M
1
2
3
4
5
N<-1 5pi5
N
1 2 3 4 5

```

Empty vectors and empty matrices print as a blank line.

The default printing precision, with `)DIGIT = 0`, uses 6 scaled digits at most, and values are printed according to their state (integer, decimal, exponential). In case `)DIGIT` is set to a value different than zero, the required printing precision is used to picture decimals

2.8. Functions on arrays

There are five ways¹⁸ in which the scalar functions of Table 2.3A, 2.3B and 2.3C extend to arrays: element-by-element, reduction, scansion, inner product, and outer product. Reduction, scansion and outer product are defined on any arrays, and the other two extensions are defined only on arrays whose sizes satisfy a certain relationship called conformability. For the element-by-element extension, conformability requires that the shapes of the arrays agree, unless one of them comprises only a single element. The requirements for inner product are shown in Table 2.7.

Element-by-element extension. All of the scalar functions of Tables 2.9A and 2.9B are extended to arrays element by element. Thus if M and N are matrices of the same size, f is a scalar function, and $P \leftarrow M f N$, then $P[I; J]$ equals $M[I; J] f N[I; J]$, and if $Q \leftarrow f N$, then $Q[I; J]$ is equal to $f N[I; J]$.

If M and N are not of the same size, then $M f N$ is undefined (and induces a length or rank error report) unless one or other of M and N is a scalar or one-element array, in which case the single element is applied to each element of the other argument. In particular, a scalar versus an empty array produces an empty array.

An expression or function definition which employs only scalar functions and scalar constants extends to arrays like a scalar function.

Reduction. The sum-reduction of a vector X is denoted by $+ / X$ and defined as the sum of all components of X . More generally, for any scalar dyadic function f , the expression f / X is equivalent to $X[1] f X[2] f X[3] f \dots f X[pX]$, where evaluation is from rightmost to leftmost as usual. The request:

```

N<-5 4 3 2 1
- / N
3

```

and corresponds to the equivalent request:

```

1 - (2 - (3 - (4 - 5)))
3

```

A user-defined function cannot be used in reduction. If X is a zero-dimensional vector or a matrix with one or two zero dimensions, then f / X yields the identity number of the function f (listed in Table 2.6) if it exists; if X is a scalar or a vector of dimension 1, then f / X yields the value of the single element of X . The result of reducing any vector or scalar is a scalar.

¹⁸ Was four for the APL1130, because it lacked the scansion operator.

Dyadic Function		Identity Number	Left-Right
Times	x	1	L R
Plus	+	0	L R
Divide	÷	1	R
Minus	-	0	R
Power	*	1	R
Maximum	}	_1.79769E+308	L R
Minimum	{	1.79769E+308	L R
Residue		0	L
Out of	!	1	L
Or	∨	0	L R
And	^	1	L R
Equal	=	1	L R
Not equal	#	1	L R
Greater	>	0	R
Not less	>=	1	R
Less	<	0	L
Not greater	<=	1	L

Table 2.6: Identity numbers for scalar dyadic functions
The identity numbers of = # > >= < <= apply for logical arguments only

The circular function used in a reduction returns probably useless values, because it's not a function that operates with two regular arguments, like the addition. The operators that are profitably meaningful in a reduction are only those contained in Table 2.6.

For a matrix M, reduction can proceed along the first coordinate (denoted by $f / -M$ or along the second coordinate (f / M). The result in either case is a vector; in general, reduction applied to any non-scalar array A produces a result of rank one less than the rank of A (hence the term reduction. Since $+ / -M$ scans over the row index of M it sums each column vector of M, and $+ / M$ sums the row vectors of M. For example, if M is the matrix

```

1  2  3
4  5  6

```

then $+ / -M$ is 5 7 9 and $+ / M$ is 6 15.

The direction can be specified also by means of indices¹⁹, with the trivial rule that the index 1 identifies the first coordinate, and 2 the second, enabling the following correspondences:

```

/- = / [1] = /- [1]
/ = / [2] = /- [2]

```

Using indices, the couple of operators $/$ and $/-$ are equivalent, and indices prevail.

Scansion. The monadic scan $f \backslash$ operator is used to derive partial results in calculating the reduction of an array. For example, if X is a vector, the expression $+\backslash X$ produces a vector of the partial sums of X, as in the following example:

```

      +\3 4 5
3  7  12

```

¹⁹ I enabled this feature, not available in the original IBM APL1130, because in the IBM APL360 (on the DEC70) this is the main rule for using the reduction (given the fact it can use more than two dimensions), and the two operators $/$ and $/-$ together can represent only two dimensions.

Each element of the resulting vector can be considered a reduction of the original vector up to that point. In the resulting vector, the first element is always identical to the first element of the original vector, and the last element is equivalent to a reduction of the entire original vector.

The general requirement for the scan operator is that the function f to the left of the scan symbol ($\backslash$) is a scalar dyadic function (see Table 2.9B).

More examples:

M			
1	2	3	4
3	2	4	6
5	3	2	4
1	5	4	2
+ \ M			
1	3	6	10
3	5	9	15
5	8	10	14
1	6	10	12
- \ M			
1	-1	2	-2
3	1	5	-1
5	2	4	0
1	-4	0	-2
^ \ M			
1	2	6	12
3	6	12	12
5	15	30	60
1	5	20	20

The scanion in matrices is performed by rows. To scan by columns, use the $f \backslash -$

+ \ - M			
1	2	3	4
4	4	7	10
9	7	9	14
10	12	13	16

The scan operator cannot be used on string items, and in case a domain error is returned.

Inner product. The familiar matrix product is denoted by $C \leftarrow A + . \times B$. If A and B are matrices, then C is a matrix such that $C[I; J]$ is equal to $+ / A[I;] \times B[; J]$. A similar definition applies to $A f . g B$ where f and g are any of the standard scalar dyadic functions.

If A is a vector and B is a matrix, then C is a vector such that $C[J]$ is equal to $+ / A \times B[; J]$. If B is a vector and A is a matrix, then C is a vector such that $C[I]$ is equal to $+ / A[I;] \times B$. If both A and B are vectors, then $A + . \times B$ is the scalar $+ / A \times B$.

The last dimension of the pre-multiplier A must equal the first dimension of the post-multiplier B, except that if

either argument is a scalar, it's extended in the usual way. For non-scalar arguments, the dimension of the result is equal to (using the drop symbol) $(_1 \setminus \setminus / pA), 1 \setminus \setminus / pB$. (See the function drop in the section on Mixed Functions.) In other words, the dimension of the result is equal to $(pA), pB$ except for the two inner dimensions (using the take symbol) $(_1 / \setminus pA$ and $1 / \setminus pB)$, which must agree and which are eliminated by the reduction over them.

Definitions for various cases are shown in Table 2.7.

pA	pB	pAf.gB	Conformability requirements	Definition $Z \leftarrow Af.gB$
	V			$Z \leftarrow f / AgB$
U	V			$Z \leftarrow f / AgB$
U	V		$U = V$	$Z \leftarrow f / AgB$
	V W	W		$Z[I] \leftarrow f / AgB[; I]$
T U		T		$Z[I] \leftarrow f / A[I;] gB$
U	V W	W	$U = V$	$Z[I] \leftarrow f / AgB[; I]$
T U	V	T	$U = V$	$Z[I] \leftarrow f / A[I;] gB$
T U	V W	T W	$U = V$	$Z[I; J] \leftarrow f / A[I;] gB[; J]$

Table 2.7: Conformability requirements for scalar functions f and g

The circular scalar operator uses the left argument (in the range $_7 . . 7$) as the chooser of the appropriate function, which uses the right argument as input. In case of the inner product, the vector on the left contains values in the range $_7 . . 7$ for choosing the appropriate functions to be used with the right vector elements, in order.

Outer product. The outer product of two vectors X and Y with respect to a standard scalar dyadic function g is denoted by $X @ . g Y$ and yields an array of dimension $(pX), pY$, formed by applying g to every pair of components of X and Y .

If X and Y are vectors and $Z \leftarrow X @ . g Y$, then $Z[I; J]$ is equal to $X[I] g Y[J]$. For example:

```

X ← -i 3
Y ← -i 4
X @ . x Y

1 2 3 4
2 4 6 8
3 6 9 12
X @ . >= Y

1 0 0 0
1 1 0 0
1 1 1 0

```

Definitions for various cases are shown in Table 2.8.

pA	pB	pA@.gB	Definition
			$Z \leftarrow A @ . g B$
	V	V	$Z \leftarrow A g B$
U		U	$Z[I] \leftarrow A g B[I]$
U	V	U V	$Z[I] \leftarrow A[I] g B$
U	V W	U V W	$Z[I;J] \leftarrow A[I] g B[J]$
T U		T U	$Z[I;J] \leftarrow A g B[I;J]$
			$Z[I;J] \leftarrow A[I;J] g B$

Table 2.8: Outer products for scalar dyadic function g

2.9. Mixed functions

Introduction. The scalar functions listed in Table 2.3A, 2.3B and 2.3C each take a scalar argument (or arguments) and yield a scalar result; each is also extended element by element to arrays. The mixed functions of Table 2.9A and 2.9B, on the other hand, may be defined on vector/matrix arguments to yield a scalar result or a vector/matrix result, or may be defined on scalar arguments to yield a vector/matrix result.

IBM operator	apple operator	Purpose	Notes
ρ	p	return entity shape	letter 'p'
,	,	ravel (vectorizing)	
ι	i	build indices vector	letter 'i'
ϕ	o	matrix rows reversal	letter 'o' plus bar
θ	o-	matrix columns reversal	letter 'o' plus dash
∂	\o	matrix transpose	backslash plus letter 'o'
$f/$	f/	f columns reduction	the slash character
$f\prime$	f/-	f row reduction	slash-dash
Δ	/^\	grade up	caret amid slash up
∇	\v/	grade down	letter 'v' amid slash down

Table 2.9A: Mixed monadic functions

IBM operator	apple operator	Purpose	Notes
ρ	p	restructure	letter 'p'
,	,	catenate	
ι	i	items location	letter 'i'
/	/	logical compression along last dimension	the slash character
$\prime$	/-	logical compression along first dimension	slash-dash
\	\	logical expansion along last dimension	the backslash character
$\backslash$	\-	logical expansion along first dimension	backslash-dash
ϕ	o	matrix rotation along last dimension	letter 'o' plus bar
θ	o-	matrix rotation along first dimension	letter 'o' plus dash
∂	\o	matrix transpose	backslash plus letter 'o'
$\in$	e	membership	letter 'e'
T	\ '	encode representation from value	bar between inverse ticks
$\perp$	_ _	decode value representation	bar between underscores

IBM operator	apple operator	Purpose	Notes
$f \cdot g$	$f \cdot g$	inner product	two scalar operators enclose a dot
$\circ \cdot g$	$@ \cdot g$	outer vector product	the 'at' sign plus dot
$\uparrow$	$/ \backslash$	take	bar amid slash up
$\downarrow$	$\backslash /$	drop	bar amid slash down
$\leftarrow$	$< -$	specification (assignment)	lesser than plus dash

Table 2.9B: Mixed dyadic functions

apple adds to the classic APL\1130 operators a number of operators derived from other APL engines or from the ISO standard, or also from some diverse needs.

These added commands, all activated by the usage of special non-character symbols or lower-characters, are listed below:

ISO operator	apple operator	Purpose	Notes
$\boxminus$	$ \% $	matrix inverse	percent amid vertical bars
$f \backslash$	$f \backslash$	scan row operator	the backslash operator
$f \backslash -$	$f \backslash -$	scan column operator	backslash-dash
$\#$	$\#$	unique mask	the 'grid' character
$< \text{absent} >$	c	type conversion	letter 'c'
ϕ	e	execute string	letter 'e'
$< \text{absent} >$	m	memory access for user variables	letter 'm'
$< \text{absent} >$	n	check for numeric entity	letter 'n'
$< \text{absent} >$	t	timing operator	letter 't'
$\cup$	u	unique	letter 'u'

Table 2.9C: Added mixed monadic functions

ISO operator	apple operator	Purpose	Notes
$\boxdiv$	$ \% $	matrix division	percent amid vertical bars
Φ	f	print formatted operator	letter 'f'
$\sim$	$\sim$	exclusion set	the tilde character
$\equiv$	$= _$	match	equal plus underscore
$\Uparrow$	$/ ^ \backslash$	relative grade up	caret amid slash up
$\Downarrow$	$\backslash v /$	relative grade down	letter 'v' amid slash down

Table 2.9D: Added Mixed dyadic functions

$(-A) \circ B$	A	$A \circ B$
Complementary error function	8	Error function
Natural logarithm of the gamma function	9	Gamma function
	10	Nearest integer

Table 2.9E: Added circular functions

Specification ($< -$) may (like any other function) occur repeatedly in a single statement. For example, the execution of the statement $Z < - X \times A < - 3$ will assign to A the value 3, then multiply this assigned value of A by X and finally assign the resulting value to Z.

Multiple specification is useful for initializing variables. For example:

$$X < -Y < -1 + Z < -0$$

sets X and Y to 1 and Z to zero. Verifying:

```

      Z
0
      Y
1
      X
1

```

Transpose. The expression $\backslash\circ A$, used in monadic contexts, yields the array A with the coordinates interchanged. For a vector V and a matrix M, the following relations hold:

$\backslash\circ V$ is the vertical vector (one column-matrix) of V
 $\backslash\circ M$ is the ordinary matrix transposition, that is, the rows of M are the columns of $\backslash\circ M$ and vice versa.

Examples:

```

      M
1  2  3  4
5  6  7  8
      \circ M
1  5
2  6
3  7
4  8
      \circ \circ M
1  2  3  4
5  6  7  8

```

Applied to a vector, will yield the vertical vector:

```

      V
1  2  3
      \circ V
1
2
3

```

The expression $V\backslash\circ A$ used in dyadic mode, yields instead the array A with the coordinates changed according to the vector V, which is a 2-values vector (comprising elements having only the values 1 and 2, where 1 identifies the rows dimension and 2 the columns dimension), according to the following rules:

- If the left argument is 1 2, this instructs **apple** not to change anything, because the order in dimensions (rows, columns) is respected, with 1 at the first place and 2 at the second place. This is equivalent to requiring the original matrix.
- If the left argument is 2 1, this instructs **apple** to transpose rows and columns, because the order in dimensions (rows, columns) is reversed, with 2 at the first place and 1 at the second place. This is equivalent to the

application of the monadic `\o`.

- If the left argument is `1 1`, this instructs **apple** to collect the values in the main diagonal (from left to right, from up downward) and return them in a vector, with dimension equal to the least dimension of A.
- If the left argument is `2 2`, this instructs **apple** to collect the values in the opposite diagonal (from right to left, from up downward) and return them in a vector, again with dimension equal to the least dimension of A.
- Any value other than 1 and/or 2 results in an index error, and any other vector length of V results in a length error.

Examples:

```

      M
1  2  3  4
5  6  7  8
      1 2\oM

1  2  3  4
5  6  7  8
      2 1\oM

1  5
2  6
3  7
4  8
      1 1\oM
1  6
      2 2\oM
4  7
      M1

1  2  3  7
2  4  6  4
3  6  9  8
1  2  3  9
      1 1\oM1
1  4  9  9
      2 2\oM1
7  6  6  1
      V
1  2  3
      1 1 \oV
1
      2 2 \oV
3

```

Please note that applied to vectors, the `1 1` combination returns the first item, and that the `2 2` combination returns the last item.

The third combination `1 1` is useful to retrieve the trace of a matrix. Suppose you have the matrix N:

```

1  2  3
4  5  6
7  8  9

```

The generalized transposition with $1 \ 1 \circ N$ will gather the diagonal elements:

```

      1 1 \o N
1  5  9

```

A subsequent sum-reduction over the result will yield the trace:

```

      +/1 1 \o N
15

```

Rotate. If K is a scalar or one-element vector and X is a vector, then $K \circ | X$ is a cyclic rotation of X defined as follows: $K \circ | X$ is equal to $X[1 + (pX) _1 + K + 1 \text{ i } pX]$. For example, if $X \leftarrow 2 \ 3 \ 5 \ 7 \ 11$, then $2 \circ | X$ is equal to $5 \ 7 \ 1 \ 2 \ 3$, and $_2 \circ | X$ is equal to $7 \ 11 \ 2 \ 3 \ 5$.

If X is a matrix, rotation of each row may be specified in the form $Z \leftarrow K \circ | X$, rotation of each column in the form $Z \leftarrow K \circ - X$. For example, for row rotation, if pX is 3 4, then K must be a scalar or a vector of dimension 3 and $Z[I;]$ is equal to $K[I] \circ | X[I;]$. For column rotation, pK must be a scalar or a vector of dimension 4, and $X[; I]$ is equal to $K[I] \circ | X[; I]$. For example:

M				0 1 2 3 o - M				1 2 3 o M			
1	2	3	4	1	6	11	4	2	3	4	1
5	6	7	8	5	10	3	8	7	8	5	6
9	10	11	12	9	2	7	12	12	9	10	11

The direction onto which apply rotation can be specified for function $\circ |$ with $[D]$, where D is 1 or 2; for example:

```

M <- 3 4 pi 12
M

1  2  3  4
5  6  7  8
9 10 11 12
  1 2 3 o | M

2  3  4  1
7  8  5  6
12 9 10 11
  0 1 2 3 o | [1] M

1  6 11  4
5 10  3  8
9  2  7 12
  1 2 3 o | [2] M

2  3  4  1
7  8  5  6
12 9 10 11

```

The direction onto which apply rotation can be specified for function $\circ -$ with $[D]$, where D is again 1 or 2; for example, using the same M as before:

```

0 1 2 3 o - M

```

```

1   6  11   4
5  10   3   8
9   2   7  12
    0 1 2 3o-[1]M

```

```

1   6  11   4
5  10   3   8
9   2   7  12
    1 2 3o-[2]M

```

```

2   3   4   1
7   8   5   6
12  9  10  11

```

The following equivalences holds:

- $K \circ - M$ is equivalent to $K \circ | [1] M$ and to $K \circ - [1] M$
- $K \circ | M$ is equivalent to $K \circ | [2] M$ and to $K \circ - [2] M$

Reverse. If X is a vector and $R < - \circ | X$, then R is equal to X except that the elements appear in reverse order. Formally, R is equal to $X[1+(pX)-ipX]$. For example:

```

      X
1  2  3  4  5  6
      R<-o|X
      R
6  5  4  3  2  1
      X[1+(pX)-ipX]
6  5  4  3  2  1

```

If A is a matrix, $\circ - A$ is like A except that the order of elements is reversed along the columns; in $\circ | A$, the order is reversed along the rows. For example:

A	o-A	o A
1 2 3	4 5 6	3 2 1
4 5 6	1 2 3	6 5 4

The direction onto which apply reversion can be specified for function $\circ |$ with $[D]$, where D is 1 or 2; for example:

```

A<-2 3pi6
A
1 2 3
4 5 6
  o|A
3 2 1
6 5 4
  o|[1]A
4 5 6
1 2 3
  o|[2]A

```

3 2 1
6 5 4

The direction onto which apply reversion can be specified for function $\circ-$ with $[D]$, where D is again 1 or 2; for example, with the same matrix A :

$\circ-A$

4 5 6
1 2 3
 $\circ-[1]A$

4 5 6
1 2 3
 $\circ-[2]A$

3 2 1
6 5 4

The following equivalences holds:

- $\circ- A$ is equivalent to $\circ|[1] A$ and to $\circ-[1] A$
- $\circ| A$ is equivalent to $\circ|[2] A$ and to $\circ-[2] A$

Compress. The expression U/X denotes compression of X by U . If U is a logical vector (comprising elements having only the values 0 or 1) and X is a vector of the same dimension, then U/X produces a vector result of $+/U$ elements chosen from those elements of X corresponding to non-zero elements of U . For example, if $X < -2 \ 3 \ 5 \ 7 \ 11$ and $U < -1 \ 0 \ 1 \ 1 \ 0$ then U/X is $2 \ 5 \ 7$ and $(\sim U)/X$ is $3 \ 11$.

To be conformable, the dimensions of the arguments must agree, except that a scalar (or one-element vector) left argument is extended to apply to all elements of the right argument. Hence $1/X$ is equal to X and $0/X$ is an empty vector. A scalar right argument is not extended. The result in every case is a vector.

If M is a matrix, then $U/-M$ denotes compression along the first coordinate, that is, the compression operates on each column vector and therefore deletes certain rows. It is called column compression. Similarly, U/M denotes row compression. The result in every case is a matrix. For example:

M

1 2 3
4 5 6
7 8 9
 $1 \ 0 \ 1/M$

1 3
4 6
7 9
 $1 \ 0 \ 1/-M$

1 2 3
7 8 9

Replication. The expression U/X denotes also replication of X by U^{20} . If U is a vector containing integers greater than or equal to 1, their numeric value is interpreted as the number of times the item in the correspondent column or

row of X has to be replicated. The replication returns a new item with the number of columns or rows equal to $+/U$.

Again, if M is a matrix, then $U/-M$ denotes replication along the first coordinate, that is, the replication operates on each column vector and deletes certain rows. It is called column compression and row replication. Similarly, U/M denotes row compression and column replication. The result in every case is a matrix.

```

M
1 2 3
4 5 6
7 8 9
  3 3 0/M

1 1 1 2 2 2
4 4 4 5 5 5
7 7 7 8 8 8
  2 5 0/-M

1 2 3
1 2 3
4 5 6
4 5 6
4 5 6
4 5 6
4 5 6

```

The conformability is the same of the compression feature (there must be as many non-zero items in the left argument as items in the right-argument).

In case of negative values in the left argument U , the correspondent rows/columns are erased:

```

3 3 _2/M

1 1 1 2 2 2 0 0
4 4 4 5 5 5 0 0
7 7 7 8 8 8 0 0
  2 _5 2/-M

1 2 3
1 2 3
0 0 0
0 0 0
0 0 0
0 0 0
0 0 0
7 8 9
7 8 9
  X<-3 4p' ABCDEFGHIJKL'
  2 0 _3 2/X

AA DD
EE HH

```

²⁰ This feature is also in the GNU APL and in the Dyalog APL, but was not available, for what I can retrieve from the manual, in the IBM APL\1130.

II LL

If X is a vector of characters, then spaces are supplied rather than zeros. In the example, column #2 is omitted, column #3 is replicated as three columns of spaces, and columns #1 and #4 are doubled.

As long as the parameters vector U contains only zeroes or ones, this operator works as the compression operator seen before.

Expansion is the converse of compression and is denoted by $U \setminus X$. If $Y < -U \setminus X$, then $U \setminus Y$ is equal to X and (if X is a vector of numbers) $(\sim U) \setminus$ is an array of zeros²¹. In other words, $U \setminus X$ expands X to the form indicated by U , the elements of X assuming the positions of the ones in U , and zeros filling in elsewhere.

To be conformable, $+ / U$ must equal pX in case of vectors. In case of matrices, $+ / U$ must equal $pX[1;]$ in case of $\setminus$ and must equal $p[; 1]$ in case of $\setminus -$.

If X is a vector of characters, then spaces are supplied rather than zeros, i.e., if $Y < -U \setminus X$ then $(\sim U) / Y$ is an array of the space character ' '. Again, for matrices, $U \setminus M$ denotes expansion along the last coordinate, and $U \setminus -M$ denotes expansion along the first. For example:

```

      1 0 1 \i2
1 0 2
      X<-3 4p'ABCDEFGHIJKL'
      X
ABCD
EFGH
IJKL
      1 0 1 1 1 \X
A BCD
E FGH
I JKL
      1 0 1 1 \-X
ABCD

EFGH
IJKL
      1 1 0 1 \-N

      1 2 3
      4 5 6
      0 0 0
      7 8 9
      1 1 0 1 \N

      1 2 0 3
      4 5 0 6
      7 8 0 9
```

A scalar left argument is transparent: $1 \setminus X$ is equal to X .

apple introduces the replication features, using values different from 1 or zero, acting as follows:

²¹ This feature is also in the Dyalog APL, but was not available, for what I can retrieve from the manual, in the IBM APL\1130; moreover, it seems the GNU APL does not feature expansion for negative numbers.

- Positive integers act on existing columns (for $\backslash$, $\backslash[2]$ and $\backslash-[2]$) or rows (for $\backslash-$, $\backslash[1]$ and $\backslash-[1]$), which are replicated according to the specified values;
- Negative integers insert null columns or rows at the indicated position and replicated according the specified values;
- Collectively, the positive numbers (including the 1s) must correspond in quantity to the columns or rows of the argument, while the negative numbers and the zero represent the added columns or rows.

Example:

```

      V
3  6  9
      2 _3 1 2\ V
3  3  0  0  0  6  9  9

```

Here the vector V has three elements, and the positive values are three. Another example:

```

      M
15 12  4
16  6  2
13  9 10
  5  8  7
14  1 11
      2 _2 1 _2 2\ [2] M

15 15  0  0 12  0  0  4  4
16 16  0  0  6  0  0  2  2
13 13  0  0  9  0  0 10 10
  5  5  0  0  8  0  0  7  7
14 14  0  0  1  0  0 11 11
      2 2 _2 1 2 _2 2\ [1] M

15 12  4
15 12  4
16  6  2
16  6  2
  0  0  0
  0  0  0
13  9 10
  5  8  7
  5  8  7
  0  0  0
  0  0  0
14  1 11
14  1 11

```

Decode. The expression $R_|_X$ denotes the value of the vector X evaluated in a number system with radices $R[1], R[2], \dots, R[pR]$. For example, if $R \leftarrow 24 \ 60 \ 60$ and $X \leftarrow 1 \ 2 \ 3$ is a vector of elapsed time in hours, minutes, and seconds, then $R_|_X$ has the value 3723, and is the corresponding elapsed time in seconds. Similarly, $10 \ 10 \ 10 \ 10 \ 10 \ 1 \ 7 \ 7 \ 6$ is equal to 1776, and $2 \ 2 \ 2 \ 1 \ 0 \ 1$ is equal to 5. Formally, $R_|_X$ is equal to $+/W \times X$, where W is the weighting vector determined as follows: $W[pW]$ is equal to 1, and $W[I-1]$ is equal to $R[I] \times W[I]$. For example, if R is 24 60 60, then W is 3600 60 1,

The result is a scalar.

The arguments R and X must be of the same dimension, except that either may be a scalar (or one-element vector). For example, $10 _ | _ 1 \ 7 \ 7 \ 6$ is equal to 1776. The arguments are not restricted to integer values. If X is a scalar, then $X_ | _ C$ is the value of a polynomial in X with coefficients C , arranged in order of descending powers of X .

The decode function is commonly applied in work with fixed-base number systems and is often called the base value function. A common usage is conversion from binary representation. For example:

```

22      2_ | _1 0 1 1 0
23      2_ | _1 0 1 1 1
19      2_ | _1 0 0 1 1

```

In these examples, the least digit (i.e. 2^0) of the binary number is on the right.

Encode. The encode function $R _ | _ N$ denotes the representation of the scalar N in the base- R number system. Thus, if $Z < -R _ | _ N$, then $(x/R) | N - R_ | _ Z$ is equal to zero. For example, $2 \ 2 \ 2 \ 2 _ | _ 5$ is 0 1 0 1 and $2 \ 2 \ 2 _ | _ 5$ is 1 0 1 and $2 \ 2 _ | _ 5$ is 0 1.

```

R <- 2 2 2 2
N <- 19
Z <- R \_ | \_ N
(x/R) | N - R\_ | \_ Z
0

```

The dimension of $R _ | _ N$ is the dimension of R . A common usage, as seen in the examples, is the conversion to binary representation. In the examples, the least digit of the binary number is on the right. A caution must be observed: using the correct number of values, to entirely represent the desired number; for instance:

```

0 0 1 1      2 2 2 2 \_ | \_ 19
1 0 0 1 1    2 2 2 2 2 \_ | \_ 19
0 1 0 0 1 1  2 2 2 2 2 2 \_ | \_ 19

```

The first example converts 19 to its binary representation, but in four bits only. So, the final restitution is 0 0 1 1, which actually is 3. Adding one more bit, as shown, produces the correct result. Adding more bits won't change the final result (third example), because zeroes are appended to the left.

The encode function is also called representation.

Index of. If V is a vector and S is a scalar, then $J < -V i S$ yields the position of the earliest occurrence of S in V . If S does not equal any element of V , then J has the value $1+pV$.

If S is a vector, then J is a vector such that $J < -V i S$ is the index in V of $S [I]$. For example:

```

'ABCDEFGH' i 'GAFFE'
7 1 6 6 5

```

If X is a numerical vector, then the expression $X i \}$ / X yields the index of the (first) maximum element in X . For example, if X is the vector 8 3 5 13 2 7 9, then $\}$ / I is 13 and $X i \}$ / X is 4 (the fourth item is 13).

The result in every case has the same dimensions as the righthand argument of $\mathcal{I}$. For example, if $Z \leftarrow \vee \mathcal{I} S$, and S is a matrix, then $Z[\mathcal{I}; \mathcal{J}]$ is equal to $\vee \mathcal{I} S[\mathcal{I}; \mathcal{J}]$.

If M is a matrix and $\mathcal{J}$ is a vector such that $p\mathcal{J}$ equals the number of rows of M ²², then the number of row is returned as scalar. For example:

```

M<-3 2pi6
M
1 2
3 4
5 6
Mi5 6
3
Mi5 7
4
Mi5 7 5
LENGTH ERROR
Mi5 7 5
^

```

The first statement finds the line 5 6 as the third row in M , and thus 3 is returned. In case no line is found, the number of rows of M plus 1 is returned (4 in this case). In case the length of the vector and the rows number of the matrix do not match, a length error is returned.

Membership. The function $X \in Y$ yields a logical array of the same dimension as X . Any particular element of $X \in Y$ has the value 1 if the corresponding element of X belongs to Y , that is, if it occurs as some element of Y . For example, $(\mathcal{I}7) \in 3\ 5$ is equal to 0 0 1 0 1 0 0 and 'ABCDEFGH' $\in$ 'COFFEE' equals 0 0 1 0 1 1 0 0. If Y is not a vector, all its items are collected into a vector, which is therefore used as the correspondence vector²³.

If the vector U represents the universal set in some finite universe of discourse, then $U \in A$ is the characteristic of the set A , and the membership function is therefore also called the characteristic function.

The size of the result of the function $\in$ is determined by the size of the left argument, whereas the size of the result of the dyadic function $\mathcal{I}$ is determined by the size of the right argument. However, the left arguments of both frequently play the role of specifying the universe of discourse.

Take and drop. If V is a vector and S is a scalar between 0 and pV , then $S / \backslash V$ takes the first S components of V . For example, if $V \leftarrow \mathcal{I}7$, then $3 / \backslash V$ is 1 2 3 and $0 / \backslash V$ is $\mathcal{I}0$, and $8 / \backslash 1$ yields a domain error.

If S is chosen from the set $-\mathcal{I}pV$, then $_3 / \backslash V$ takes the last $|S|$ elements of V . For example, $_3 / \backslash V$ is 5 6 7.

If A is an array, then $W / \backslash A$ is valid only if W has one element for each dimension of A , and $W[\mathcal{I}]$ determines what is to be taken along the $\mathcal{I}th$ coordinate of A . For example, if $A \leftarrow 3\ 4pi12$, then $2\ _3 / \backslash A$ is the matrix

```

2 3 4
6 7 8

```

The function drop ($\backslash /$) is defined analogously, except that the indicated number of elements are dropped rather than taken. For example, $_1\ 1 \backslash / A$ is the same matrix as the one displayed in the preceding paragraph.

The rank of the result of the take and drop functions is the same as the rank of the right argument. The take and drop

²² This feature was not in the IBM APL\1130.

²³ Apparently, as far as I know, this is an enhancement with respect to the APL\1130 because in the APL69 $\vee$ is defined as a vector.

functions are similar to the transpose in that the left argument concerns the dimension vector of the right argument.

Grade up and down. The monadic grade up function $/^{\wedge}\backslash V$ produces the permutation which would order V , that is $V[/math> $/^{\wedge}\backslash V$] is in ascending order. For example, if V is the vector 7 1 16 5 3 9, then $/^{\wedge}\backslash V$ is the vector 2 5 4 1 6 3, since 2 is the index of the first in rank, 5 is the index of the second in rank, and so on.$

If P is a permutation vector, then $/^{\wedge}\backslash P$ is the permutation inverse to P . If a vector D contains duplicate elements, then the ranking among any set of equal elements is determined by their positions in D . For example:

```

      /^\5 3 7 3 9 2
6 2 4 1 3 5

```

The right argument of $/^{\wedge}\backslash$ may be any array A of rank greater than zero, and the coordinate J along which the grading is to be applied may be indicated by the usual notation $/^{\wedge}\backslash [J] A$. The form $/^{\wedge}\backslash A$ applies as usual to the last coordinate. The result of $/^{\wedge}\backslash A$ is of the same dimension as A ²⁴.

Example:

```

      /^\'Mississippi'
1 2 5 8 11 9 10 3 4 6 7

```

The monadic grade down function $\backslash v/$ is the same as the function $/^{\wedge}\backslash$, except that the grading is determined in descending order.

Example:

```

      \v/'Mississippi'
3 4 6 7 9 10 2 5 8 11 1

```

Because of the treatment of duplicate items, the expression $^{\wedge}/ (/^{\wedge}\backslash V) = o | \backslash v/V$ has the value 1 if and only if the elements of the vector V are all distinct. For example:

```

      V<-i6
      ^/ (/^{\wedge}\V) = o | \v/V
1
      V<-1 2 2 4 4 6
      ^/ (/^{\wedge}\V) = o | \v/V
0

```

apple introduces²⁵ also the dyadic grade up operator $S1 /^{\wedge}\backslash S2$ for vectors only, which selects elements in vector $S2$ that are ordered and grouped according to their presence in vector $S1$, and adding at the end the remaining elements in the order as they appear. In practice $S1$ gives the collating sequence for ordering $S2$.

Examples:

```

      'an' /^\ 'Banana'
2 4 6 3 5 1
      'an' /^\ 'BananaA'
2 4 3 5 1 6
      'is' /^\ 'Mississippi'

```

²⁴ This statement, present also in the original APL1130 manual, leaves no room for doubt, and means that if A is a matrix, then $/^{\wedge}\backslash A$ is still a matrix. However, it's curious noting how in subsequent versions of APL, sealed by the ISO Standard, this characteristic will change, setting the result of the grade operators to return a vector in all cases.

²⁵ Again, apparently, this was not in the original APL1130 because all examples and the text are related to the monadic version only.

```

2 5 8 11 3 4 6 7 1 9 10
      1 2/^3 4 5 1 2 6 7
4 5 1 2 3 6 7

```

The second example selects all 'a' (pos. 2 and 4) and then all 'n' (pos. 3 and 5), and then adds the remaining characters 'B' and 'A' in the order as they appear (pos. 1 and 6).

Similarly, the dyadic grade down operator $S1 \setminus v / S2$ first selects elements in vector $S2$ which are not in $S1$, in the order as they appear, and adding at the end the matching elements in reverse order.

```

      'an' \v / 'Banana'
1 3 5 2 4 6
      'an' \v / 'BananA'
1 6 3 5 2 4
      'is' \v / 'Mississippi'
1 9 10 3 4 6 7 2 5 8 11
      1 2\v/3 4 5 1 2 6 7
1 2 3 6 7 5 4

```

The second example selects first all characters that won't appear in 'an', so all 'B' and 'A' (pos. 1 and 6) in the order as they appear, then selects all 'n' (pos. 3 and 5) and then all 'a' (pos. 2 and 4). Notice that the result is not the opposite reverse result of the previous counterexample for $/^{\wedge}\setminus$, because elements in $S2$ are not all distinct.

Random generation. The monadic function $?N$ (where N is any item), produces an item with the same dimensions of the argument, with elements as random numbers. If the argument is any positive value, the random returned is an integer in the range $1..N$; if the argument is any negative value, the random returned is an integer in the range $-N..-1$; if the argument is zero, the random value returned is a real in the range $]0.0, 1.0[$.²⁶

```

      ?5 6 7
3 2 4
      ?5 6 7
4 4 6
      ?5 6 7
4 4 7
      ?5 6 7
4 2 1
      ?_4
_2
      ?_4
_1
      ?0 0
0.13693665 0.82355258

```

Deal. The dyadic function $M?N$ (where both M and N are scalars or one-element arrays), produces a vector of dimension M obtained by making M random selections, without replacement, from the population in the range $1..N$. If either is a vector (with more than one element) or a matrix, only the item at position $[1; 1]$ is taken for the evaluation.

The value of N in the statement $M?N$ is taken modulo 32768.

```

      5?8
5 6 7 8 3
      5?30

```

²⁶ In the random number generation, the original IBM APL\1130 used only scalars and one-element arrays, returning presumably a rank or length error otherwise. This is applied also to the dealing function (see next paragraph).

25 14 17 9 26

In particular, N?N yields a random permutation of order N of all the population in 1..N.

```

      5?5
1   3   2   4   5
      15?15
11  8   1   14  5  15  2   6  12  7  10  13  3   9   4

```

Please, observe that the random number generation algorithm (which is my own) is fairly good in distribution:

```

      EPS<-400p400?400
      (+/EPS)%400
200.5
      EPS<-400p400?1256
      (+/EPS)%400
623.837
      EPS<-400p400?12560
      (+/EPS)%400
6610.9

```

The result is that the mean is always the 50% of the generated numbers; this means that the numbers are distributed quit uniformly.

Matrix inversion and matrix division. See the dedicated chapter.

Unique values. The monadic operator # and the dyadic operator u perform a similar task, isolating unique values of a set and produce a result.

The operator # is called *unique mask*, because it returns a numerical object with the same rank of the argument, with 1 in positions where the item is met for the first time, and 0 in positions where the item is repeated. The search in matrices is performed by rows.

Examples:

```

      M
1   2   3   4
3   2   4   6
5   3   2   4
1   5   4   2
      #M
1   1   1   1
0   0   0   1
1   0   0   0
0   0   0   0
      \o#\oM
1   1   0   0
1   0   1   1
1   0   0   0
0   0   0   0
      #1 2 3 4 3 4 2 5

```

```
1 1 1 1 0 0 0 1
```

The example with `\o#\oM` shows how to perform the search by columns: it's sufficient passing a transposed version to `#` and to transpose the result.

The operator `u` instead, returns a vector containing one copy of each element in the argument, in the same order as they appear in the argument itself, whether it's a scalar, a vector or a matrix. Again. The search in matrices is performed by rows. Differently from the operator `#`, the type of the object is maintained.

Examples:

```

M
1 2 3 4
3 2 4 6
5 3 2 4
1 5 4 2
uM
1 2 3 4 6 5
u\oM
1 3 5 2 4 6
u'WALKING ALONG THE MISSISSIPPI'
WALKING OTHEMSP
```

The example with `u\oM` shows how to perform the search by columns: it's sufficient passing a transposed version of the argument matrix to the operator `u`.

Execute code contained in strings. The monadic operator `eX` executes as APL code the text contained in the string argument `X`. No numeric value or any object that is not a vector is admissible as argument.

This is a very powerful command, that can be used to interpret and execute APL code (for instance, from strings input by the user).

Examples:

```

e'23+1'
24
```

The executed code uses the very same routines and workspace of the current session, so that there is no difference from the code executed in the string and the APL statement typed from console:

```

e'G<-23+1'
G
24
```

Here, the variable `G` was created by the string execution, nonetheless it's a normal variable that is available after the execution.

A caution must be observe for quotes in the text; if the APL command contains single quotes, you can use the double quotes as delimiters for the literal argument of `e`; if the APL command should contain single and double quotes, you should store this string into a variable (by composition, for instance), and then execute the variable.

Formatted output. `A f B` returns a character array representing the numbers in `B` formatted according to the specifications in `A`, which consists of pairs of integers. Each pair is associated with each column of `B`. The first integer in

the pair specifies the field width, that is the total number of characters used to depict the number (including the dot), the second the decimal precision (after the dot). If the second of each pair in A is negative, the output is in exponential format. The total width must be greater than the decimal precision plus 2, or a value error is raised.

If A has only two elements, then all elements of B are formatted according to A.

If A has only one element, then all elements of B are formatted as integers in the characters width according to A.

If the output does not fit the total specified width, this is filled with asterisks.

See for instance the following output, where varying the decimals and maintaining the total width prints all numbers in the same space:

```

10 2fN
1.00      2.00      3.00
4.00      5.00      6.00
7.00      8.00      9.00
10 3fN
1.000     2.000     3.000
4.000     5.000     6.000
7.000     8.000     9.000
10 4fN
1.0000    2.0000    3.0000
4.0000    5.0000    6.0000
7.0000    8.0000    9.0000
10 7fN
1.0000000 2.0000000 3.0000000
4.0000000 5.0000000 6.0000000
7.0000000 8.0000000 9.0000000

```

If one number only is used as a specifier, it must be positive and identifies the integer width; look at the following examples, that progressively extend the width:

```

N
1 2 3
4 5 6
7 8 9
1fN

123
456
789
2fN

1 2 3
4 5 6
7 8 9
4fN

```

```
1  2  3
4  5  6
7  8  9
5fN
```

```
1  2  3
4  5  6
7  8  9
10fN
```

```
1      2      3
4      5      6
7      8      9
```

Each matrix column can be depicted in its own way:

```
10 4 5 3 12 _5fN
```

```
1.0000 2.000 3.00000E+00
4.0000 5.000 6.00000E+00
7.0000 8.000 9.00000E+00
```

In case the format won't fit, asterisks are printed:

```
8 _5f4.5
*****
```

Why? Let's examine the required final result of the exponential: 4.50000E+00, the final result totals 11 characters, but the specified width was 8. Setting it to 12, solves the problem:

```
12 _5f4.5
4.50000E+00
```

A final advice: separation spaces are not provided, so you should specify a width larger enough to return even some spaces that separate the various output; see the following example:

```
10 4 5 3 12 _5fN

1.00002.000 3.00000E+00
4.00005.000 6.00000E+00
7.00008.000 9.00000E+00
```

In this case the second column is 5 characters wide and filled entirely by the number, not providing some extra spaces to the left. The solution is to increase the width:

```
10 4 7 3 12 _5fN

1.0000 2.000 3.00000E+00
4.0000 5.000 6.00000E+00
7.0000 8.000 9.00000E+00
```

Exclusion set. The dyadic ~ operator (called exclusion set) works only on scalars and vectors, and returns a vector containing all the elements in the left argument without the elements present in the right element. Elements in the right element may also be replicated, without any effect on the result.

Examples:

```

      (i9)~1 3 5 7 9
2  4  6  8
      'MISSISSIPPI'~'MIS'
PP
      'MISSISSIPPI'~'MPS'
IIII
      'MISSISSIPPI'~'MPSI'

      'MISSISSIPPI'~'MPSSSS'
IIII

```

The penultimate example shows that if all occurrences in the left argument are contained (at least once) in the right argument, the empty vector is returned in place, printed as a blank line. The last example shows that multiple occurrences of an element in the right item do not influence the result (the replicated items from the second on are simply ignored).

Matrices cannot be used here²⁷:

```

      M
      1 2 3
      4 5 6
      7 8 9
      M~9
RANK ERROR
      M~9
      ^

```

Matching items. The dyadic *match* operator `=_` compares two items, and if they match value-by-value (that means also dimension-by-dimension), it returns 1, and zero elsewhere.

```

      1=_1
1
      1=_0
0
      1 0 1=_1 0 1
1
      1 0 1=_1 0 1 3
0
      'JACK'=_ 'JACK'
1
      'JACK'=_ 'JACKY'
0
      'JACK'=_ 'BARE'
0
      M
      1 2 3
      4 5 6
      7 8 9
      M=_3 3pi9

```

²⁷ This is true also for the ISO version, anyway...

1

Accessing the variables memory. The monadic operator `m` can be used to indirectly access the user variables memory; its argument can be:

- **a string**, and the result is the variable code, if the string corresponds to its name
- **a numeric code**, and the result is the content of the variable whose code corresponds to the numeric argument.

For instance, supposing you have variable `K` in memory:

`m' K'`

2

This means that the variable `K` has code 2 in memory. The interesting fact is that this operator can be iterated:

`mm' K'`

0	0	6
1	0	_11
0	1	6

`K`

0	0	6
1	0	_11
0	1	6

The first `m' K'` returns 2 and thus the second operation becomes `m2` and the content of `' K'` is returned.

Please note:

- 1 This command was not in the original APL\1130, and is added to **apple** for the advanced programming.
- 2 Since the internal code of the variables can change during the session because the memory space is reorganized and optimized both in deleting the variables and at the end of the execution of the programs, the code of the variables is not a constant, and therefore this operator is useful because it can be used without knowing the code of the variable, but only its name.
- 3 The I-Beam variable `I_48` returns the number of the user variables currently in memory, so the two tool `m` and `I_48` can be used to dynamically access the memory variables contents in an exact range.

Comments. The symbol `co` identifies what follows as a comment, for illustrative purposes and not to be performed; it can start at any character position, at the beginning of a statement or after the semicolon or directly after an expression, and can also be used in programs.

This symbol recalls the original $\oslash$, called *Lamp*, and if you ideally juxtapose the letter `c` (typed as `⏟`) and the letter `'o'` you get α , which is a kind of *Rotated Lamp*. Furthermore, since the word *comment* begins with "co", the word *comment* itself introduces a comment. Great!!

The comment extends to the end of the input line. Here are some examples:

```
comment sum ; 2+3 ; 4+5
2+3 ; comment sum ; 4+5
```

5

```
2+3 ; 4+5 ; comment sum
```

5

9

```
2+3 comment sum ; 4+5
5
```

Numeric comparison. Numeric comparison is a delicate operation, in computer math. Well, if only integers are involved in a numeric comparison, this can be done without special care, because integers have proper and definite math (of course). But with decimal numbers things are different, because we must decide in advance the depth of our comparison, for not all decimal numbers can be rendered perfectly. To this extent, the `) COMP` statement comes in handy. It can be used in two ways:

- The first, with `) COMP 0`, uses the natural depth of the double type to be used in comparison (watch out! For what is the natural behaviour of the C double type, a number may host some rubbish).
- The second is fixing the comparison tolerance, by setting a number in the range $[10^{-1}, 10^{-77}]$, so that all numbers smaller than the tolerance limit are considered as zero.

An example is one thousand words.

```
A<-0.00000001
B<-0.0000001
) COMP 5
WAS 0
A=B
1
) COMP 0
WAS 1E_05
A=B
0
```

In the first comparison of A and B (which differ by the quantity -9×10^{-8}), the comparison was set to 10^{-5} so that the difference between the two numbers is not detected, and so they are considered equal (the value 1 for *true* is returned).

In the second comparison, the tolerance was reset, so that now we can detect that the two numbers are different, provided that the two numbers fit perfectly the double C type.

Another example with matrices; here, H and H1 are two 5×5 matrices that differ in the lower right element, which is zero in H and 10^{-8} in H1:

```
H1
1      2      3      4      5
2      4      6      8      10
3      6      9      12     15
4      8      12     16     20
5      10     15     20     1E_08
H
1      2      3      4      5
2      4      6      8      10
3      6      9      12     15
4      8      12     16     20
5      10     15     20      0
H=H1
```

```

1  1  1  1  1
1  1  1  1  1
1  1  1  1  1
1  1  1  1  1
1  1  1  1  0
      ) COMP 5
WAS 0
      H=H1

```

```

1  1  1  1  1
1  1  1  1  1
1  1  1  1  1
1  1  1  1  1
1  1  1  1  1

```

The comparison tolerance is null (from the previous example), so that the two matrices are different (note the lower right zero). But setting the tolerance to a value greater than 10^{-8} (here 10^{-5}), makes zero and 10^{-8} undistinguishable, and so the comparison returns all ones²⁸.

The comparison tolerance must be taken in great consideration for all the mathematical comparison processes, because a wrong tolerance limit may upset your calculations.

String comparison. To compare if A is equal to 'Jack' you may be tempted to type:

```
A=' Jack'
```

but what is the result? If A is a four-letter word, the result is not a single 1 or zero (to attest the truth or falsity of the comparison) but a 1 or a zero for any character in the sequence; so that if A is effectively 'Jack', the result will be

```
1  1  1  1
```

and if A is for instance equal to 'Bare', the result will be

```
0  1  0  0
```

This is because we are not testing strings, but vectors (of characters), that are checked item-by-item. If we should compare strings of different length, a problem of conformability arises; for example, if A is 'Queen', the test

```
A=' Jack'
```

will return a length error.

The test should be divided in two parts, the first to check the lengths:

```
(pA) =p' Jack'
```

and in case of true value, perform the check:

```
A=' Jack'
```

And the string comparison succeeds if all ones are returned. Now, to return a proper True or False flag, a first method involves the compression:

²⁸ Can you see now how important is using the right comparison tolerance limit?

```

      ^/A=' Jack'
1
      ^/A=' Bare'
0

```

Another method implies the usage of the inner product, multiplying in series all the results of the equal sign (this is a singular approach, using the inner product between vectors of equal length, but note that this is valid for **apple** but it may not for other compilers or interpreters):

```

      Ax.=' Jack'
1
      Ax.=' Bare'
0

```

The problem in comparing strings using APL structures is that if they differ in length an error is returned, and not a false-test value. The following program COMP compares two strings of any length, and returns 1 in case they match in length **and** in characters, 0 otherwise:

```

      \/ Z<-L COMP R
[1]    Z<-(pL)=pR
[2]    ->0xiZ=0
[3]    Z<-Lx.='R
      \/
      A<-' Jack'
      A COMP ' Jack'
1
      A COMP ' Bare'
0
      A COMP ' Queen'
0

```

The same task anyway is performed by the dyadic operator =_ (*match*, see the previous chapter), that does this for you in one turn:

```

      A<-' Jack'
      A=_ ' Jack'
1
      A=_ ' Bare'
0
      A=_ ' Queen'
0

```

String reversing. An operator for reversing a string does not appear in the list of APL operators... why? Because the reversing of a string is obtainable with a simple rearranging of the indices. See the following example:

```

      |_|<-G<-'RING TO RING'
RING TO RING
      pG
12
      G[13-i12]
GNIR OT GNIR

```

The call to pG is done to get the string length (and if you know it, it's superfluous).

The 'secret' (if you let me pass this term) is the rearranging indices list command `13-i12` that, if you ever wonder, returns:

```
12 11 10 9 8 7 6 5 4 3 2 1
```

which is the reversed order of the characters of the string. This is another demonstration of the power of APL.

Type conversion. A vector containing numbers in the range of the ASCII table cannot be printed automatically as string, because the ASCII values are printed as numerical items as in the following example:

```
      M
65  78  78  65
```

To convert this vector to type string, it suffices to concatenate to it an empty string (which is the last item, but the first to be interpreted, influencing the assembling of the rest of the vector):

```
      Q<-M, ''
      Q
ANNA
      pQ
4
```

The string has no added items (like New Line or so), and the string vector has the same length of the string 'ANNA', which is 4.

The same can be said for the opposite operation, where an empty numeric vector is concatenated to the string Q:

```
      N<-Q, i0
      N
65  78  78  65
      pN
4
```

In case of a matrix (or a vertical vector) containing ASCII numbers to be converted to string, the catenation of a single empty char is of no use:

```
      M<-3 3p65 78 67 69 79 72 68 74 76
      M
65  78  67
69  79  72
68  74  76
      N<-M, ''
LENGTH ERROR
      N<-M, ''
      ^
```

because the dimensions involved don't match. Of course, one could ravel the whole matrix before adding the empty char, like this:

```
      N<-(,M), ''
      N
ANCEOHDJL
```

but maybe this is not what the programmer would want. The only solution for maintaining the matrix format is to recreate the matrix using the new string as a filler:

```

N<-3 3p(,M),''
N
ANC
EOH
DJL
pN
3 3

```

As you can see, the last empty char is not really added to the matrix, and N is the string version of M.

There is no practical ways instead to convert a number contained into a string into its proper numeric version, or vice versa. This can be done through specific math/ASCII calculations, of course, in proper programs, but `apple` has a specific tool for this.

The operator `c` is used to convert a string that contains a number into this number, or a number into its string representation. For example:

```

A<-'24'
cA
24
36+cA
60
36+A
DOMAIN ERROR
36+A
^
F<-c24
pF
2
F,i0
50 52
pF,i0
2

```

The quantity `cA`, with `A = '24'` returns a number, and this can be proved by the calculation `36+cA`, which returns regularly `60`. The same with `A` only, returns an error, because numbers and strings cannot be added.

The further example shows that when the number `24` is converted into the string `F`, this is a regular string (printed in its numeric constituents with a trick exposed earlier in this chapter) of length `2`.

Type check. It seems strange that APL constructors and designers put a great attention in adding elements to check the rank of APL items (like the operator `p` - rho - that is used to check the rank and the dimensions), but didn't use the same care to add a means to check if an item is a string or a number. Put in other words, they enabled the system to deal with strings and numbers, emitting errors where a number is expected or vice-versa, but didn't give the programmer a means to check this on her/his own.

This lead each environment to develop its own strategy (for instance, the Dyalog APL uses the `⌈VF⌈` system variable).

`apple` uses the `n` monadic operator (*number*) in a very simple way: if the argument is of type string, it returns `0`, otherwise `1`.

So, to check for strings, check for 0, to check for numbers, check for 1. See the examples:

```
s' ANNA'
0
s24
1
```

Timing Operations. To measure the timing of a single operation, the usual timing features are not useful, because they measure the session length. One could think to use the `I_21` I-Beam variable, registering two recordings (one at the start and one at the end) and returning the difference. But the whole process takes up its own time, and I bet this is something that alter significantly the measure itself.

You can build your own program, of course, and in the literature there are programs to record timing data. But **apple** has its own timing operator (which was not in the original APL\1130 compiler): it's the operator `⌈` for *timing* (what fantasy!); it measures the timing execution of the processing of what is its argument, that is whatever is at its right. For instance:

```
⌈19x19x19x19
TIMING: 0.0242 ms
130321
```

In all, the multiple operations take 0.0242 milliseconds. What if we measure each iteration?

```
⌈19x⌈19x⌈19x⌈19
TIMING: 0.00391 ms
TIMING: 0.0149 ms
TIMING: 0.0188 ms
TIMING: 0.022 ms
```

The progression is evident. The total is 0.022 milliseconds, which is the same of the previous total; the other show that each multiplication adds its operation on the previous, in a very proportional way, taking in account that the first measure is not an operation but a simple integer.

The output is in three measure units: if the timing is less that 1000 ms, milliseconds are output (ms); if the timing is less that 1000 s, it's in decimal seconds (s), otherwise it's in integer seconds. Examples:

```
TIMING: 0.124 ms
TIMING: 2.34 s
TIMING: 1230 s
```

This operator can be used to speed up some programs, if you feel they are slow. For instance, the `INV` program and the `|%` inversion operator can be measured this way:

```
⌈INV M
TIMING: 0.879 ms

_1.72222    0.777778   _0.0555556
 1.44444   _0.555556    0.111111
_0.0555556   0.111111   _0.0555556
  ⌈|%|M
TIMING: 0.0161 ms

_1.72222    0.777778   _0.0555556
 1.44444   _0.555556    0.111111
```



```
_0.0555556    0.111111  _0.0555556
```

The builtin operator is 50 times faster than INV (as it should), and this is a good starting point for an attempt to improve INV (provided this can be done, and I'm not sure it can...).

Separation symbols. Two symbols play a very special role in APL: they are the colon and the semicolon. The colon : is used only to end a label; for instance

```
LABEL: <statement>
```

is a legal usage of the colon. In any other context, it's a syntax error. E.g.

```
3:5
SYNTAX ERROR
3:5
^
```

The semicolon ; has three main scopes; the first is to separate two indices groups in the index selector [. .], as seen in many examples like M[1 2;3 4]. The presence of the semicolon informs **apple** about a request to retrieve matrix data, and if directed to a non-matrix object, it generates a rank error. For instance:

```
      V
1  2  3  4  5
      V[2;2]
RANK ERROR
      V[2;2]
      ^
```

Or for instance:

```
      M
1  2  3
4  5  6
7  1  2
      M[1;4;7]
INDEX ERROR
      M[1;4;7]
      ^
```

The second scope is used to separate the local variables from the right variable in a header of a program in the internal editor. E.g.:

```
\ /Z<-FAC N;I;J
```

The example shows a header that returns a value (Z), uses an argument in monadic mode (N) and uses also two local variables, called I and J, that are declared during the program execution.

The third scope, also seen in many examples, is used to separate two instructions, in a command or programming line, as for example:

```
'THE RESULT IS ' ; 4+5
THE RESULT IS 9
```

Each of the instructions are executed from left to right, in order.

Using the semicolon in different contexts is an imprecise error, depending on the particular situation; for instance, invoking:

```

      4%; A<-3
SYNTAX ERROR
      4%; A<-3
      ^

```

a syntax error is generated, because the statement 4% has no meaning. A comment, as seen in a previous section, interprets as a comment (and thus ignores) anything that follows, up to the end of the line, including the semicolon sections within:

```
A<-3 ; comment ; A
```

The previous statement has no output, because the assignment has no output and everything after `comment` is ignored.

Note: quite all statements, including the system commands, the tracing/stopping commands and the jumping to a line in runtime, can be be sequenced within semicolon sections, but the internal editor cannot because the local variables in the header, separated by semicolon, interfere with this mechanism, as seen earlier for the second scope.

2.10. MATRIX INVERSION

`apple` introduces, with respect to the original IBM APL\1130 version, the matrix inversion, using the symbol `| % |`, which tries to emulate the symbol of the percentage sign enclosed in a square²⁹.

Many methods were developed in the years for the determination of the inverse of a matrix; the ones used in `apple` are the Feddeev-LeVerrier algorithm, that is applied to invertible matrices, and the Ben Israel-Cohen algorithm for the pseudo-inverse, which is applied to non-invertible matrices. In the following they are both exposed.

2.10.1. The Feddeev-LeVerrier algorithm

This algorithm tries to find the coefficient of the characteristic polynomial, expressed as:

$$p_A(\lambda) = \det(\lambda I_n - A) = \sum_{k=0}^n c_k \lambda^k$$

with the assumption that

$$c_n = 1$$

and

$$c_0 = (-1)^n \det(A)$$

The process is iterative, and for successive positions, it evolves into a matrix linearly dependent on the inverse of A ; it starts from a fixed starting position which, if A is the matrix to be inverted, is expressed by:

$$M_0 = 0$$

$$c_n = 1$$

where M is an aid-matrix which will become close to the inverse of A , and c_k , for $k = 1..n+1$, are the

²⁹ This is the symbol used in the DEC80 and ISO APL environment.

coefficients of the characteristic polynomial associated to A .

At each step

$$M_k = A \cdot M_{k-1} + c_{n-k+1} I$$

$$c_{n-k} = -\frac{1}{k} \text{tr}(AK_k)$$

where $\text{tr}()$ is a function that returns the trace of the matrix argument.

When the process stops, the final configuration is:

$$M_n = \sum_{k=1}^m c_k A^{k-1}$$

$$c_0 = -\frac{1}{n} c_n \text{tr}(A)^n + c_{n-1} \text{tr}(A^{n-1}) + \dots + c_0 \text{tr}(A)$$

From this final set, we can observe that:

$$A^{-1} = -\frac{M_n}{c_0} = (-1)^{n-1} \frac{M_n}{\det(A)}$$

and at the end of the iteration, we have found:

- The inverse of A ;
- The characteristic polynomial whose coefficients are c_k (operator j);
- The determinant, which is the last coefficient c_0 , corresponds to the known value of the polynomial (operator d).

I chose this algorithm because it's fast, robust, simple and never fails. A characteristic polynomial is generated for all matrices, provided they are square, and the only condition of failure (for the inversion) is the null determinant, in which case the matrix is sent to the Ben Israel-Cohen algorithm for the solution of the pseudo-inverse, and this is the argument of the next section.

2.10.2. The Ben Israel-Cohen algorithm

If the Feddeev-LeVerrier algorithm cannot find an inverse (because the determinant is null), a pseudo-inverse search is tried; a pseudo-inverse does not respect the strong condition $A \cdot A^{-1} = I$. The pioneering work of Moore and Penrose³⁰ establishes four weaker conditions for such an inverse to exist (where A^+ is the pseudo-inverse):

1. $A \cdot A^+ \cdot A = A$
The product $A \cdot A^+$ is not the identity matrix, but it maps all columns of A to themselves
2. $A^+ \cdot A \cdot A^+ = A^+$
This is the *weak inverse* principle.
3. $(A \cdot A^+)^T = A \cdot A^+$
 $A \cdot A^+$ is Hermitian.
4. $(A^+ \cdot A)^T = A^+ \cdot A$
 $A^+ \cdot A$ is also Hermitian

If the found inverse satisfies the first two conditions, this is called a *reflexive inverse*.

If the found inverse satisfies all of the four conditions, the inverse is called *the Moore-Penrose inverse*, or *pseudo-*

³⁰ See for instance https://en.wikipedia.org/wiki/Moore%E2%80%93Penrose_inverse

inverse. If the found inverse satisfies only some but not all of the conditions, it's called a *generalized inverse*. The uniqueness of the pseudo-inverse is guaranteed by the third and fourth principles.

In case the pseudo-inverse respects the condition $A \cdot A^{-1} = I$ but not the contrary, it's a right-inverse; in case the pseudo-inverse respects the condition $A^{-1} A = I$ but not the contrary, it's a left-inverse. This type of pseudo-inverse is as close as can be found to proper inverses, even with zero determinant.

A pseudo inverse is unique, and this is the power of the Moore-Penrose theory. If the matrix is non-singular, its generalized inverse is also unique.

In case of singular matrices their generalized inverse is not unique, and there may exist different inverses. So the result returned by **apple** may differ from those of other methods, even if I noticed that much of the generalized inverses returned by **apple** are also returned by Wolfram Alpha (<https://www.wolframalpha.com/>).

The algorithm that bases the Ben Israel-Cohen algorithm for the calculation of a generalized inverse is the following equation:

$$A^+ = A^+ \cdot A \cdot A^+$$

where A^+ is the supposed pseudo inverse. So, after a bit of matrix rearrangement, the following expression is calculated:

$$A_{i+1}^+ = 2A_i^+ - A_i^+ A \cdot A_i^+$$

This equation can be solved by iteration, and at each step $i+1$, the result is an approximation of the expression, which evolves towards an inverse. The iteration takes few steps to converge, and this is the major advantage of this algorithm.

The only problem is that this iteration must be started with an appropriate A_0 matrix satisfying the equation:

$$A_0 \cdot A = (A_0 \cdot A)^T$$

The A_0 value chosen for **apple** is the transpose of A multiplied by a very small term which is 10^{-12} and the difference between two consecutive results uses the same 10^{-12} as epsilon; this ensures that convergence occurs and that the result is achieved in no more than 100 iterations at most (you can know how many by checking the I-Beam variable `I_37`).

2.10.3. Inversion rules

The inversion mechanism of the operator `| % |` acts on any numeric item, according to the following rules:

- If the inversion of a scalar is required, its reciprocal is returned if not null, otherwise zero is returned.

```

    | % | 5
0 . 2
    | % | 0
0

```

The inversion, if found, satisfies of course the following condition, where N is any non null scalar:

$$N \cdot N^{-1} = 1$$

$$N^{-1} \cdot N = 1$$

and this is always true, even for zero, but in this case we must consider that $0 \cdot 0^{-1} = \frac{0}{0} = 1$.

- If the inversion of a horizontal vector is required, a normalized vertical vector is returned:

```
V<-2 3 4 5 6
| % | V
```

```
0.0222222
0.0333333
0.0444444
0.0555556
0.0666667
```

The normalized vector satisfies the following condition, if V is the vector to be inverted:

$$V \cdot V^{-1} = 1$$

The reciprocal, of course, returns a matrix.

- If the matrix is a pure diagonal square matrix, the solution is given returning a matrix which is also diagonal and square, with the non-null elements as reciprocals. For instance, the following holds:

```
A<-3 3p1 0 0 0 2 0 0 0 0
A
```

```
1 0 0
0 2 0
0 0 0
```

```
AI<-| % | A
AI
```

```
1 0 0
0 0.5 0
0 0 0
```

The inverted matrix, if found, satisfies the following condition:

$$A \cdot A^{-1} = I'$$

$$A^{-1} \cdot A = I'$$

where I' is the identity matrix with nulls in the diagonal where A has nulls in the diagonal:

```
AI+ .xA
```

```
1 0 0
0 1 0
0 0 0
```

- If the inversion of a vertical vector is required, a normalized horizontal vector is returned:

```
V<-5 1p2 3 4 5 6
V
```

```
2
3
4
5
6
```

| % | V
0.0222222 0.0333333 0.0444444 0.0555556 0.0666667

The normalized vector satisfies the following condition, if V is the inverted vector:

$$V^{-1} \cdot V = 1$$

The reciprocal, of course, returns a matrix.

- If a matrix inversion is required and if and only if the matrix is squared and invertible, the search of the inverse is performed using the Feddeev-LeVerrier algorithm. In this case the determinant is not null, and is assigned to the variable DETERMINANT (if already existing as scalar). So, in order to access the determinant, it suffices that DETERMINANT is created as a scalar before the inversion:

DETERMINANT<-0

The inverted matrix, if found, satisfies the following conditions:

$$A \cdot A^{-1} = I$$

$$A^{-1} \cdot A = I$$

- If all the previous attempts failed, the matrix is singular, or not square, so a generalized inverse is tried using the Ben Israel-Cohen algorithm, **that returns a result in any case**.
- The console command) INV can drive the calculation by admitting to the inversion only invertible matrices, so that, if enabled with) INV 1, if matrix M is not invertible or it is not a matrix, a domain error is returned. If disabled, instead (case of) INV 0, the default state), if M is not invertible or is not a matrix, the pseudo-inverse is tried.
- The inversion code of the I-Beam variable I_35 and the descriptor string returned by I_36 are calculated during the inversion, as follows (the symbol A^+ indicates the pseudo-inverse, and A^T identifies the transpose):
 - 0: for a unique inverse, I_36 contains "PROPER INVERSE", and I_35 is 0; this is the case when the determinant is not null;
 - 1: for a special inverse item, I_36 contains some specific object strings (like "SCALAR INVERSE", "VECTOR INVERSE", etc), and I_35 is -1; this is the case of scalars, diagonal matrices, vectors, vertical vectors (i.e. one-column matrices), and in general items which are not matrices.
 - 2: For a generalized inverse, the two I-Beam variables I_35 and I_36 are found as follows:
 - a) for a generalized inverse, I_36 contains the word "GENERALIZED", and I_35 is incremented by 2^0 ; this is when the matrix satisfies the first Moore-Penrose condition $A \cdot A^+ \cdot A = A$;
 - b) for a generalized reflexive inverse, I_36 contains also the word "REFLEXIVE" and I_35 is incremented by 2^1 ; this is the case when the matrix satisfies the first Moore-Penrose condition of the above case and also the second Moore-Penrose condition $A^+ \cdot A \cdot A^+ = A^+$;
 - c) if the matrix satisfies the third Moore-Penrose condition $(A \cdot A^+)^T = A \cdot A^+$, I_36 contains also the words "LEFT-HERMITIAN", and I_35 is incremented by 2^2 ; this is also the case when the fourth Moore-Penrose condition is not satisfied.
 - d) if the matrix satisfies the fourth Moore-Penrose condition $(A^+ \cdot A)^T = A^+ \cdot A$, I_36 contains also the words "RIGHT-HERMITIAN", and I_35 is incremented by 2^3 ; this is also the case when the third Moore-Penrose condition is not satisfied;
 - e) if the matrix satisfies both the third and fourth Moore-Penrose conditions, I_36 contains the sole word "HERMITIAN", and I_35 is incremented by $2^2 + 2^3$;
 - f) if the matrix satisfies all the four Moore-Penrose conditions (i.e. I_35 contains 15), I_36 contains the sole text "PSEUDO-INVERSE"; the matrix is a Moore-Penrose matrix, unique, Hermitian and Reflexive, but not a simple generalized inverse;

- g) in case the pseudo-inverse is a left inverse, I_36 contains the string "LEFT PSEUDO-INVERSE"; this is the case when the matrix has linearly independent columns, satisfying the rule $A^+ = (A^T \cdot A)^{-1} \cdot A^T$, which is to say that $A^T \cdot A$ is invertible but $A \cdot A^T$ is not; the counter is incremented by 2^4 ;
 - h) in case the pseudo-inverse is a right inverse, I_36 contains the string "RIGHT PSEUDO-INVERSE"; this is the case when the matrix has linearly independent rows, satisfying the rule $A^+ = A^T \cdot (A \cdot A^T)^{-1}$, which is to say that $A \cdot A^T$ is invertible but $A^T \cdot A$ is not; the counter is incremented by 2^5 ;
 - i) in case the pseudo-inverse is both a left and right inverse, I_36 contains the string "QUASI-INVERSE"; this is the case when the inverse is not the proper inverse but it is the best approximation available; the counter is incremented by 2^6 . Note: this case is very rare.
- 3: the number of iterations of the last successful process launched for finding the proper inverse is contained in the I-Beam variable I_37.

Here are some examples; the first is the inversion of a proper invertible matrix. How do we know? The I-Beam variable I_35 returns zero, that is the flag for *proper inverse* and see what returns the I-Beam variable I_36:

```

H
1  2  3
4  5  6
7  8  _9
| % | H
_1.72222    0.777778    _0.0555556
 1.44444    _0.555556    0.111111
_0.0555556    0.111111    _0.0555556
I_35
0
I_36
PROPER INVERSE

```

The second example involves a non-invertible matrix. How do we know? The I-Beam variable I_36 returns "PSEUDO-INVERSE", and the I-Beam variable I_35 returns 15:

```

G
1  2  3
4  5  6
7  8  9
| % | G
_0.638889    _0.166667    0.305556
_0.0555556    0    0.0555556
 0.527778    0.166667    _0.194444
I_35
15
I_36
PSEUDO_INVERSE

```

The third example shows another pseudo-inverse:

M1

```

1  2  3  7
2  4  6  4
3  6  9  8
1  2  3  9
    | % | M1

_0.0106383    0.0192503    0.02077    _0.0187437
_0.0212766    0.0385005    0.04154    _0.0374873
_0.0319149    0.0577508    0.06231    _0.056231
 0.0673759   _0.0425532   _0.0283688    0.102837
    I_35
15
    I_36
PSEUDO-INVERSE
M1+.x(| % | M1)+.xM1

1  2  3  7
2  4  6  4
3  6  9  8
1  2  3  9
    ( | % | M1)+.xM1+.x| % | M1

_0.0106383    0.0192503    0.02077    _0.0187437
_0.0212766    0.0385005    0.04154    _0.0374873
_0.0319149    0.0577508    0.06231    _0.056231
 0.0673759   _0.0425532   _0.0283688    0.102837

```

The last two operations are the demonstration of the first two Moore-Penrose pseudo-inverse properties $A \cdot A^+ \cdot A = A$ and $A^+ \cdot A \cdot A^+ = A^+$.

Another example involves a 2x6 matrix and a right pseudo-inverse:

```

F

1  2  3  4  5  6
7  8  9 10 11 12
    | % | F

_0.253968    0.111111
_0.163492    0.0777778
_0.0730159    0.0444444
 0.0174603    0.0111111
 0.107937   _0.0222222
 0.198413   _0.0555556
    I_35
47
    I_36
RIGHT PSEUDO-INVERSE
F+.x| % | F

1  0
0  1

```

The last operation shows that, with this matrix, the right pseudo inverse returns the identity matrix if matrix F is

right-multiplied by its inverse.

Note: the) INV console command, if a value different from zero (which is the default state) is used, disables all inversions that are related to singular square matrices, not square matrices and non-matrices items. In practice, only invertible matrices (square matrices with non-null determinant) are inverted; all other cases return a domain error. E.g.

```

M<-3 3pi9
M

1 2 3
4 5 6
7 8 9
) INV 1
WAS 0
| % | M
DOMAIN ERROR
| % | M
^
) INV 0
WAS 1
| % | M

_0.638889    _0.166667    0.305556
_0.0555556  _1.52933E_14    0.0555556
0.527778    0.166667    _0.194444

```

2.11. MATRIX DIVISION

apple introduces the matrix division by using operator `| % |` also in dyadic form; for not singular square matrices (i.e. for those returning `I_35` equal to zero) this corresponds to the exact solution of the system represented by the matrix on the right, and the knowns vector on the left (this happens only if the number N_u of the unknowns and the number N_k for the knowns are equal, and this corresponds to the dimensions of the square coefficient matrix); for singular matrices, this corresponds to the least square method approximates solutions.

2.11.1. Exact solutions

The exact solutions for the non singular matrix respond to the following criteria, where x is a vector of size N with the unknowns variables, b is the vector of size N with the known values (*knowns*), and A is the coefficient matrix of size $N \times N$:

$$A \cdot x = b \quad A^{-1} \cdot A \cdot x = A^{-1} \cdot b \quad x = A^{-1} \cdot b$$

The power of the matrix division operator is that it returns the solutions of a linear system in a single operation! For example, if M is the coefficient matrix and F is the knowns vector:

```
M<-3 3p1 2 3 4 5 6 7 8 _9
F<-2 4 8
```

you can obtain the result in the classic way:

```
( | % | M) + . xF
_0.777778
_1.55556
_0.111111
```

or using the fast way of the dyadic inverse:

```
F | % | M
_0.777778
_1.55556
_0.111111
```

with the very same result.

If F is a matrix, its dimensions should be equal in rows to the dimensions of the coefficient matrix M ; each column of F (which need not be equal to the dimension of the coefficient matrix) is the collection of knowns vectors; the result is a matrix with the same dimensions of F in which every column is a solution for the correspondent knowns column in F .

```
M<-3 3p1 2 3 4 5 6 7 8 _9
F1<-3 2p3 2 8 4 5 8
F1
3 2
8 4
5 8
F1 | % | M
0.777778 _0.777778
0.444444 1.55556
```

```
0.444444  _0.111111
```

The second column in F1 replicates the previous vector F, and therefore, the second column in the result matrix replicates the previous vector solution.

2.11.2. Least squares analysis

When there are more rows than the solutions, or the coefficient matrix cannot be inverted (because it's a singular matrix, or not square) the least square algorithm is tried. The least squares method tries to find the best fit for a set of data points that cannot be used for a normal equations system. The method works by minimizing the sum of the residuals of points of curve that origins from the matrix data. See the Matrix QR Decomposition chapter for the theory behind the algorithm.

The least square analysis is performed using the same QR Decomposition algorithm of the operator `q`, and here is one numerical example (the same example will be expanded and justified in the paragraph about the least square analysis):

```

M
3   5   9
2   1  _3
3  _5   2
2   4  _1
_1   2   8
4   1   1
T
0.691473
2.48101
0.404958
0.31173
0.608437
1.47789
T | % | M
0.322974
0.0725512
_0.0276467
```

To see which kind of results is this, let's pre-multiply it by the original coefficients matrix:

```

X<-T | % | M
X
0.322974
0.0725512
_0.0276467
M+.xX
1.08286
0.80144
0.550873
0.9638
_0.399045
```

1.3368

The error estimation is easily done:

```
+ / + / T - M + . x X
1.63878
( + / ( + / ( T - M + . x X ) * 2 ) ) * 0.5
2.11082
```

We can see that the error (that is the deviance of the current known values from the result curve that is generated) is not negligible, and is indeed a big error; anyway, these are not exact solutions in the sense used for non-sparse regular systems of equations), but approximate solutions, that minimize the error.

The `) INV` command does not affect the Least Square analysis of the `| % |` operator, because it always deal with non-invertible matrices.

As a final consideration, let me say that the matrix division operator in APL is one of the most powerful tools in all the programming languages that I have met.

2.12. I-Beam system variables

The I-Beam system variables were a first attempt to grant the access to several items of information, available both at console and at the programming level. The versions of APL that followed, and above all the ISO versions, removed the I-Beam variables in favour of the so called `square variables`, that could be addressed as any other variable.

I-Beam Code	I-Beam scope
I_1	The integer fuzz value (can be assigned). As an extension to <code>) FUZZ</code> , the assignment to <code>I_1</code> can store values in the range 1 to 308, which are the power of 10 that sets the computing level under which all numbers are to be considered zero.
I_2	The integer printing precision value (can be assigned) for the mantissa of the number. As an extension to <code>) DIGIT</code> , the assignment to <code>I_2</code> can store a printing precision up to 52 (to show the entire C-double mantissa), but the number is largely useless, because of the garbage content in the final part of the mantissa, typically after the 17th position. The values ranges from 1 to 52. Please note: the values specified by <code>) DIGITS</code> are those of the mantissa of the number; so that an operation like <code>4%9000</code> yields <code>0.000444444</code> with <code>) DIGITS</code> set to 6, because the mantissa is <code>4.44444</code> , but with effective 9 digits printed after the decimal dot
I_3	The integer screen width (can be assigned). It ranges from 30 to the current system width.
I_4	The integer seed value (can be assigned). It ranges from 2 to 32768.
I_5	The comparison tolerance (can be assigned). As an extension to <code>) COMP</code> , the assignment to <code>I_5</code> can store values in the range 1 to 308, which are the power of 10 that sets the comparison tolerance under which two numbers are to be considered equal.
I_6	The zero-division tolerance (can be assigned), which is 0 or 1. When zero, a division by zero causes error; when not zero, zero is returned.
I_7	The index origin, which is 1 or 0 (can be assigned).
I_8	The cumulative milliseconds integer since session start; <code>) CLEAR</code> resets value.
I_9	Now's hour integer 0-23
I_10	Now's minute integer 0-59
I_11	Now's second integer 0-59
I_12	Today's year integer \v/Y
I_13	Today's month integer 1-12
I_14	Today's day integer 1-31
I_15	Today's date string DD-MMM[text]-\v/Y (es. 06-JUN-2022)
I_16	Now's hour string with seconds HH:MM:SSn (es. 12:46:35)
I_17	Now's hour string without seconds HH:MM (es. 12:47)

I-Beam Code	I-Beam scope
I_18	The workspace status
I_19	The exponential flag, which is 0 or 1 (can be assigned). When 1, all numbers are printed in exponential.
I_20	Time of day in 60th of a second
I_21	The computer clock in 60th of a second since ignition (equals uptime)
I_22	A numeric vector containing the number of remaining declarable variables and user programs, in this order
I_23	The number of users/terminals currently signed on (which is always 1)
I_24	The time in 60ths of a second since the beginning of the apple session.) CLEAR resets value
I_25	Today's date in a composed integer (e.g. 20220330 for March 30, 2022) ³¹ .
I_26	The line number of the function currently being executed
I_27	A numeric vector containing the line numbers of all functions whose execution has been started but not yet completed, with the most recent function first
I_28	The current file name (a string) (if any) T{ I_29:T{ The name of he function currently being executed (a string)
I_30	The current user name (a string, that can be input with option --user=<u>)
I_31	The upper alphabetic characters set A..Z
I_32	The lower alphabetic characters set a..z
I_33	The digit character set 0..9
I_34	The punctuation characters set !"#\$%&'()*+,-./:;<=>?{ }~
I_35	The last matrix inversion code
I_36	The last matrix inversion description
I_37	The last pseudo-inversion iterations count
I_38	The inversion flag (can be assigned), which is 0 or 1. When zero, all inversion are accomplished; when not zero, only square invertible matrices return a result, the rest returns a domain error.
I_39	The memory status
I_40	The last recursion iterations count
I_46	The maximum number of declared variables during a running program (iterations included); this number includes the label variables instantiated during a program execution if any, and also the argument and the local variables.
I_48	The number variables in memory
I_49	The number of programs in memory
I_50	Restore the error messages for the execution operator
I_51	Suppress the error messages for the execution operator
I_60	Clear the execution state indicator
I_99	Terminate the APL session

Table 2.10: I-Beam variables list

The APL69 does not report any I-Beam list, and does not explain if the compiler could dispose of them or not. The APL70 does, and so **apple**, following this line, has implemented the I-Beam variables, with the ability, for some of them, to be assigned on runtime, to change some program's behaviour.

The original symbol for the type of variables called *I-Beam* used a pair of superimposed T and ⊥, which gave rise to a sort of I with the braces and without the serifs (a sort of T), and hence the name **I-Beam**. For **apple**, the symbol is I_, or the capital I followed by the underscore.

The I-Beam variables can be read:

```
F<-I_4
```

and, where available, assigned:

```
I_4<-0
```

³¹ Use) DIGIT with an argument 8 or greater to see the full integer values.

The original I-Beam list, available for the common user, didn't include a number of variables destined to administrators only. But being **apple** a standalone application, with one only user, the programmer can dispose of all the I-Beam variables. The original list was not contained in the APL69, so I had to squeeze my mind and imagine what could be useful for the programmer that the language does not already return. All the I-Beam variables can be consulted in reading, and some also can be assigned (see Table 2.10).

If the I-Beam variable returns a vector, its items can be selected using the same indexing technique of normal variables:

```

      I_17
10:53      I_17[2 1 3]
01:

```

The difference between the `I_46` and the `I_48` I-Beam variables is the following: `I_46` returns the total number of variables in the most recent program execution (including variables that were created in some sub programs and destroyed afterwards); it is useful to determine the total volume of the calculation. `I_48` instead returns the number of variables currently in the workspace, the same values that can be viewed with the command `) VARS`. E.g.

```

      ) VARS
M
      I_48
1
      I_46
0
      M
1 3
0 1
      SVD M
10
      I_48
4
      I_46
379
      ) VARS
M      SIGMA      U      V

```

The last four I-Beam variables work as commands rather than variables. They modify a state, rather than return an output, and they cannot be used in assignment. In particular, `I_60` is equivalent to `) RESET` and `I_99` is equivalent to `) OFF`; the suppression of the error messages for the execution operator `⍉` (for `I_50` and `I_51`) disables/enables only the message printing, not the error state, that continues to act by interrupting the command.

A final consideration: the I-Beam values are contained in special memory blocks, not in user variables, so that they don't appear when the `) VARS` system command is used.

2.13. Special 'square' variables

Two variables deserve attention. They are called *square* variables but they are not variable at all. The first is the pure square variable (`□` - called *quad* - in original APL) that is used to assign user numerical quantities to a variable:

```

      G<-|_|
|_|:
      @

```

When using this command, after printing the prompt `|_` :, the cursor moves to the position indicated by @ (which does not really appear on the screen) and the user is invited to type a numeric quantity (numbers, expressions, vectors or whatever is a numerical expression to be assigned).

The variable `|_` is not a variable, but rather an input operator, that parses only mathematical expressions. It's called *variable* because it appears as a variable-to-variable assignment.

The assignment to this variable prints whatever is the final result of the assigned expression; for example³²:

```
|_ <- 45x78
3510
```

but this is rather useless, because the statement `45x78` would print the result anyway; the proper function of the assignment to the quad operator is that even assignments are revealed; generally, assignments are silent:

```
A <- 45x78
A
3510
```

To reveal the assigned value, you have to invoke the variable name, as in the previous example. The assignment to the quad operator has the handy property to show the result of the assignment for every statement, included assignments:

```
|_ <- A <- 45x78
3510
```

A famous and special example is at page B.1 of the APL70; in explaining that all programs in the library are alterable, the book explains that at any time a statement like this:

```
P <- (P, 0) + 0, P
```

can be changed by line editing to the following form:

```
|_ <- P <- (|_ <- P, 0) + |_ <- 0, P
```

This offers a glance to the inside of the calculation; here is a run-time example:

```
P <- 1
|_ <- P <- (|_ <- P, 0) + |_ <- 0, P
0 1
1 0
1 1
```

Repeating the calculation reveals the algorithm:

```
|_ <- P <- (|_ <- P, 0) + |_ <- 0, P
0 1 1
1 1 0
1 2 1
|_ <- P <- (|_ <- P, 0) + |_ <- 0, P
0 1 2 1
1 2 1 0
1 3 3 1
```

³² There can be no more than 100 quad print operators on a single line, but I bet this is sufficient for anyone of us, isn't it?

This is something that I like a lot!

The second *square* variable is called *quad-char*, because as with the quad, is used to get input from the user, in the form (this time) of a string, that is a collection of characters. The input mask is different than before:

```
G<-|'|
HAPPY THE MAN
G
HAPPY THE MAN
```

The input starts from the first character of the line, without any description, and the input string is typed without quotes. To add a description, the quad-char variable can be assigned:

```
|'|<- 'ENTER A NUMBER: '
G<-|'|
ENTER A NUMBER: 24
G
24
```

In this case, G would be set to the string 24, which is not a number, but a string. As a string, it cannot be used into calculations:

```
G+24
LENGTH ERROR
G+24
^
```

Please, note that if more than one assignment to `|'|` is met³³, its evaluation proceeds as usual; take a look at the following examples:

```
|'|<- 'ENTER A: ', '|'|<- 'ENTER B: '
A<-|'|
ENTER A: ENTER B: BLUEBIRD
A
BLUEBIRD
```

In the previous example, the two values are concatenated by the comma, following the usual APL conventions (firstly, the description string is set to "ENTER B: "; secondly, this result is concatenated to the second evaluated operation, that sets the description string to ENTER A: ENTER B:). If we divide the two operations, instead:

```
|'|<- 'ENTER A: '; '|'|<- 'ENTER B: '
A<-|'|
ENTER B: BLACKBIRD
A
BLACKBIRD
```

The two requested statements are performed from left to right, and the second (the one after the semicolon) is performed as last, so it overwrites the description string of the previous call. The result is that only ENTER B: is printed.

2.14. File Input/Output

Following the suggestions of the DEC APL-11, and adapting the rules to Unix, the OS on which apple runs, I designed four more dyadic operators that enable the APL user to read from a file or write to a file.

³³ A very useless operation...

They are simple tools, working in two modes: *object-mode* and *line-mode*; the first treats APL objects (scalars, vectors and matrices of two classes, numeric and string, written as they would appear in print and rebuilt in the reading phase); the second mode simply treats textual lines (objects are printed on a line and the reading phase does not rebuild them, assigning all characters to a string vector); as such, this mode is not limited to APL data but can treat any text, but remember that an object (e.g. a matrix) written to file, when it's read back, it does not transformed back to matrix rank, but it remains a string vector which hosts the *picture* of the matrix.

The file operators have also tools for knowing the file length, the position of the file pointer, the alignment to the next object and also for the EOF flag.

Only 12 contemporary files are available, but this is surely enough for any usage.

Note: the precision of the decimal numbers written to file is maximized so as to allow a more effective reconstruction in reading. The)DIGIT set up is not influenced by this procedure.

The operators involved are:

1. **Open/close a file** (dyadic |v|). There are four invocation types for opening a file and one for closing it:

```
Z<-C|v|'file-name'      : open file for read (no specifiers); set pos=0;
Z<-C|v|[1]'file-name'   : open file for read (specifier 1); set pos=0;
Z<-C|v|[2]'file-name'   : open file for write (specifier 2); set pos=EOF (append-mode);
Z<-C|v|[3]'file-name'   : open file for write (specifier 3); set pos=0 (erase mode);
Z<-C|v|0                : close file
```

Any of the file-open invocations will open file 'file-name' to channel C, which is a scalar in the range 1 to 12; the connection will be in read or write mode and the opening process will return in Z the scalar zero if all went well, otherwise an error code (see ahead). 'file name' must respect the Unix file system naming conventions and may also contain a path.

For example, here is how to open a file name called data.txt (with a relative path) on channel 1 in read mode in a program:

```
W<-1|v|'/provider/elements/data.txt'
->0xiW#0
```

Of course, the file (or the relative path start) must reside in the same directory where **apple** is run, otherwise it has to be called with the whole absolute path before the name.

The call Z<-C|v|0 will instead close the file on channel C, after saving the file if in write mode; it will return in Z the scalar zero if all went well, otherwise an error code. The right argument is the scalar zero or the empty vector. If the file was already closed, this is irrelevant, and the call will return zero as well: after all, closing a closed file has no risks...

2. **Setting the file pointer and getting info** (dyadic |^|). This operator must be invoked this way:

```
Z<-C|^|POS
```

This call sets the file position on the C channel to POS (which can be any positive scalar not greater than the length of the file); The returned value Z is set to scalar zero if all went well, otherwise it contains an error code. An error is raised if POS is greater than the length of the file.

This operator can also be used in interrogation mode or action mode about the file pointer status:

- if POS is _1, it returns in Z the current file pointer (a positive scalar)

- if POS is _2, it returns in Z the current last possible file pointer, that is the file length (a positive scalar)
- if POS is _3, it returns in Z a true value (the scalar 1) if the end-of-file was reached, or a false value (the scalar zero) if not.
- If POS is _4, it aligns the cursor to the first useful object saved in object-mode; this is because, in case of reading in object-mode, the set position may not exactly correspond to the first correct saved object. It returns in Z the scalar 1 if EOF is reached, or the scalar zero in case textual data is available.

3. **Read data from file** (dyadic |<|). This operator must be invoked this way:

Z<-C|<|RES

If, during the reading phase, this call recognizes that an object has been written to a file in object-mode, it reads it back and tries to rebuild it, assigning it to Z, otherwise it reads a textual line from channel C and stores it as a string vector in Z; RES is the flag that controls the action, which is executed if it contains a scalar value other than zero; if instead it contains the scalar zero, the data is read, but not stored in Z, which will instead contain the scalar zero (skip-mode).

Warning! While reading any textual file that contains specific terminal commands (usually introduced by ESC, for instance for setting the colours of text or the background), these characters are executed, rather than stored (this is an effect of the terminal, not of **apple**).

4. **Writing data to file** (dyadic |>|). There are three invocations types::

Z<-C|>|OBJ : write OBJ in object mode;
 Z<-C|>|[1]OBJ : write OBJ in object mode;
 Z<-C|>|[2]OBJ : write OBJ in line mode;

This call writes the OBJ object to the C channel (which must have a file attached in write mode) in object-mode or line-mode; if everything went well it returns the scalar zero in Z, otherwise it returns an error code. In case of object-mode, an APL object is saved in order to guarantee a subsequent reconstruction of the object, while in line-mode, the APL object is saved as an image (in characters), which can be subsequently printed but not reconstructed (for example, a numeric matrix written in line-mode is read as a character string representing the output of the matrix, but this object is not a numeric array, but rather a string vector).

The file operators' indices are not influenced by) ORIGIN, because they are operational indices, not axis.

2.14.1. File error messages

The four added file-operators required new error messages, because the official APL\1130 error messages were largely insufficient; so I created a bunch of error messages, in the same vein of the original error messages, that could help in dealing with errors in the file operators.

They were conceived to be self explanatory, nonetheless in the next table you can see these error messages with some explanation, just to let you know their extents:

Type	Causes	Corrective actions
WRONG WRITE SPECIFIER ERROR	Illegitimate [index] in the file open file write operators	Retype
CANNOT OPEN THIS FILE	Problems in opening the file: 1. Maybe the file is in an unreachable directory 2. Maybe the file is not available 3. Maybe the file name or path was misspelled	1. Check the file position 2. Check the file existence 3. Check the file name or path

Type	Causes	Corrective actions
TOO MUCH OPENED FILES ERROR	There are already 12 opened files	Close at least one previous opened file before opening this new
FILE ALREADY OPENED ERROR	The channel is already associated to a file	Close the file before reassigning the channel
CANNOT OPEN THIS FILE FOR READ ERROR	The file-open for read failed for some reasons	Check the file
CANNOT OPEN THIS FILE FOR WRITE ERROR	The file-open for write failed for some reasons	Check the file
FILE NOT ALREADY OPEN ERROR	The call to read or write is on a channel not already associated to a file	Open the file
TRYING TO READ FROM A WRITE-FILE ERROR	The read operator is used on a channel opened for write	Open a read file or use the right channel number
CORRUPTED DATA ERROR	The data exported in object-mode was not correctly saved to file	
TRYING TO WRITE TO A READ-FILE ERROR	The write operator is used on a channel opened for read	Open a write file or use the right channel number
FILE CONTROLS AVAILABLE ONLY IN READ MODE ERROR	The position operator can be used only while reading from a file and not while writing to a file	
ILLEGAL FILE POINTER ERROR	The position operator cannot set a pointer past the file length	Review the value

Table 2.11: File error reports

3. SYSTEM COMMANDS

The system commands help in getting information from the system core (the workspace) and with the file system (of the host computer). They form the basis of the operating system of the APL\1130 (and of **apple**, with the addition of some commands that help working under Unix). They are substantially divided in *Console commands* and *Special commands*.

Here they are.

3.1. Console commands

The console commands constitute the first important group of commands. The following object-conventions are used:

- [WSID] stands for the name of a workspace file, with extension `.wks`; don't confuse [WSID], which is a file name, with `)WSID`, which is a proper system command.
- [NAMES] stands for a list of one or more objects separated by blanks, that are variables or program names. In case [NAME] (singular) is used, it stands for one object only.
- [VAL] stands for a numeric quantity.
- [FILE] stands for a file name, for which the letters depend on the file system (for Linux, for instance, even lower letters or symbols like the dash or the underscore can be used). Typing the Tab character after a few characters (when a file is searched) will offer a list of all the files in the current directory that match those few characters; when the list is reduced to one item, the whole file name is printed; this avoids typing the entire name³⁴.

Watch out! A file with name made of the sole character zero (ASCII 48) is misinterpreted in **apple** and can cause some bad behaviour in the normal execution. So, if you have a file with such a name, add an extension (such as `.apl` or `.txt`.) to solve the problem.

The *current directory* is the directory where **apple** is run. Any file name can be given with the complete path, if necessary.

The console commands are preceded by the closing bracket `)`, and are followed by any spaces and by the command itself, of which only the first four letters are considered.

Here are the console commands that govern an **apple** session; the string "**Standard**" defines entries that belong to the APL\1130, the string "**Not Standard**" defines entries that were added to **apple** and that were not in the original APL\1130.

)CLEAR

Standard

This command starts a new empty workspace, erasing variables and programs that were eventually created. `)CLEAR` sets the workspace as if started from scratch and resets timing.

)COMP [VAL]

Not Standard

This command, if used without argument, shows the current comparison tolerance value; if used with a numeric argument, sets the comparison tolerance. The comparison tolerance value is either zero, or an integer value in the range 1-77, to define the tolerance range in the interval $[10^{-1}, 10^{-77}]$, and represents the lower limit to be interpreted as the acceptable difference in comparisons.

)CONTINUE

Standard

This command, just like `)OFF`, closes the current session, saving the workspace data into a hidden system file, including the total timing measurement. When **apple** is run again, it finds the hidden file and reloads the workspace, so that the session starts as a continuation of the previous.

³⁴ This is a features built-in in the `readline` library, not my own creation.

)COPY [WSID] Standard

This command copies the entire workspace identified with [WSID], without erasing the current. If an object with the same name should exist, it's rewritten.

)COPY [WSID] [NAMES] Standard

This command, a variant of the)COPY command, copies only the objects identified by [NAMES], from the stored workspace [WSID], without erasing the current workspace.

)DATE Not Standard

This command prints a line with date, like this

Date: JAN 23, 2024

)DEBUG [OFF] Not Standard

This command enables the debug property. At start it's disabled. When enabled, it prints a copy of the function line that is about to be executed; this process helps in understanding the program flow, and see where it fails to behave as intended. Of course it clutters up the whole output, so it must be used with care, and only if necessary. To stop debugging, use OFF.

)DESCRIBE Not Standard

This command prints the description of the workspace (if any), that was set by the last)SAVE or loaded with the last)LOAD commands. In case no description is available, the NO DESCRIPTION. message appears.

)DIGIT [VAL] Not Standard

This command, if used without argument, shows the current output precision; if used with a numeric argument, sets the output precision to that number. The precision is an integer between 1 and 16, and represents the significant digits shown in the output. The precision does not alter the memory contents of the items, which retains always the widest precision.

)DISPOSE Not Standard

This command clears the screen.

)DIV [VAL] Not Standard

This command sets the division tolerance; if)DIV is set to 0 (the default), a division like $N\%0$ generates a domain error, unless N is zero, generating the division $0\%0$ which returns 1; if)DIV is set to 1, divisions by zero return zero in all cases, without errors. This reflects also in the reciprocal function % (e.g. $\%0$ returns zero).

)DROP [WSID] Standard

This command erases the workspace [WSID] from the current directory.

)EDIT [FUNC] Not Standard

This command uses the editor program set in the definition VEDIT in custom.h, and starts editing the function [FUNC]; if [FUNC] is not found in the workspace functions list, a new empty file is offered; if [FUNC] is found in the workspace functions list, it's printed in a temporary file by means of the same routine of the)LIST command (without the leading numbers in square brackets); at the end of the editing session, the APL session is regained and the function is reloaded.

)EDITFILE [FILENAME] Not Standard

This command uses the editor program set in the definition VEDIT in custom.h, and starts editing the file [FILENAME]; at the end of the editing session, the APL session is regained. The file is not automatically loaded, and you should)IMPORT it, before calling the program.

)ERASE [NAMES] Standard

This command erases the objects identified by [NAMES] from the current workspace. If in the workspace there is a variable and a program with the very same name NAME (e.g.),)ERASE NAME will erase the variable, and not the

program; another `) ERASE NAME` will erase the program also. If you should want to erase the program `NAME` only, save the variable `NAME` in a copy, to be restored after the double erasure. If the argument is `@VARS`³⁵, all variables are cleared, but programs remain. If the argument is `@PROGS`, all programs are cleared, leaving variables untouched. To clear both programs and variables, use `) CLEAR`.

)EXP [VAL] Not Standard

This command, if used without argument, shows the current exponential flag; if used with a numeric argument, sets the exponential format for all numeric output. The output is conditioned by `) DIGIT`, which establishes the number of decimals of the exponential format. It accepts only 1 (exponential active) or 0 (exponential not active, default).

)EXTENDED Not Standard

This command prints the list of APL ASCII linear algebraic operators understood by apple.

)EXPORT [NAME] Not Standard

This command (that may be written also as `) WRITE`) exports the program `[NAME]` to the file identified by the same name (in upper characters) plus the extension `.apl`. The exported file has in the first line the header, that instructs `IMPORT` to correctly set the imported program with the right parameters. If the documentation variable `DNAME` (where `NAME` is the name of the program) is present in the workspace, it's also saved as an assignment, after the program³⁶. See the definition for `IMPORT` about the file names.

)FILES Standard

This command lists all the channels' state of the current session.

)FNS Not Standard

This command lists all the defined functions present in the current workspace.

)FUZZ [VAL] Not Standard

This command, if used without argument, shows the zero threshold; if used with a numeric argument, sets the zero-threshold in the range 1-77. The zero-threshold is the value below which all numbers are printed as zero. For instance, if `) FUZZ` is set to 14, all values lower in absolute dimension than 10^{-14} are printed as zero. The fuzz value does not alter the memory contents of the items, which retains always the widest precision.

)HELP [<op>] Not Standard

This command, without arguments, prints a brief summary of the system commands. In case an operator is specified as argument, it prints a brief help about its functionality, both in monadic and/or dyadic forms. The forms `) HELP TRACE`, `) HELP STOP` and `) HELP TRACE` give help respectively about the tracing and stopping features and about the extended operator `⌈` for matrix tests (in the extended manual).

)IBEAM Not Standard

This command prints a summary of the I-Beam variables and their properties.

)IMPORT [FILE] Not Standard

This command (that may be written also as `) READ`) imports the program which is written in the APL file `[FILE]`; if no extension is provided, `.apl` is assumed. If the file has a lower-cases characters name, for instance `fo-cal.apl`, the file name must be typed in lower case. The name of the program will be the name specified in the file, and not the file name. If `[FILE]` is a headless program (in practice, it's only a list of APL statements), the program is loaded as an argument-less function with the same name of the file name, without extension and in upper case. Three messages are printed: if the loading goes well, the "XXXX IMPORTED" message appears; if the loading fails (for some reasons), the "NOT IMPORTED" message appears; if the program is already in memory, the "ALREADY IMPORTED" message appears, to force loading, use `) ERASE` before importing. The command will also look for an assignment that follows the program. This assignment should use a variable whose name is `'D' +` by the

³⁵ The clauses `) ERASE @VARS` and `) ERASE @PROGS` are an apple feature, not present in the original APL1130.

³⁶ Needless to say, but the program name should be at least one character shorter than the maximum length, to let the documentation variable name be valid!

function name (e.g, if the imported program is `INV`, the variable must have the name `DINV`); this variable constitutes the documentation (D) file of the program and is generally a string variable, but not necessarily. This command sets the inversion flag; if `) INV` is set to 0, all inversions are accomplished (from scalars to singular matrices); this is the default state; if it's set to a value different from zero, only an invertible square matrix returns a result; all other cases return a domain error. This is useful for testing matrices, when only invertible matrices are useful.

)LIB [OPTION] Not Standard

This command lists the files in the current directory, and works as the Unix program `ls`; the [OPTION] string is passed *as-is* for some specialization of the output (options are the same of `ls`).

)LIST [NAME] Not Standard

This command lists the program in the current workspace whose name is [NAME]. This is a more convenient way of getting the listing of a program, rather than using an editor. The command `) LIST` can be used even when an error has occurred, without disturbing the state indicator.

)LOAD [WSID] [NODESCRIBE] Standard

This command loads the workspace [WSID] from the current directory. It's not necessary to specify the extension. The current workspace, if not saved, is lost and erased before loading the required [WSID]. The keyword `NODESCRIBE` avoid the printing of the describe string.

)MEMORY Not Standard

This command prints a memory report about the used bytes of the current session; this is the total amount of reserved bytes, but not all may be really used. It uses the same routine that calculates the `I_18` I-Beam value.

USED MEMORY = 4096

)OFF Standard

This command closes the session with a regular logoff, without saving the workspace on the system hidden file. Hitting `CTRL-D` during an `apple` session is equivalent to `) OFF`.

)OPS Not Standard

This command prints the list of APL ASCII operators understood by `apple`.

)ORIGIN [VAL] Not Standard

This command sets the index origin to 1 or zero,

)PCOPY [WSID] Standard

This command operates like `) COPY`, but protecting all objects with the same name.

)PCOPY [WSID] [NAMES] Standard

This command operates like the `) COPY` variant, but protecting the objects with the same name.

)PROG Not Standard

This command lists the files with extension `.apl` in the current directory.

)PRWS Not Standard

This command prints the content of the current workspace in a readable format. Non-scalar variables (vectors and matrices) are printed along with their dimensions; functions are displayed as via the `) LIST` command. This command is programmable.

)PWD Not Standard

This command prints the name of the current directory.

)RENAME OLD NEW Not Standard

This command changes the name of an existing user program in the current workspace³⁷. The change refers only to the current workspace and does not extend to the files and saved workspaces in the current directory.

) RESET

Not Standard

This program resets the state indicators and all suspended functions, returning to the global level.

) SAVE [WSID] [DESCRIBE] [APPEND]

Standard

This command saves the current workspace under the name [WSID]; if the argument is not given, and the name of the workspace is not empty, uses the name of the workspace as the name of the workspace file. The used name becomes in any case the name of the current workspace. See further for the expansion commands on SAVE. The keyword DESCRIBE forces the asking of the describe string, which will be saved to disk in the workspace file; the keyword APPEND forces appending the current describe string to the existing one, rather than overwriting it.

) SCRIPT [APPEND] [OFF] [FILE]

Not Standard

This command activates the session scripting to file [FILE]; if a file name is not provided, uses the same default file of the option --script; the keyword APPEND instructs the function to append the scripting rather than putting it on a new reset file; the keyword OFF instead, turns off the scripting. The script is a complete transcription (errors included) of the console session.

) SEED [VAL]

Not Standard

This command, if used without argument, shows the current seed; if used with a numeric argument, sets the current seed. If the seed is zero, the random number generation is randomized at each random generation; if the seed is in the range 2...32768, it's used as the random number generator for the current session; this assures a repetition of the generated random numbers, if necessary for testing purposes.

) SESSION

Not Standard

This command prints a time report for the current session. The value is the same of I_24 in sixties of seconds, but also a time report (in usual HH:MM:SS mask) is printed.

) SHELL [NAME]

Not Standard

This command, if used without argument, starts a shell session with the default shell specified in the file custom.h (which is bash); if an argument is specified (possibly with the complete path), this must be an executable that acts like a shell, at the exit of which the apple APL session is restored; so) SHELL will start the default shell, and for instance) SHELL csh will start a csh shell. When back to the APL session, the message "(restoring APL session)" is printed, to inform you that you are still in the APL session³⁸.

) SI

Standard

This command lists the interrupted programs (the ones that could not end their duty because the user has typed CTRL-C). It's in simplified form with respect to the one in the APL69, because apple does not distinguish between suspended and pendent interruptions...

) SIV

Standard

This command is a variant of) SI, that adds the list of the program local variables. Variables are printed in the declaration order, not in alphabetical order. As for) SI, it's in simplified form with respect to the one in the APL69, because apple does not distinguish between suspended and pendent interruptions...

) SPACES

Standard

This command lists the workspaces in the current directory (i.e. the files with extension .wks).

) STATE

Not Standard

This command prints a report of information about APL, apple, and the computer on which apple is running. The mask is as follows (the figure XXX is substituted by the current datum and VV is substituted by the current flag

³⁷ Thus in memory, not in the file system.

³⁸ A stupid-proof feature that I have introduced... and don't ironize who is the stupid here!

value³⁹).

```
APL is running on a XXXX system
Environment is XXXX
APL interpreter is apple (GPL version)
and current version is XXXX
compiled on XXXX
Runtime expressions are limited to XXX characters
No limits for array dimensions
Current workspace occupies XXXX bytes (that is XXXX Kb)
```

Flags:

```
DEBUG IS XXXX WIDTH IS VV DIGIT IS VV
DIV IS VV FUZZ IS VV EXP IS VV
ORIGIN IS VV SEED IS VV COMPARISON TOL. IS VV
Current XXXX width is VV
```

```
JAN 22, 2024 - 20:15:21 - session time: 1.10 secs
Total time: 02:37 mins
```

) TIME

Not Standard

This command prints a line with time, like this

```
Time: 16:45:04
```

) TYPE [FILE]

Not Standard

This command prints on the screen the content of [FILE], which is any textual file; if no extension is provided, .apl is assumed.

) VARS

Standard

This command lists the variables in the current workspace. If) SI is empty, then the list includes only global variables. If) SIV is not empty, then the list includes also the local variables of the interrupted programs.

) WIDTH [VAL]

Not Standard

This command, if used without argument, shows the current width of the screen; if used with a numeric argument, sets the width of the screen. The width is in the range from 30 to the width of the screen. The width influences the data that if printed are longer than width, and thus broken with an indentation of six blanks.

) WSID [WSID]

Standard

This command, if used without argument, shows the current workspace name; if it's followed by any characters list, this is set as the current workspace name.

-> [VAL]

Standard

This command, if the last program was interrupted by the user or an error occurred for some reasons, enables the wiping of the program state, to assure that a re-run is possible. It must be followed by the number zero; other line numbers generate an error, because the go-to from session is not supported in apple.

There may be any or no spaces between) and the command, and only the first four characters of a command are meaningful; you are free to complete the word or to use other letters; moreover, commands are case insensitive; under these rules, the command) CLEAR, for instance, can be typed as) CLEA,) clean,) CLEAVER or even) cleantosaurus.

³⁹ It's worth noting here that WIDTH, with emulated terminals, shows the current dynamic width when) WIDTH is zero or the fixed width when) WIDTH is not zero; the string *dynamic/fixed* appears in the final line, which shows the current usable width value.

There are some short-cuts that can be used:) V stands for) VARS,) F stands for) FNS) and) S stands for) STATE.

The commands) CLEAR) COPY,) ERASE,) SAVE,) LOAD,) PCOPY,) RENAME, cannot be programmed. The rest of the commands can be programmed.

Whenever you use a programmable command in the form) COMMAND (where COMMAND is any command), it's loaded and executed during run-time. The command] IMPORT instead assumes the form of the so called *pragma command*⁴⁰; in this form, it's executed during the loading phase (with) LOAD or) IMPORT), rather than during run-time; this is useful to load program libraries during load-time, rather than run-time, that would cause errors and problems.

For instance, if the program CALC pragma-imports the program gdc.apl, as if in the following piece of code:

```
\ / Z<-CALC N
] IMPORT libs/gdc.apl
...
\ /
```

when LOAD or IMPORT is invoked for loading CALC, this would cause the appearing of the following messages:

```
      ) IMPORT CALC
GDC IMPORTED
CALC IMPORTED
```

signalling that now GDC is available for CALC but also for the user. Remember that while) IMPORT can be typed from console and into programs, the form] IMPORT can be used into programs only.

The hidden system file cannot be deleted directly; to erase it you may:

- Type) CLEAR followed by the command) CONTINUE; this will erase the workspace and save it the hidden file as empty.
- Use option --clear at start, and type) CONTINUE as the first command; this again will save an empty workspace to the hidden file.
- Delete the file .aplcont.wkc from the file system through the Unix console; **apple** won't complain if the hidden file is missing; it will simply start a new empty active workspace.

The complete execution line of the SAVE command is this:

```
) SAVE [WSID] [DESCRIBE] [APPEND]
```

The [WSID] token is the name of the destination file of the workspace. If not specified, the current workspace name is written.

The [DESCRIBE] token, when added, causes SAVE to ask a describe text from keyboard, with the prompt > and CTRL-D to end the input, after which the description text is saved to file; if the [APPEND] token is used, the text is appended to the current text, before saving, otherwise the input text is saved as new. The command) DESCRIBE can be used to print the current description text. Only the first four characters of DESCRIBE and APPEND are parsed. An example follows:

```
      ) SAVE BOZO DESC APPEND
> This workspace contains some programs that deal with math
>
> - the program LOG2 returns the logarithm of base 2
```

⁴⁰ This **apple** features is not available in other compilers, though this form is used by GNU APL for special system commands.

```
> - the program BASE treats numbers in any base
> - the program SEC calculates the secant
>
> See the comments in the listings for more info.
BOZO SAVED.
```

At the prompt >, start typing. Type CTRL-D when over.

The complete execution line of the LOAD command is this:

```
)LOAD WSID [NODESCRIBE]
```

The)LOAD command knows that, when a workspace is loaded and a descriptive text is found at the end of the data, it must retrieve it, print it and save it to memory (for the possible use of) DESCRIBE). E.g., for the BOZO example:

```
)LOAD BOZO
This workspace contains some programs that deal with math

- the program LOG2 returns the logarithm of base 2
- the program BASE treats numbers in any base
- the program SEC calculates the secant

See the comments in the listings for more info.
BOZO LOADED.
```

In case you don't want the description to be printed, use token NODESCRIBE (that can be abbreviated to the first four characters):

```
)LOAD BOZO NODE
```

The workspace is loaded silently, and no description is output.

Note: the commands)COMP,)DIGITS and)FUZZ deal with the limits 1-77 for)COMP and)FUZZ, and 1-16 for)DIGITS. These limit can be overridden by using the IBeam variables I_1, I_2 and I_5 that accept wider ranges, and in particular the limits are 1-308 for I_5 (comparison tolerance) and I_1 (fuzzing), and 1-52 for I_2 (digits).

One final note: there is an Easter-egg command. Can you find it, without peeking at the code?

3.2. Special commands

Some commands play a very special role in the APL environment; the following table 3.1 lists them.

IBM operator	apple operator	Purpose	Notes
→	->	branch	
<i>absent</i>	CTRL-C	program and session interrupt	
<i>absent</i>	CTRL-D	input stop	
TΔ	T/\	tracing feature	
SΔ	S/\	stopping feature	
∇	\/	programming editing and items deletion	

Table 3.1: Special commands

A **branch** (goto) may occur in a programmed statement together with one or more specifications, provided that the branch is the last operation to be executed (i.e., the leftmost). For example, the statement `->SxI N>I<-I+1` first augments I by 1; and then branches to statement S if N exceeds the new value of I. Look at the following program:

```
\ / SHOULD;N;I
N<-3
I<-2
->5xI N>I<-I+1
' THIS SHOULD APPEAR'
' THIS SHOULD ALSO APPEAR'
```

The result is simple: if the check of `N>I` should be made *after* the increasing of I, only the last line would be printed, because the jump would occur. In case it should be made *before*, `N>I` would fail, and thus the fourth line would be printed as well; the execution by apple follows:

```
THIS SHOULD APPEAR
THIS SHOULD ALSO APPEAR
```

In the expression `Z<-(A+B) x (C+D)` it's immaterial whether the left or the right argument of the `x` is evaluated first, and hence no order is specified. The principle of no specified order in such cases is also applied when the expressions include specification. Suppose, for example, that A is assigned the value 5 and the expression `Z<-(A<-3) x A` is then executed.

apple's behaviour is this: the argument to the left of `x` is executed first (because all assignments are executed first, see the workflow in chapter 9), then A is assigned the value 3; the argument on the right then has the value of 3, and Z is finally assigned the result 9.

I underline here that the branch is a programming feature, not available from console.

The **execution-interrupt** command CTRL-C stops every process and should (should...) return to the session, for a command to be typed. The message:

```
apple: CTRL-C pressed.
```

appears, to inform about the interruption. I must warn that this feature is very experimental, and that probably, at present, does not work in all context as I desire.

The **input-interrupt** command CTRL-D, if typed during an input (as the first input character), acts according to the following features:

- If CTRL-D is typed before any input during an apple session it's equivalent to `) OFF` (that is quit without saving).
- If CTRL-D is typed before any input in programming mode, it re-asks for the same line number in case if no characters were typed.
- If CTRL-D is typed in a quad-input, it returns a null value in case of strings and zero in case of numbers.

If some characters were typed, instead, the CTRL-D is ignored.

The **Del** `\ /` operator is better described in the next chapter "The internal editor".

The **Trace** `T / \` operator is better described in the next chapter "Tracing".

The **Stop** `S / \` operator is better described in the next chapter "Stop control".

4. PROGRAMMING IN APL

APL programs are simply list of expressions, with the added ability to jump to specific program lines, identified by their line number or by an alphanumeric label. They can contain any expression, and even call other programs in any form, or any depth.

There are two methods for writing programs in APL:

- with an external editor; in this case the program is written in a textual file, with extension `.apl` (one file for one program is the law); it can then be imported in a workspace with the `) IMPORT` command, and also saved to file from within a session with the `) EXPORT` command; the program in the file must have as the first not commented line the complete header (along with the `\ /` (*del*) symbol preceding the header); the closing `\ /` is not required, but necessary if the program is followed by the documentation variable assignment (see also `) IMPORT` for the rules about it).
- with the internal editor, whose rules are exposed in the next paragraph. In this case, the program can be later saved to file with the `) EXPORT` command, with identical results as in the previous case. The documentation variable must be built from console in order to be found by `) EXPORT`.

4.1. The internal editor

The internal editor is activated by means of the command `\ /`, followed by the header of the program; the procedure lets the programmer to insert in memory a program, with the techniques exposed later, which adhere mostly to the rules of the APL69.

4.2. The programming elements

The header is somehow the most complicated step in writing programs in APL; let's summarize its form.

Programs (also defined as *functions*, in APL) may have 2, 1 or 0 explicit arguments, and either with or without an explicit result. The form of the header used to define each of these six types is shown in table 4.1. Each of the six forms permits the appending of semicolons and names to introduce local variables. The names appearing in any one header must all be distinct; e.g. the header `Z<-F Z` is invalid.

Number of arguments	Number of results	
	0	1
0	<code>\ /F</code>	<code>\ /Z<-F</code>
1	<code>\ /F Y</code>	<code>\ /Z<-F Y</code>
2	<code>\ /X F Y</code>	<code>\ /Z<-X F Y</code>

Table 4.1: Form of defined functions

It's not mandatory either for the arguments of a defined function to be used within the body, or for the result variable to be specified. A function definition which does not assign a value to the result variable will engender a value error report when it's used within a compound expression. This behaviour permits a function to be defined with a restricted domain, by testing the argument(s) and branching out in certain cases without specifying a result. See for example the following `apple` session excerpt:

```

\ /Z<-SQRT X
[1]  ->0xiX<0
[2]  Z<-X*.5
      SQRT 16
4
      SQRT _16 VALUE ERROR SQRT[1] ->0xiX<0
      ^

```

To explain in detail the header properties, the widest and most complete form is the following (the `\ /` substitutes the

∇ , called *del*, used by the IBM APL\1130):

```
\ / Z<-X NAME Y;AUS1;AUS2;AUS3
```

This specifies a dyadic form of the user programs, with the following components:

- Z is a dummy variable that represents the return value; a final assignment to Z (of course Z can be any legal name) is also the return value of the user procedure; and since there is a return value, the user program can be used in any expression in dyadic form.
- X is a dummy variable that represents the left value; when the user program starts, X is assigned the left value and in such form it can be read and re-assigned.
- NAME is the name of the user procedure; this is the name that can be used in any expression, in this case enclosed in two values because it's a dyadic procedure.
- Y is another dummy variable that represents the right value; when the user program starts, Y is assigned the right value and in such form it can be read and re-assigned.
- the semicolon divides the proper dummy and the local variables listing.
- AUS1, AUS2... are proper auxiliary local variables that are created during the user program execution and removed at the end; their utility is to hide other global variables with the same name, and protect them. When the user program ends, they become visible again. This part is completely optional, and all variables names but last are separated by semicolons.

To clarify the state of *global* vs *local* variables, the rules that govern the state of variables are here enlisted:

- All variables declared from console are global. As such, they are available to all programs, at any call level, in the same workspace.
- A variable is local if it appears in the procedure header. Then, it exists only for the duration of the execution of the function itself.
- Whenever a variable is instantiated within the body of a function, and it's not local to that function (i.e., not appearing in the header), it refers to the local meaning of the most recently called function whose execution was started earlier than this one, but hasn't yet been completed. If there aren't any recently called function (this happens for the function called directly from console), the global meaning is understood.

Other header forms are possible, with some missing features that alter the header property; including the previous, and omitting the local variables, which insertion is always possible, the complete list follows:

Z<-X NAME Y

All dummy variables are present, and this characterises a dyadic program with two arguments; this can be used in expressions, in the same manner of any dyadic operator.

X NAME Y

The missing return value characterises a standalone program with two arguments; this cannot be used in expressions, because there is no return value.

Z<-NAME Y

The missing left value characterises a monadic user program with a right argument; this can be used in expressions, in the same manner of any monadic operator.

NAME Y

The missing return and left value characterises a monadic user program with a right arguments; this cannot be used in expressions, because there is no return value.

Z<-NAME

The missing left and right parts characterises a standalone program that needs no arguments but that returns a value; this can be used in expressions, wherever a number is expected.

NAME

The missing left and right parts and the missing return value characterises a standalone user program that needs no arguments; this cannot be used in expressions, because there is no return value.

From another point of view:

one argument - NAME

This unique datum is the name of the standalone⁴¹ argument-less user program

two plain arguments - NAME Y

The first is the name of the monadic user program, the second is the argument

two arguments, the first assigned - Z<-NAME

The assigned first is the return value, the second is the name of the argument-less user program

three plain arguments - X NAME Y

The first is the left argument, the second is the name of the dyadic user program, the third is the right argument

three arguments, the first assigned - Z<-NAME Y

The assigned first is the return value, the second is the name of the monadic user program, the third is the right argument

four arguments, the first assigned - Z<-X NAME Y

The assigned first is the return value, the second is the left argument, the third is the name of the dyadic user program, the fourth is the right argument

Another perspective shows what programs can represent:

standalone without return value - NAME

This program acts like a console command without arguments.

standalone with return value - Z<-NAME

This program acts like a variable, that is it returns a value that can be used into an expression.

monadic without return value - NAME Y

This program acts like a console command with an argument.

monadic with return value - Z<-NAME Y

This program acts like a monadic operator, that is applied to the entity on the right.

dyadic without return value - X NAME Y

This program can only be run from console with two arguments, the first on the left.

dyadic with return value - Z<-X NAME Y

This program acts like a dyadic operator, that is applied to the entity on the left and on the right.

The IBM APL\1130 rules requires that the name of the user program is preceded and followed by blanks when used into expressions; **apple** ignores all leading blanks, but requires the name must be followed by at least one blank; the reason is this: suppose that A is the name of a user monadic function; the following request:

A1

is error-prone, because the missing blank after A forces **apple** to retrieve a variable with name A1, and if it should exist, a (possibly) wrong result is returned; independently of the fact it exists or not, the result is not what you think; so the space is mandatory:

A 1

In this case, A is found as a monadic procedure, and 1 is its argument.

⁴¹ Notice that the DEC80 classifies such program as *niladic*, but I stick to *standalone* because it's a more modern name.

4.3. Program editing

After specifying the header, the editor shows the number of the first available empty line in the program, enclosed in square brackets. If this is the first editing, the number is 1 (the first line is numbered from 1), but if this is not the first editing and supposing the program has 6 lines, the number is 7, because the editor assumes that you want to add lines to the existing (possibly empty) program.

At this point, the programmer can type several things:

[| _ |]

This command instructs the editor to print the complete numbered listing of the program so far; the numbers are all consecutive integers starting from 1.

[N | _ |]

This command instructs the editor to print the listing of line N and start editing it.

[N | _ | M]

This command instructs the editor to print the listing of the program from line N to line M, and start editing the next line after the last shown line.

[| _ | M]

This command instructs the editor to print the listing of the program from line M to the end, and start editing the next line after the last.

<line of text>

a normal programming line instructs the editor to store it at the indicated numbered position.

[I]

This command, followed by nothing, instructs the editor to remove the line whose number is I.

[I] <line of text>

This command, followed by a programming line, instructs the editor to overwrite the line whose number is I.

[X] <line of text>

This command, followed by a programming line, and if X is a decimal number, instructs the editor to place it amid the two enclosing integers; for instance, typing [1.5] <line> instructs the editor to position the line amid lines 1 and 2.

\ /

This command alone exits the editor and saves the program lines to the workspace. It can also be typed as the closing sequence of the last line.

Typing Enter when a line number is presented, and thus inputting an empty string, would cause the deletion of that line, if present.

The command [N | _ | M] (originally used to edit line N from character position M) lists the lines from N to M included⁴².

The arguments of a programming line may be any valid APL statement, assignment, branching or any system command, with its proper arguments. If) OFF is used in a programming line, this will cause the exiting from the program and also from the **apple** session. (To ensure only exiting from the program, use ->0, that is *go-to-line-zero*.)

After typing a line, the next line proposed by the editor is calculated from the rightmost value of the last insertion place; so, in case last line was 2, the next will be 3, and in case last line was 1.5, the next will be 1.6. This can be tricky, for the first times, because at every change of step, all lines are not renumbered. For instance, here is a (quite useless) test sequence:

```

      \ / ATTEMPT
[1]      12

```

⁴² This is another difference from the original APL\1130.


```
[2] 23
[3] 34
[4] [1.5]555
[1.6] 666
[1.7] 777
[1.8] [4]45
[5] 56
[6] [2.5]252525
[2.6] [ | _ | ]
  \ / ATTEMPT
[1] 12
[1.5] 555
[1.6] 666
[1.7] 777
[2] 23
[2.5] 252525
[3] 34
[4] 45
[5] 56
  \ /
[6] \ /
```

The starting sequence inserts lines 12, 23 and 34 at lines 1, 2 and 3 respectively. The second sequence inserts lines between 1 and 2, so that now 555, 666 and 777 become the 'new' lines 2, 3 and 4, and the original are migrated to positions 5 and 6; next, a new value is added between 2 and 3, so the original 34 value is pushed ahead to position 7; the request for a complete listing during the program definition does not renumber the lines, and at the end of listing, it shows the next number from the higher position; the final \/, instead, produces the effect of resetting the line numbers to a continuous list starting from 1:

```
  \ / ATTEMPT
[1] 12
[2] 555
[3] 666
[4] 777
[5] 23
[6] 252525
[7] 34
[8] 45
[9] 56
  \ /
```

If an error is detected (mainly, it depends on not closing brackets), the message "EDITING SYNTAX ERROR" appears, and the editor prints the current line number and re-ask the input. See the following example, where we want to add a line between the 5th and the 6th:

```
  \ / GCD
[7] [ | _ |
EDITING SYNTAX ERROR
[7] [4.1 | _ |
EDITING SYNTAX ERROR
[4.1]
```

The first error, during the request of a complete listing, prints the error message, and then re-asks the 7th line; when

line 4 . 1 is chosen for the listing, but an error still persists (second error message), the editor shows the new number 4 . 1 and asks for the new input.

The same in the following case:

```
      \ / GCD
[7]   [ | _ | 4
[4]   N < - G
[5]   M , N
[6]   END : -> START
EDITING SYNTAX ERROR
[7]
```

In this case, the required partial listing is performed the same, and the message appears after the listing; then the editor predisposes for a 'new' line 7 to be added to the listing.

Editing of the header. Whenever the user specifies a command for editing the line [0], followed by a text, the interpreter assumes that the editing object is the program header. (The line zero is not a valid programming line.) After the [0], a new complete header may be specified, that will substitute the previous one under all aspects, subject to the same rules specified earlier in this manual.

Adding casual numbers. Suppose a final line must be entered in an editing program, but the programmer does not remember how many lines the program has. The [| _ |] listing command may help, but if new intermediate lines were added in the process, the total could not be easy to calculate. At this point, the programmer may simply specify a huge number, a number which is surely higher than any other number used so far, which is added to the count; for example, if the program is short, the line:

```
.....
[60]'End of program'
```

is surely beyond the last; actually, whenever the user specifies an exaggerated number in the editing line, as shown, the line is simply added to the count (as last, obviously), and renumbered when the editing is closed. In this manner, the sequence is guaranteed to be ordered in a user-specified order, whichever number is used as specifier.

4.4. Local and global variables

The terms local and global are defined in the APL69, as follows (pages 29-30):

Local and global significance. In the execution of defined functions, it's often necessary to work with intermediate results which have no significance either before or after the function is used. To avoid cluttering the workspace with a multitude of variables introduced for such transient purposes, and to allow greater freedom in the choice of names, the function definition process [...] provides a facility for designating certain variables as local to the function begin defined. Variables not so designated, and all functions, are said to be global.

[...] During the execution of a defined function, a local variable will supersede a function or global variable of the same name, temporarily excluding it from use. If the execution of a function is interrupted (leaving it either suspended, or pendent, [...]) the local variables retain their dominant position, during the execution of subsequent APL operations, until such time as the halted function is completed. System commands, however, continue to reference the global homonyms of local variables under these circumstances.

Let's examine two cases in the APL69 that involve the creation of local and global variables.

Case 1

At page 63, the two programs SUM3 and SUM2 are used to contextualize the essence of a global variable,

because the variable *S*, which is used by both programs, is never declared as local (like *I* in SUM3). The two programs are here copied for clarity as if input manually:

<pre> \ / SUM3; I [1] S<-0 [2] I<-1 [3] S<-S+I [4] I<-I+1 [5] ->3x i I<=N [6] \ / I<-20 N<-5 SUM3 S 15 I 20 </pre>	<pre> \ / SUM2 [1] S<-0 [2] I<-0 [3] S<-S+I [4] I<-I+1 [5] ->3x i I<=N [6] \ / I<-20 N<-5 SUM2 S 15 I 6 </pre>
--	--

As seen, *S* remains available when both programs end, because in both cases *S* is printed with success. Not so for the variable *I*, which is local only to SUM3; so, when SUM3 is run, it uses its local *I* for the counter, and leaves the global *I* untouched, while the execution of SUM2 makes use of the global value itself, and thus altering its content.

Conclusion: The expression *S<-0* has caused the creation of a **global variable**.

Case 2

At page 97, the APL69 reports the program BIN, here copied for clarity as if input manually:

```

\ / BIN [ | _ | ] \ /
[1] LA: Z<-(Z,0)+0,Z
[2] ->LAxN>=pZ
[3] \ /

```

This program, launched as BIN 3, has an intentional bug: it refers, in the assignment expression *Z<-(Z,0)+0,Z*, to a non-existent variable. Actually, the result variable *Z* has not been declared already, and no other global instances of another variable *Z* is found. This causes the generation of a value error.

A first *cure* in the same text requires the addition of this variable manually:

```
Z<-1
```

so that now BIN 3 completes regularly. But a subsequent re-run with BIN 4 suffers from the same value error, because *Z* is not found again.

Conclusion: the expression *Z<-1* typed after a program break, and which has the same name of the return value of BIN, has caused the creation of a **local variable**, created during a program interruption (even manually) under the circumstances of the existing program state.

If, after assigning *Z<-*, the programmer would type this:

```
Q<- 'ISOLATE'
```

this variable would not refer to the program state, even if created during a program break. So, at present, *Q* is created as a global variable.

The DEC APL manual, at page 3-3, gives a more compact definition for global variables:

"3.2.2.3 Global Variables - Variables that have essentially the same significance inside and outside a function definition are considered *global variables*".

Let's confront the previous declaration with the latter definition:

- Z has a significance inside the program (it's the result variable), but not outside, hence it's local.
- Q has the same significance inside and outside the program, hence it's global.

The matter is also explained at page 179 of the APL68, you are invited to check.

Local variables declaration: how apple works. As long as a local variable is one that is mentioned in the header of a function, one can wonder when they are declared, in order to know which may or may not be present after an interrupt.

apple behaves this way:

- All local variables, except the return value variable, are declared when the function is invoked, before the first line is even read, so that the execution of the function can dispose of all the local variables.

The return value variable, the one that is assigned in the header and that represents the return value of the function, the same that permits the function to be used into any math expression, is not declared before the first line, but is declared afterwards, during the program execution, when it's explicitly assigned. This is done for several reasons:

- * the first is that the assigning expression cannot be influenced by the previously assigned return value variable, because this may interfere with the existing variables. Suppose in fact what the BIN program previously described would be designed as such to make use of a global variable Z which influences the first assignment only (because next assignments would find the new local Z); this design would not work.
- * the second is that it's not clear what would be the assigned value *before* the start of the program; actually, if it would be set to the scalar 0, this may not be correct if the function would return strings. Or vice-versa. And, even setting it as the empty vector could be not the expected behaviour, in case of anticipated termination.
- * the third reason is that pre-assigning the return value variable would contradict the statements in all APL texts asserting that is an error not instantiating the return value variable before the end, and this may also be an intended effect (see for instance the SQRT program defined elsewhere in this manual). Pre-assigning the return value variable would ignore this effect.

So, in case of a suspended execution, the only local variable that the user can install manually is the return value variable, because argument variables (the left and right arguments), the proper local variables listed in the queue of the header, and even all the variables that represents the labels, are all created before the program starts, and they don't need to be re-installed.

Suspended or pendent execution: how apple works. The APL69 used two different terms to define an interrupted program: a program that breaks up with an error message is suspended, because it has found a problem that makes impossible to continue; if this program was started by another program that called it, then also this program is stopped, but it's defined as pendent, because no problem has happened in it, and it's still waiting for the other program to complete.

4.5. Usage of undeclared variables

Though not explicitly reported on the APL69, it's required⁴³ that a statement like:

```
I<-I+1
```

generates an error if *I* was not previously declared, because the analysis proceeds from right, and the first required operation is *I*+1, which fails. Things are different in other programming languages, where the analysis proceeds from left; for instance, in BASIC, this statement:

```
10 LET I=I+1
```

bears no problem, because in BASIC (having no need for declaring and initializing variables), in case a new variable is required, it's automatically declared and initialized to zero. In the previous example, the first element of the expression is *I* [=], which causes the implicit declaration of *I*=0; then, after the equal sign, comes the expression *I*+1 which, at this point is perfectly legal.

4.6. Branching

Statements in a function are normally executed in the order indicated by the statement numbers, and execution terminates at the end of the last statement in the sequence. This normal order can be modified by branches. Branches make possible the construction of iterative procedures, and with the possibility to use variables, this makes APL a real programming language, and not a mere procedural instruction coder.

The expression `->4` denotes a branch to statement 4 and causes statement 4 to be executed next. In general, the right-arrow may be followed by any expression; if the result of this expression is not an integer, its integer part is taken and if not a scalar, the element in position `[1;1]` is taken and used as a jump target. This value is the number of the statement to be executed next. If this value is lesser than 1 or greater than the last statement in the program, the execution is terminated (a jump to zero `->0` is a common way to exit any program).

If the result of the expression is the empty vector, the jump is ignored and execution continues with the next statement.

Whenever an error or an interrupt stops the execution of a program, the variables state is frozen, to let the programmer inspect the variables content. At this point, new variables can be created (and they will be inserted in the global space), or existing variables can be changed, and then the user can:

- Recall the original expression that caused the trouble (and in this case the variables are reset, keeping only the global variables, and the program data are also reset); your only care is to test if the wrong value that caused the error is in the source, causing the failure to happen again.
- Use the right arrow `->` followed by zero to wipe out and clear the program state, so that a subsequent re-run is possible.

As an example, here is an excerpt of a trivial execution; the program `CALC` does some calculations, and in doing this, it calls the program `DATA`, which needs a global existing variable `UU` in order to return a value to `CALC`, but failing because `UU` does not exist:

```
) IMPORT CALC
CALC IMPORTED
) IMPORT DATA
DATA IMPORTED
) LIST CALC
\ / Z<-CALC; Y; T
```

⁴³ See APL69, page 97, and look for the `BIN` example.

```
[1]    Y<-56
[2]    T<-Y*2
[3]    Z<-DATA +T
      \ /
        ) LIST DATA
      \ / W<-DATA
[1]    W<-UU*2
      \ /
        CALC
VALUE ERROR
DATA[1] W<-UU*2
      ^
```

Instantiating the missing variable and then re-running CALC from scratch will solve the problem:

```
      UU<-23
      CALC
3665
      ) VARS
UU
```

The result can be calculated and returned because UU was found, this time. Whenever you run a program from start, the variables are reset to global (i.e. all the local variables of the previous execution are deleted). At the end, as you can see, only variable UU remains, because it's a global variable.

In apple, the command -> to start the program from the stopped position is not available, so that a re-run is always necessary in case of unwanted breaks.

4.7. Labels

Labels in APL are very peculiar. Whenever a statement is of the form:

```
[4]    LABEL: <expr>
```

is met, the parsing of the statement itself proceeds as follows:

- firstly the label is removed, because it's not part of the expression <expr>;
- then a local variable with the name of the label is created, setting it equal to the line number where the label appears. In the example, LABEL is a variable containing the scalar 4. Before saving the memory, all labels are readdressed, to point always to their proper lines;
- The <expr> is evaluated as usual.

Whenever a jump to line 4 is to be performed, it suffices to type ->LABEL, without bothering to remember the line number.

Labels are a particular case of *local variables*.

4.8. Tracing

A trace is an automatic type-out of information generated by the execution of a function as it progresses. In a complete trace of a function P, the number of each statement executed is typed out in brackets, preceded by the function name P and followed by the final value produced by the statement. The trace is useful in analyzing the behaviour of a defined function, particularly during its design. The tracing of P is controlled by the trace vector for P, denoted T/\P, which can be regularly assigned, though it can't be listed with) VARS. For example, typing T/\P<-2 3 5, the lines 2, 3 and 5 will be traced in any subsequent execution of P. Typing T/\P<-0 will disable the tracing of P. More generally, the value assigned to the trace vector may be any vector or scalar (integer). A complete trace of P is

set by entering $T/\backslash P \leftarrow -iS$, where S is the number of statements in P .

For example take a look at this session excerpt:

```

      \ / Z<-FAC N; I
[1] Z<-1
[2] I<-0
[3] L1:   I<-I+1
[4] ->0xI>N
[5] Z<-ZxI
[6] ->L1
[7] \ /
      FAC 5
120
      FAC 6
720
      FAC 3
6
      T/\FAC<-3 5
      X<-FAC 3

      X<-FAC 3
FAC[3] 1
FAC[5] 1
FAC[3] 2
FAC[5] 2
FAC[3] 3
FAC[5] 6
FAC[3] 4
      X
6
      T/\FAC<-0

```

Note: the tracing command is not programmable.

4.9. Stop control

The Stop vector⁴⁴ for a function P is denoted by $S/\backslash P$. It's set in the same manner as the trace vector (i.e., by $S/\backslash P \leftarrow -I$, where the vector I specifies the numbers of the statements controlled), and stops execution just before each of the specified statements. At each stop, the function name and the line number of the statement next to be executed are printed. After the stop, the system is in the normal suspended state; resumption of execution may therefore be initiated by a branch. Typing $S/\backslash P \leftarrow -0$ will reset the stop vector, and a subsequent execution of the program will normally continue until its natural end.

Note: the stopping command is not programmable.

Note: the stopping command determines the stop of execution of P , and when the reprise is commanded (by $->$) the execution restarts; this process ignores the possible assignment that were requested as the first command; for instance:

```

      S/P<-3
      P
P [3]
      ->

```

⁴⁴ This function is not described in APL69, but in APL70; I therefore decided to import it into **apple** due to its extraordinary usefulness.

```
P [3]
->
24
```

(We suppose that `P` returns 24). This is a regular stopped and reprised execution; but if we has assigned it to, say, `Q`:

```
S/P<-3
Q<-P
DEFN ERROR
  Q<-P
  ^
```

a definition error is printed, because the reprise of the process cannot re-instate the assignment, that would be forgot. So, to assign `Q` with the result of `P`, first emend `P`, and make it a working function, and then:

```
S/P<-0
Q<-P
```

The removal of the stopping control makes `P` regularly assignable.

4.10. Scripting

`apple` uses scripting in two ways: in output, by means of the option `--script`, and in input using the characteristics of the shell. Let's review both of them.

The **Output scripting** is performed through option `--script`. This option enables the *session recording*, that is everything that is typed or printed on the screen will also be flushed to the scripting file. The scripting file is thus a *memory* of the session, that can be used to record, re-elaborate or print the mathematical results. The script is a textual file that is accepted in all kind of word processors.

The **Input scripting** exploits the shell input operator `<` to send all lines of a textual file to `apple`; each line of the textual file is the same you would type from a session (so not the "results" of calculations, but the "commands" that produce those results). Even programs may be input (and in any number) provided they are all begun with a correct header (`\ / NAME`) and terminated with `\ /`, to inform the editor that the function is ended and to store it. This method is straightforwardly useful, see for instance the following example; The input file is `irrat.txt` and is started with:

```
$ apple --clear < irrat.txt
```

When `apple` terminates the file reading, it will also terminate the session, so that any result should be saved to file, better to workspace.

Caution! It's strongly advisable, in such operations, to use the option `--clear` to ensure a complete reset of the workspace; otherwise, the variables and programs of the script can interfere with the new programs you are loading. You can omit this option only if you are sure that variables and programs do not overlap. If you encounter strange behaviours, consider adding this option to your command line.

4.11. Style guidelines

Writing programs in APL may be a difficult task, specially for those who are used to the classic imperative programming of C or Pascal or BASIC. What I feel to say is that a correct writing, following definite guidelines, may help in writing programs that are readable.

What I present here is a series of the guidelines that I used in writing the programs, guidelines that aim to be easy to follow but that I don't force you to follow, you are to free to use your own.

These guidelines are related to:

- results output
- variable assignment
 local variable
- cycles and go-to
- comments
- documentation variable

Results output. In general, any program that is marked to return a result must not be forced to print anything, apart from some intermediate text, it's sufficient to assign the return variable, and the simple program invocation, with or without arguments, will print the result.

Different from the previous is the case of a program that is marked as to not return anything. In this case is bad behaviour (in my opinion) letting it execute in silence, and possibly assigning some global variables that will be available in the workspace after the program termination. My conviction is that any program that execute silently does not inform the programmer in any sense; at least a 'Done .' before exiting is the least that can be done. In general it's not good practice using such programs; it's not that I ban the global variables, I ban the fact the program does not return anything, so I always assure that a) a value is returned (it may be the most important global variable that is changed to a return value, or it may be the number of iterations, or it may be a flag, or any other value that is up to you); b) in case no return value is possible or useful, provide a means to print the results, the process, the intermediate calculations, and so forth.

Variable assignment. The assignment of variables is one of the most important feature of any language; in APL all initialization variables should be put together, and glued as in

```
A<-1+C<-B<-0
```

an operation that instantiates A, B and C in one turn, or in

```
A<-0 1 ; B<-0 ; C<- 'NULL'
```

where the three instructions are lined, so that they occupy less lines of code.

An advice is to test all the initialization part on its own, to get sure the assigned values are the right values.

Local Variables. A strong advice that I always try to follow is the following: apart from the values that you really mean to leave back as global variables, **don't create service variables that are not marked as local**; this clutters up the workspace and soon you won't know what the hell that variable was for anymore...

So start your test with a quasi-clean workspace, in which you declare only the variables that will serve for the test; after the execution, the state of variables must be conserved; if some newly created variables appear, they were created by the program and never deleted, so go to the sources, and add them to the header. The next execution will remove these instances at the program termination, to preserve the workspace.

5. PROGRAMMING TECHNIQUES

In the following I gather some subjects that help in completing an APL programmer formation. Let me know if something is not clear, and as usual, write to

<ing dot antonio dot maschio at gmail dot com>

for any help.

5.1. On the mathematical precision

The precision achieved by **apple** in performing calculation is quite good: for a single operation, the results obtained are precise at least up to the 15th decimal place, perhaps more.

The problem, in computer calculation, is that repetitions of calculations accumulate the error, which sooner or later will reduce the precision range to an undesirable state.

For example, consider the following snippet, which reports a single logarithmic calculation (I use the IBeam variable I_2 instead of) DIGIT because I need a wider range of possibilities):

```
I_2<-17
2.3&4.5
1.8058121292705116
2.3*(2.3&4.5)
4.5000000000000009
I_2<-15
2.3*(2.3&4.5)
4.5
```

The precision of the exponential $2.3^{\log_{2.3}(4.5)}$ is quite good, and it reaches the 16th digit; so, using 15 as the parameter of I_2, the output is a plain 4.5.

But if there are repetitions, the precision is reduced at every turn; for instance, see the following matrix inversion:

```
M<-3 3pi9
M[3;3]<-_9
M

1 2 3
4 5 6
7 8 _9
I_2<-16
)FUZZ 16
WAS 12
| % | M

_1.722222222222222 0.7777777777777778 _0.05555555555555555
1.444444444444444 _0.5555555555555556 0.1111111111111111
_0.0555555555555555 0.1111111111111111 _0.05555555555555555

M+.x | % | M

1 _1.110223024625157E_16 0
0 1 0
_1.77635683940025E_15 0 1
```

The more repetitions there are (in an inversion process they occur many times), the greater the accumulated error, which progressively reduces the precision involved; in the example you can see that, in building the identity matrix by multiplying a matrix by its inverse, a pair of values are not actually zero (at least from the 15th digit on); so, the precision does not go beyond the 14th digit and in some cases it can be even worst.

5.2. On the form of the jump condition

Often, a jump condition appears as this:

```
->NxiX<0
```

Let's separate the various components.

```
->    N    x    i    X<0
```

- 1 Let's start from the bottom. The $X<0$ condition is the proper condition that causes the jump to be performed when it's true; so that if, for instance, $X=_4$, the condition is true (otherwise it's false). It can be any expression that returns a true/false value as a scalar, where 0 is false, and true elsewhere.
- 2 The second item is the monadic ι (iota) operator, which, in case of an argument greater than zero, returns the index list from 1 up to that index; if the argument is zero, $\iota 0$ returns an empty vector.
- 3 The third item is the multiplier; leave it here, for the moment.
- 4 The fourth item is the value of the jump, N in our example. N may be any integer, and if zero or if greater than the higher line number of the program, the program ends, otherwise N is the destination of the jump. If N is any array, the item in position $[1; 1]$ is taken as the address. As seen previously, the second item is either a list vector (of length greater than or equal to 1) or an empty vector. Now:
 - If the second item is a list, it's multiplied by N , giving a list whose element in position $[1; 1]$ is taken (and which is 1); the result means *jump to N*.
 - If the second item is an empty vector, the multiplication of an empty vector by any value returns an empty vector; the result means *ignore the jump*.

The key figure $\times i$ is called, in APL jargon, *if and only if*; let's examine the translated request:

```
->N if and only if X<0
```

which turns to be, in English: *jump to N if and only if X is lower than zero*.

Caution! The result on the right of ι (iota) must be a positive value, or zero; in case it should yield a negative value, this ends up in a domain error issued by iota, and the program stops. See for example the following program:

```
    )LIST DUMMY
    \ / DUMMY
[1]   A<=_5
[2]   ->4xiA
[3]   ->0
[4]   'Regular end'
    \ /
    DUMMY
DOMAIN ERROR
DUMMY[2] ->4xiA
```

In this case, the jump in line 2 calculates ι_5 , which returns domain error, and the program stops abruptly.

Another expression for the jump is the following:

->X<0/N

Let's again separate the various components.

-> X<0 / N

- 1 Now let's start from the beginning. The $X<0$ expression is the proper condition that causes the jump to be performed when it's true; again, if for instance $X=_4$, the condition is true (otherwise it's false). It can be any expression that returns 1 (or any positive integer) in case of jump and 0 in case of no jumps.
- 2 The second item is the compression operator / which, in case of a left argument different from zero, replicates the item on the right for the indicated times, otherwise, when zero, returns the empty vector.
- 3 The third item is the value of the jump, N in our example. N may be any integer, and if zero or if greater than the higher line number of the program, the program ends, otherwise N is the destination of the jump. If N is an array, the item in position [1;1] is taken as the address. So:
 - If the condition result is 0, the compression of N returns the empty vector; the result means ignore the jump;
 - If the condition result is any integer different from zero, N is transformed into a list of Ns, and the value at position [1] is therefore N, and this is taken as the address to jump to. In particular, if the condition result is 1, then the compression returns the plain N value.

This kind of jump is called *jump if to*; let's examine the translated request:

-> if X<0 -> N

which turns out to be, in English: *jump, if X is lower than zero, to N*

Watch out! If the condition result is negative, the compression returns zero, equivalent to the zero case; so, if you mean to evaluate a mathematical condition, avoid the case non-zero/zero, and use the case positive/non-positive.

5.3. On console execution

The console execution can be performed in various ways. Let's see how.

5.3.1. Argument-less execution

In case the program in file `prog.apl` is regular and has no arguments, it can be called from console like this:

```
$ apple prog.apl
```

This instruction runs programs written under the following conditions:

- The program `prog.apl` must be an argument-less program; the typical header is plain: `\ / PROG`
- The return value is useless, and can be avoided; if not, it will be treated as a simple variable in the source
- The program must contain in itself the output, that is the result variable (along with all the needed strings) must be **explicitly** invoked at the program termination
- Recursive programs have a drawback: each instance shall print its result, so that the input in such cases, includes all the results of the recursion. Of course, the useful one is the last

5.3.2. Execution with arguments

In case the program in file `prog.apl` is regular and has left or right arguments, it can be called from console like this:

```
$ apple --args=<str> prog.apl
```

This instruction runs programs written under the following conditions:

- The program `prog.apl` must contain the arguments as specified in the `--args=` option; the typical header is complex: `\ / PROG M` in case of right argument `M` or `\ / N PROG M` in case of left argument `N` and right argument `M`; in `<str>` the first case is `<str>=M_value`, the second case is `<str>=N_value;M_value`, where `N_value` and `M_value` are the arguments of the program.
- The return value is useless, and can be avoided; if not, it will be treated as a simple variable in the source
- The program must contain in itself the output, that is the result variable (along with all the needed strings) must be **explicitly** invoked at the program termination
- Again, recursive programs have a drawback: each instance shall print its result, so that the input in such cases, includes all the results of the recursion. Of course, the useful one is the last

5.3.3. Indirect execution

In case the file `prog.apl` is a simple list of APL expressions (not a program), and these expressions are the same that can be typed on console, it can be called from console like this:

```
$ apple < prog.apl
```

This instruction runs such an indirected list of expressions under the following conditions:

- Any line of text is a console instruction;
- The output is the same that you obtain by typing the very same commands on console.

This indirect execution is a sort of script-engine but, since it uses the current workspace, the user should invoke also the `--clean` option because, if you build a function in the called file, and the function exists in the workspace, the effect is to add instruction to the existing program, with unpredictable results.

5.4. On program abrupt stops

Generally speaking, a running program may irregularly stop for the following reasons:

- A) after an error (with message)
- B) after a commanded stop (`S/\fR`)
- C) after a segmentation error (undesirable)
- D) because of a mistake in the `apple` code (undesirable as well)

If the error described in case *A*) occurs, it can origin from:

- A.1) incorrect algorithms (apparently exact algorithms that, in certain situations, lead to various unjustified errors);
- A.2) incorrect expressions (e.g. division by zero, syntax error, unrecognized operators, uninitialized variables);
- A.3) incorrect logic (e.g. incorrect mathematical logic, missing lines, go-to incorrectly routed, nonconforming matrices dimensions).

If the stop is commanded, as in case *B*):

- B.1) it's the programmer's will, and this is not necessarily a program failure.

In all the previously cited cases, it's possible to perform the following operations:

⇒ Re-run the incorrect line, say *N*, with possible label `LAB1`, by:

- ◇ Correction of the error condition, if possible (for instance, by assigning or re-assigning a variable);
 - ◇ Typing `->` (without number) to restart from the stopped line *N*; please, observe that that in case of intentional stop, the stop occurs *before* the line execution;
 - ◇ Typing `->N` (explicit numeric redirection);
 - ◇ Typing `->LAB1` (explicit redirection to label).
- In case line *N* contains an error which is solvable only by changing the program, you step to a further place.
- ⇒ Run the same program starting from another line than the failing one, say *M*, with possible label LAB2, by:
- ◇ Typing `->M` (explicit numeric redirection);
 - ◇ Typing `->LAB2` (explicit redirection to label);
- In case of the repetition of the error in line *N* or in some other places, you may take actions to correct the failure, but you have now figured where the error occurs.
- ⇒ Re-run the program on new basis by:
- ◇ Correction of line *N* (with the internal editor or with an external editor followed by `) ERASE` and `) IMPORT`); this resets the program state, anyway;
 - ◇ Re-run the program from scratch, seeing if the error was resolved.
- ⇒ Terminate the program execution by:
- ◇ Typing `->0` or `) RESET` to reset the program state;
 - ◇ Providing then some modification in the source to resolve the error, for further analysis, to be performed later.

As seen earlier in this manual, the `->` command may be used in many flavours:

- a. `->`
without argument, to start execution from the stopped line (even if unknown to the user);
- b. `->N`
to continue execution from line *N*; if *N* is null or negative or greater than the program length, the program termination is commanded;
- c. `->LAB`
to execute from label LAB, which must be a legal program label, or a value error is returned;
- d. any non digit/non alphabetical upper character typed after `->` (ignoring possible blanks) returns a value error.

I underline here the fact that the program editing with the internal editor or the calling of the `) ERASE` / `) IMPORT` commands resets the program state if their argument is also a stopped program, so that after them, the continuation of an existent program instance is not possible. In other words, if after the stopping for failure the error is solvable by some variable make-up, the execution can be continued; if, instead, it requires changes in the program source code, the execution must be run from scratch if sources are altered.

The case C) is instead out of this discussion, because unsolvable. In this case, please, write to `ing dot antonio dot maschio at gmail dot com`, and tell me what you were doing and with which program/case.

The case D) deserves special comment: in case you cannot justify the error, and you believe the error is in the **apple** source code, send me the program listing and the command that you typed that caused the error, along with the variables that were present at that moment (the easiest way is to `) SAVE` the workspace and send it to me). If your assumptions get confirmed, I'll try to solve the problem and send you the new version; on the contrary, you'll be warned about your wrong assumptions.

5.5. On bitwise calculus

APL does not feature specific bitwise operators, but this is of little importance, since the remaining operators (and in specific, the encode/decode and the logical operators) can be excellent substitutes of those supposed missing bitwise characteristics.

Take for example the following operation, assuming we have a program called BAND (for **B**inary **A**ND) that can be used like this:

```
13 BAND 27
9
```

How to build such a program? Let's find out how.

The first operation in BAND is setting the Depth of the calculus, i.e. the Dimension of the numbers in bits; we can manage as many bits as we need, but once we have decided for one size, all numbers must fit that size; we select, for testing purposes, the 8-bit size:

```
D<-8p2
```

The second operation in BAND is changing the input values encoding them in base 2, using the mask D just created:

```
V1<-D`|`13
V1
0 0 0 0 1 1 0 1
V2<-D`|`27
V2
0 0 0 1 1 0 1 1
```

The third operation is applying the normal ^ ('and') operator (which acts element by element, so in this case, as to say, bit by bit):

```
V1^V2
0 0 0 1 0 0 0 1
```

The fourth (and last) operation is restoring the decimal format:

```
D_|_V1^V2
9
```

This is what we expected! The whole operations can be accomplished in one turn as this:

```
D_|_(D`|`13)^D`|`27
9
```

without any intermediate variable. This is so simple also because the 'and' operator can be replaced by other bitwise operators without changing the remainder; for example, here is the bitwise 'or':

```
D_|_(D`|`13)vD`|`27
31
```

and here is the bitwise 'xor':

```
D_|_(D`|`13)#D`|`27
22
```

The last operation uses the following truth table of 'xor':

```
A B XOR
----+----
```

1	1		0
1	0		1
0	1		1
0	0		0

from which we derive that the 'true' value occurs when A and B are *different*, and not equal. See also at page 18 where the logical apparatus is discussed.

Finally, here is the BAND program summarized:

```
\Z<-A BAND B;D
comment Binary AND
D<-32p2
Z<-D_ | _ (D ` | `A) ^D ` | `B
\/
```

Simple and effective. To generalize, I used `D<-32p2`, for 32-bit representations that can manage the full integer possibilities (to represent unsigned integers from 0 up to $2^{32}-1$, that is 4294967295, or signed integers from -2147483648 to 2147483647).

Here are some other programs:

```
\Z<-A BOR B;D
comment Binary OR
D<-32p2
Z<-D_ | _ (D ` | `A) vD ` | `B
\/
```

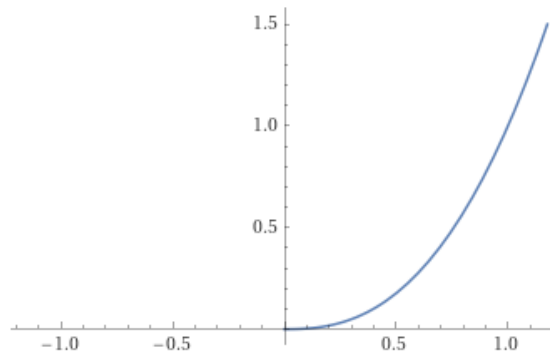
```
\Z<-A BXOR B;D
comment Binary XOR
D<-32p2
Z<-D_ | _ (D ` | `A) #D ` | `B
\/
```

5.6. On roots

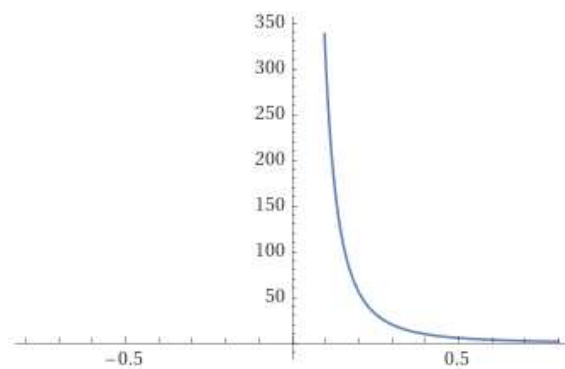
APL does not use a rooting operator, because the exponential operator is specialized enough to calculate roots, basing on the well-known formula $^y\sqrt{x} = x^{1/y}$, so that for instance, the square root of y can be obtained through $y^{1/2} = y^{0.5}$ in a very natural way. The above formula used for roots has the heavy conditions that $y \neq 0$ and that $x \geq 0$, which represent some limitations in the usage of the formula for retrieving roots, as will be discussed next.

In details, the exponential function, in its more generalized form, is of the type $y = x^b$, with $b \in \Re$ and $x \in \Re$: $x > 0$; if $b > 0$ it has the following graphic⁴⁵:

⁴⁵ Thanks to Wolfram Alpha for the graphics in this chapter.



If instead $b < 0$ the graphic is of the form:



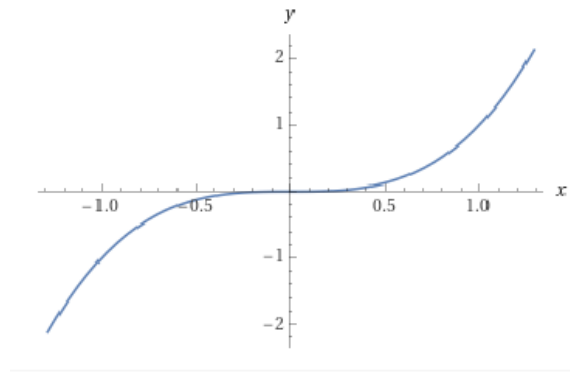
The calculation process must proceed as follows:

- if $x < 0$ then signal domain error (in $\Re$ there is no solution); this is evident looking at the above graphics.
- if $b = 0$, the result is 1.0, regardless of the argument x (and also including the case of $x = 0$).
- if $x = 0$ and $b < 0$ then signal domain error, as it's evident in the second graphic: actually, the figure 0^b with $b < 0$ is equivalent to $\frac{1}{0^{|b|}} = \frac{1}{0}$, which in $\Re$ does not return an acceptable value.
- else return x^b
- In case of the exponential used to retrieve roots, since the root index c is used to calculate the exponent by means of its reciprocal $b = 1/c$, we must add the obvious condition that $c \neq 0$.

If b is integer, the function $y = x^b$ becomes one of the Cartesian family, with the following properties:

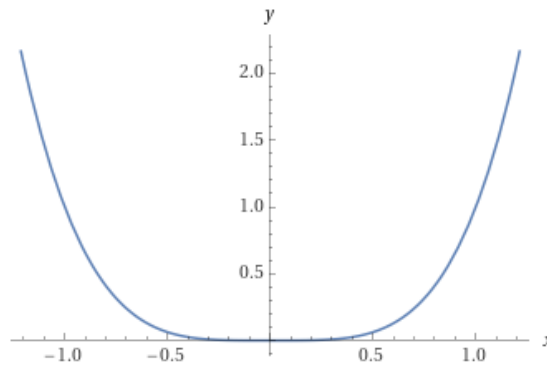
Odd exponent

This family comprises formulae that can be inverted (e.g. $y = x^3$ if inverted becomes simply $x = \sqrt[3]{y}$); the argument of this kind of root can be also negative, because the range of y is $\Re$.



Even exponent

This family comprises formulae that cannot be inverted because symmetric along the y axis, but we can limit the dominion to the positive side (e.g. $y = x^4$ can be inverted as $x = \sqrt[4]{y}$ but here y is defined only for $y \geq 0$). Anyway, the dominion of the exponential is $\Re$.



Considering all the previous facts, we must add some conditions for the calculations of the exponential and the roots:

- before invoking the exponential formula $y = x^b$, we should take into account that in case of integer exponent the result is $\text{sign}(x)|x|^b$, because the formula is valid for $x \in \Re$.
- before invoking the exponential formula $y = x^{1/c}$ for the calculation of the root, we should take into account that in case of integer-and-odd root index and negative argument, the result is $-|x|^{1/c}$, because the formula is valid for $x \in \Re$, and the check for $c \neq 0$ is also added.

From the practical point of view, the root-calculation is activated only if the sequence of operators $*\%$ is found in the expression and its right argument is an odd integer. This means that if the expression is

`_8*%3`

the root-calculation is applied (returning `_2`), but if you write:

`_8*0.33333333`

which is mathematically equivalent, a domain error is returned, because in this way the exponential laws are applied.

These points guided the design of the expression evaluator, that now responds to the imperative posed by professor Jeffrey Shallit (School of Computer Science, Waterloo ON) in his essay "*Why I'm Still Using APL*"⁴⁶ The observations posed by Mr. Shallit in the slide "*Getting Definitions Correct*" are that APL thinks mathematically in the right way, returning the following results:

```

      0*0
1
      _8*%3
_2
      +/i0
0
      x/i0
1

```

5.7. On the matrix diagonal

Let's suppose all elements of the diagonal of a certain matrix, square or not, are needed. And let's suppose the value n is the dimension of the square matrix, or the lesser of the dimensions of a non-square matrix. This task can be computed with the dyadic operator $\backslash\circ$, which starts from position 1,1 and gathers all items on the diagonal (going downright) until the n th element. Here are the possible schemes:

○ X X X X	○ X X	○ X X X X
X ○ X X X	X ○ X	X ○ X X X
X X ○ X X	X X ○	X X ○ X X
X X X ○ X	X X X	
X X X X ○	X X X	

The values identified by ○ are gathered to form a vector in all the possible matrix configurations shown, where n is 5 in the first case of a 5×5 matrix, 3 in case of a 5×3 matrix and again 3 in case of a 3×5 matrix. The operation on matrix M is:

```
1 1\oM
```

The returned value is a vector.

The opposite case, the population of the diagonal of a matrix, from a starting vector, cannot be satisfied by an APL operator. The reasoning lying behind is another. You know how to build an identity matrix? Here's how:

```
I<-4 4p1 0 0 0 0
```

This is the same of writing:

```
I<-4 4p1 0 0 0 0 1 0 0 0 0 1 0 0 0 0 1 0 0 0 0
```

because after the last zero, the cycle repeats from the first 1.

The last vector has the same collection of items of the following matrix:

```

1 0 0 0 0
1 0 0 0 0
1 0 0 0 0
1 0 0 0 0

```

which has the first column populated by the values that should be on the target diagonal, with, at its right side, a 4×4

⁴⁶ available here: <https://cs.uwaterloo.ca/~shallit/Talks/APL50-talk2.pdf>

zero matrix. This means that if this matrix is called for instance S, the declaration above becomes:

```
I<-4 4pS
```

So here's how we use this scheme, extending the reasoning to any values. If the vector of n dimension is V:

```
V
3.4 5.6 7.8 9.1
```

then we can write:

```
N<-4
M<-(N,N)pV,(N,N)p0
```

From right:

- (N,N)p0 is a 4×4 zero matrix

```
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
```

- V, (N,N)p0 is the matrix

```
2.3 0 0 0
5.6 0 0 0
7.8 0 0 0
9.1 0 0 0
```

The previous matrix is the source from which we take the values to populate the M matrix, in the same way we previously populated the identity matrix I. Now, suppose this is stored in S, as before. The command (N,N)pS is easy to understand.

- M<-(N,N)pV,(N,N)p0

The global command returns the final matrix:

```
2.3 0 0 0
0 5.6 0 0
0 0 7.8 0
0 0 0 9.1
```

which has the vector V in the diagonal, and zeroes elsewhere.

The case of an existing matrix, populated with values that must be retained and changed only along the diagonal is different. The values which don't lie in the diagonal must be returned unchanged. So we roll each row with the operator |o until the diagonal is on the left column, change that column with our vector and roll back each row to rebuild the matrix. Here's a worked out example:

```
M<-4 4pi16
M
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
V<-2.2 3.3 4.4 5.5
M<-(0,i3)o|M
M
1 2 3 4
```

```

6      7      8      5
11     12     9      10
16     13     14     15
      M[;1]<-V
      M

2.2      2      3      4
3.3      7      8      5
4.4     12     9      10
5.5     13     14     15
      M<-(0,-i3) o | M
      M

2.2      2      3      4
5      3.3      7      8
9      10     4.4     12
13     14     15     5.5

```

This is not an astonishing one-liner, but the best I can achieve. In short, the process is:

```

M<-(0,i3) o | M
M[;1]<-V
M<-(0,-i3) o | M

```

5.8. On recursion

Recursion may be the key of success, for an algorithm. Anyway, any recursion algorithm must observe the golden rule that the algorithm itself must converge, not diverge.

For instance, suppose that the monadic function FUN calls itself in such a way:

```
[4] RES<-FUN N-1
```

it's easy to see that FUN is trying to converge to zero, by progressively reducing the argument. As such, starting the program with:

```
FUN 40
```

Will end in 40 nested calls, but ultimately it will return a result.

But what if we call FUN like this?

```
FUN _4
```

This time FUN will start an endless recursion, because in no way subtracting 1 to a negative number would lead to 0, neither sooner nor later⁴⁷.

Moreover, what if we call FUN this way?

```
FUN 1000000000
```

Apparently, this is a converging succession. But there is a problem: the Operating System stack is required to store 1

⁴⁷ Well, of course this is true in the integer math world. In the integer computing world, going on subtracting one, when reaching -2147483648, one more subtraction will turn the integer to +2147483647. But I bet no computer can wait more than 2 billion suspended instructions...

billion suspended instructions... a task that is beyond any computer, I think.

Since I cannot preview the programmer's behaviour, and since distraction is common among us humans, I prevented segmentation errors in such cases by adding a program fence, and one more error message, which were not (as far as I know) on the IBM APL\1130 or the APL\360; see for instance the following snippet:

```
FUN _4
FN RECURSION LIMIT ERROR
FUN[4] RES<-FUN N-1
      ^
```

The recursion control was designed on basis that Linux grants up to 24 MB per thread, so that when the computer's stack structures reach this limit (which could cause a SIGSEGV-type interrupt), **apple** stops and discharges the program stacks and flags. This protective fence also acts on converging cases of larger dimensions. In this regard, the I-Beam variable `I_40` can be used to retrieve the last recursion level, useful in case of a recursion error, to see the physical limit of your computer.

Actually, the recursion limit involves the amount of memory dedicated to the system stack, which is not an **apple** limit; rather, when the program starts, the stack limit is increased to the maximum, and is restored when **apple** finishes, but even so, the system stack width is not infinite: when such an error is met, the program structures are removed, and the variables state is restored at the moment of the call, the recursive function itself and all the remaining pending ones are discharged and no pending execution results.

The programming techniques may or may not use recursion, and often there is a non-recursive technique for any recursion algorithm. This is often more acceptable, because recursion is a two-sided edges knife. But, anyway, recursion is anyway a good programming argument, so that now I am going to discuss the implementation of the factorials generation, using two different techniques, recursive and iterative.

5.8.1. The direct recursion

The first kind of recursion is the what is called the direct recursion. It of the kind:

$$\begin{aligned}\phi(0) &= N \\ \phi(n) &= f(\phi(n-1))\end{aligned}$$

This kind of algorithm converges only if the evaluation of the function goes towards $\phi(0)$ where the return value is not recursive; this guarantees to back-evaluate all the suspended calls and return a final value.

A first and famous example is the factorial recursive algorithm, contained in the following program `fact_rec.apl`:

```
\ / RES<-FACTORIAL N
-> (N=0) /RES1
RES<- (FACTORIAL N-1) xN
->0
RES1:RES<-1
\ /
```

Its philosophy is clear:

- if N is zero, return 1 (jump to the RES1 label);
- otherwise return $N \times (N - 1)!$

To execute the program from console, type what follows, (the `DIGIT` call is added to ensure the return value is

shown not in exponential form, at least up to 17 characters):

```
./apple --apl=")DIGIT 17" --args="6" fact_rec
WAS 0
720
```

The recursion key is in the call of FACTORIAL inside a FACTORIAL execution. If you trace the execution of this program, you will notice that the final execution is differed:

```
./apple --apl=")DIGIT 16" --args="6" --trace="i5" factorial.apl
WAS 0
FACTORIAL[1] -> 2
FACTORIAL[1] -> 2
FACTORIAL[1] -> 2
FACTORIAL[1] -> 2
FACTORIAL[1] -> 2
FACTORIAL[1] -> 2
FACTORIAL[1] -> 4
FACTORIAL[4] 1
FACTORIAL[2] 1
FACTORIAL[3] -> 5
FACTORIAL[2] 2
FACTORIAL[3] -> 5
FACTORIAL[2] 6
FACTORIAL[3] -> 5
FACTORIAL[2] 24
FACTORIAL[3] -> 5
FACTORIAL[2] 120
FACTORIAL[3] -> 5
FACTORIAL[2] 720
FACTORIAL[3] -> 5
720
```

Line 1 is called seven times, the first is the call with $N = 6$, the last with $N = 0$; at this point ($N = 0$) the jump to line 4 make FACTORIAL to return 1, and the next iterations all return the result returned by the just called FACTORIAL. The final result is returned only in the last execution. The recursion technique, when available, is fast, because generally less instructions are performed.

Needless to say, recursion is fascinating, but involves a number of iterations to be saved somewhere, waiting for the completion of the procedure. For few iterations (like the one just shown) the problem is meaningless, but for large iterations, there could be a problem of exhausting the machine capabilities (called *stack-full state*), for examples with the Ackermann recursion (see next).

So, any programmer knows that his or her genial and Grammy-award-potentially-winner recursive algorithm must be abandoned in favour of a less genial and more robust iterative version, less memory-consuming.

Here is an iterative version of the same factorial algorithm, contained in the file `fact-it.apl`:

```
\/ Z<-FAC N;I
Z<-1
I<-0
L1:I<-I+1
->0xiI>N
Z<-ZxI
```

```
->L1  
\
```

The same execution can be invoked from console:

```
./apple --apl=")DIGIT 16" --args="6" fact-it  
WAS 0  
720
```

The tracing is much more illuminating:

```
./apple --apl=")DIGIT 16" --args="6" --trace="i5" fact-it  
WAS 0  
FAC[1] 1  
FAC[2] 0  
FAC[3] 1  
FAC[4] -> 5  
FAC[5] 1  
FAC[3] 2  
FAC[4] -> 5  
FAC[5] 2  
FAC[3] 3  
FAC[4] -> 5  
FAC[5] 6  
FAC[3] 4  
FAC[4] -> 5  
FAC[5] 24  
FAC[3] 5  
FAC[4] -> 5  
FAC[5] 120  
FAC[3] 6  
FAC[4] -> 5  
FAC[5] 720  
FAC[3] 7  
FAC[4] -> 7  
720
```

There are no stacked procedures. The calculation proceeds from 1, calculating all the factorials, up to the desired one, repeating lines 3-5, There is one program instance only.

In case we'd require a stupid limit, we'd see this effect:

```
./apple --clean --apl=")DIGIT 16" --args="120" --trace="i5" fact-it  
[...]  
6.689502913449124E+198
```

The calculation, even if absolutely useless, is reached without errors. And in case of not converging input? See this:

```
./apple --clean --apl=")DIGIT 16" --args="_4" --trace="i5" fact-it  
WAS 0  
FAC[1] 1  
FAC[2] 0  
FAC[3] 1  
FAC[4] -> 7
```


1

The output is simple... the algorithm stops if $I > N$, which is valid from the first instruction (I is zero, N is -4...). No error, which is elegant and safe⁴⁸.

Another analog technique starts from the desired number and gets down to 2 (knowing that multiplying by 1 is ineffective). The file is `fact-it2.apl`:

```
\ / Z<-FAC N;I
Z<-N
L1:N<-N-1
->0xiN<2
Z<-ZxN
->L1
\ /
```

This algorithm is even shorter, and probably, though uselessly, about 10-20% faster, as shown by the trace:

```
./apple --apl=")DIGIT 16" --args="6" --trace="i5" fact-it2
WAS 0
FAC[1] 6
FAC[2] 5
FAC[3] -> 4
FAC[4] 30
FAC[5] -> 2
FAC[2] 4
FAC[3] -> 4
FAC[4] 120
FAC[5] -> 2
FAC[2] 3
FAC[3] -> 4
FAC[4] 360
FAC[5] -> 2
FAC[2] 2
FAC[3] -> 4
FAC[4] 720
FAC[5] -> 2
FAC[2] 1
FAC[3] -> 6
720
```

My 2¢: if you have only one recursive algorithm, assure a) that it converges and b) the number of iterations are not much⁴⁹; an improper value is dangerous and can lead to the "FN RECURSION ERROR", or in case you pass even this limit, to the "FN SPACE FULL ERROR". If you have two algorithms, both working, but one is recursive and the second is not recursive, adopt the second: it can evaluate much more levels, as shown previously.

5.8.2. The Ackermann recursion

A notable example of indirect recursion is the Ackermann function⁵⁰. This is a classic example of a non primitive recursive algorithm, that always produces a result, but in some cases with the creation of a lot of variables, those that are generated by the multiple recursions that happens. The three-argument function, $\phi(m, n, p)$ is defined such that for $p = 0, 1, 2$ it yields the basic operations of addition, multiplication, and exponentiation as follows:

⁴⁸ I think it's understood which I prefer, isn't it?

⁴⁹ How much depends on your computer.

⁵⁰ See also https://en.wikipedia.org/wiki/Ackermann_function

$$\begin{aligned}\phi(m, n, 0) &= m + n \\ \phi(m, n, 1) &= m \times n \\ \phi(m, n, 2) &= m^n\end{aligned}$$

and for $p > 2$ it goes like:

$$\begin{aligned}\phi(m, n, 3) &= m[4](n + 1) \\ \phi(m, n, p) &= m[p + 1](n + 1), p \in N\end{aligned}$$

Many authors developed two-arguments equivalents that are more computable in terms of programming languages. A first attempt by Péter and Robinson is:

$$\begin{aligned}A(0, n) &= n + 1 \\ A(m + 1, 0) &= A(m, 1) \\ A(m + 1, n + 1) &= A(m, A(m + 1, n))\end{aligned}$$

A naive algorithm in APL\1130 that reproduces this sequence can be found in the file `ACK.apl`:

```
\ /Z<-M ACK N
->SKIP1xiM#0
Z<-N+1
->END
SKIP1:->SKIP2xiN#0
Z<-(M-1) ACK 1
->END
SKIP2:Z<-(M-1) ACK (M ACK N-1)
END: \ /
```

It's a dyadic function that takes m as the left argument and n as the right argument; it's obviously fast for the case $m = 1..3$; unfortunately, the architecture of `apple` prevents `ACK` from being used for a greater m , and at present it works within these limits: $m = 1$ and $n = 1..310@$, $@m = 2$ and $n = 1..140$ or $m = 3$ and $n = 1..5$; beyond these limits, the system reaches the recursion limit⁵¹.

As for the non-primitive recursion algorithms, bear in mind that in general it's not the algorithm that fails, but the memory management of the used programming language that limits the variables creation number and/or the recursion levels; this means that you'll never know if the algorithm works because you cannot demonstrate it.

Here it is in action:

```
) IMPO libs/ACK
ACK IMPORTED
3 ACK 3
61
3 ACK 5
253
2 ACK 100
203
```

5.9. On the axis commands

You may have noticed that some commands can be activated as is, with no other indication, or with some numbering that specifies the axis (or option) within square brackets. This technique increases the APL functions, because we

⁵¹ You can change the value of `RECURS` and `COMPVAR` in `custom.h` and see if that helps. I warn you, however, that in case `RECURS` is set to an unnaturally high value (generating an uncontrolled recursion value) or in case of `COMPVAR` is set to an unsustainably high value (giving birth to an uncontrolled number of variables), the error messages appearing on the console may not all come from `apple`, and this is not desirable.

can obtain more targeted results.

The following table explains all these commands and offers useful information for the casual APL programmer:

Operator	Axis range	Default axis
$\wedge\backslash$ (grade up)	1-2	2
$\backslash\vee$ (grade down)	1-2	2
$\circ $ (rotation)	1-2	2
$/$ and $/-$ (compression)	1-2	2
$\backslash$ and $\backslash-$ (expansion)	1-2	2
$,$ (ravel)	1-2	2

Table 4.2: Axis direction

Usually, axis are interpreted as follows: 1 is the columns direction and 2 is the rows direction. For instance, the rotation operator $\circ|$ rotates each row by default:

```
M<-4 4p1 2 3 4 3 2 4 6 5 3 2 4 1 5 4 2
M
1 2 3 4
3 2 4 6
5 3 2 4
1 5 4 2
  o|M
4 3 2 1
6 4 2 3
4 2 3 5
2 4 5 1
  o|[2]M
4 3 2 1
6 4 2 3
4 2 3 5
2 4 5 1
  o|[1]M
1 5 4 2
5 3 2 4
3 2 4 6
1 2 3 4
```

The axis 1 is not the default one, and must be explicitly required. The axis 2 is instead available by default.

This particular instruction has also a companion: the operator $\circ-$ is the direct version of the call of $\circ|$ along axis 1:

```
o-M
1 5 4 2
5 3 2 4
3 2 4 6
1 2 3 4
```

This was conceived because in older days the APL commands had to be printed on a printer, in order to be interpreted, and typing 'o' and '-' was faster than 'o', '|', and the group '[1]'.

Another example involves the comma for the ravel operation. If used without axis, it acts upon rows (axis 2), in the sense that each row is affected by the ravel operation:

```

      8 8 8 8,M
      8 1 2 3 4
      8 3 2 4 6
      8 5 3 2 4
      8 1 5 4 2
      M,8 8 8 8

      1 2 3 4 8
      3 2 4 6 8
      5 3 2 4 8
      1 5 4 2 8
      M,[2]8 8 8 8

      1 2 3 4 8
      3 2 4 6 8
      5 3 2 4 8
      1 5 4 2 8
      M,8 8 8 8

```

For columns, the axis 1 must be explicitly written (this is when all columns are affected by the ravel operation):

```

      M,[1]8 8 8 8

      1 2 3 4
      3 2 4 6
      5 3 2 4
      1 5 4 2
      8 8 8 8
      8 8 8 8,[1]M

      8 8 8 8
      1 2 3 4
      3 2 4 6
      5 3 2 4
      1 5 4 2

```

The dashed versions. In general, one should bear in mind that the main direct operators such as for instance / for compression and \ for expansion were targeted to act on columns, that is on the second axis; this means that an assertion such as 1 1 0 1/M compresses the columns of M, and the assertion 1 1 0 1 0 1\M expands the columns of M, inserting a new third and fifth column of nulls.

The dashed versions were created to be a quick reference for addressing rows, instead of columns, that is acting on the first axis, not the second; this means that the assertion 1 1 0 1/-M compresses the rows of M, and the assertion 1 1 0 1 0 1\M expands the rows of M, inserting a new third and a new fifth row of nulls.

When the introduction of more axis was completed (the modern APL engines deal with many dimensions in the arrays), the simple addressing on the first and last axis was not enough, and the APL programmers made available the

addressing of explicit numerical indices enclosed in square brackets; this let the programmer write assertions such as `1 1 0 1 0 1/[3]M` for the explicit third axis, for instance. The introduction of the explicit indices makes superfluous which command is called, so `/[2]` and `/-[2]` must refer to the same index (columns) and so the two operators behave identically. The same for the rest of the dashed indexed commands.

`apple` can deal with two dimensions at most; generalizing, the values that can be used as indices are 1 and 2 only, so the following equivalences arise:

<code>/</code>	<code>==</code>	<code>/[2]</code>	<code>==</code>	<code>/-[2]</code>
<code>/-</code>	<code>==</code>	<code>/[1]</code>	<code>==</code>	<code>/-[1]</code>
<code>\</code>	<code>==</code>	<code>\[2]</code>	<code>==</code>	<code>\-[2]</code>
<code>\-</code>	<code>==</code>	<code>\[1]</code>	<code>==</code>	<code>\-[1]</code>
<code>o </code>	<code>==</code>	<code>o [2]</code>	<code>==</code>	<code>o-[2]</code>
<code>o-</code>	<code>==</code>	<code>o [1]</code>	<code>==</code>	<code>o-[1]</code>

5.10. On catenation and lamination

The difference between catenate and laminate may not be easy to understand, at a first sight. They work on the same vectors, using indices of different nature, but why and how they yield so much different results?

The ISO document "*Programming Language APL, Extended International Standards Organisation - ISO/IEC 13751 : 2000 (E)*" reports at paragraph 10.2 a formal definition of the procedure "**Join Along an Axis**", which is reported in the following, converted to the ASCII symbolism adopted by this manual:

10.2 Join Along an Axis

```
Z ← A , [K] B
```

Informal Description: Z is formed by joining A and B along a designated axis. There are two suboperations, catenate and laminate. The catenate suboperation joins the arrays along an existing axis, the laminate suboperation along a new axis. The choice of axis and the choice of operation is determined by K: if K is a near-integer, the operation is catenate and the axis is K; if K is not a near-integer, the operation is laminate, the new axis is }K, and the axes greater than or equal to }K are renumbered."⁵²

The interpreter `apple` follows strictly the ISO definition, and for what applies to scalars and vectors, the ISO is respected (the final result cannot be of order greater than 2, because matrices are the highest order `apple` can manage); moreover, the description in the manual contains references to the ISO document.

So, what does the expression `1 2 3 4,5 6 7 8` mean? According to the previous definition, this is a concatenation procedure that adds the second argument to the existing row of first argument, which has number 1. The same for `1 2 3 4,[2]5 6 7 8`.

```
1 2 3 4,5 6 7 8
1 2 3 4 5 6 7 8
```

And what does the expression `1 2 3 4,[1]5 6 7 8` mean? According to the previous definition, this is a concatenation procedure that adds the second argument to the existing column of first argument, which has number 1 (obviously, vectors are interpreted as vertical, here).

```
1 2 3 4,[1]5 6 7 8

1
2
3
```

⁵² In this definition, a *near-integer* is any number that, with the current approximation technique, is undistinguishable from an integer.

4
5
6
7
8

And now, what does the expression $1 \ 2 \ 3 \ 4, [0.5]5 \ 6 \ 7 \ 8$ mean? According to the previous definition, this is a lamination procedure that creates a new row *before the first* which now has number 1, adds the left argument to this new row, and the second argument to the existing row, which now has number 2.

$1 \ 2 \ 3 \ 4, [0.5]5 \ 6 \ 7 \ 8$

1	2	3	4
5	6	7	8

And what does the expression $1 \ 2 \ 3 \ 4, [1.5]5 \ 6 \ 7 \ 8$ mean? This is a lamination procedure that creates a new column (obviously, vectors are interpreted as vertical, here) *before the first* which now has number 1, adds the left argument to this new column, and the second argument to the existing column, which now has number 2.

$1 \ 2 \ 3 \ 4, [1.5]5 \ 6 \ 7 \ 8$

1	5
2	6
3	7
4	8

5.11. On compression and expansion

Compression and expansion are two similar procedures that need to be detailed.

Compression removes rows (or columns) of a matrix, or adds zero lists, while expansion duplicates rows (or columns) to a matrix or adds zero lists; in case of vectors (intended as single rows), items are removed or added in column fashion, but the process is the same.

The main difference is that compression acts on the existing matrix, and you can obtain other matrices by using elements of the object matrix mixed with zero lists (of any dimensions), while expansion adds lists to the existing object matrix by replicating existing lists or by intermixing zero lists (of any dimensions).

The reduction applied by the compression operator is related to the fact that not all the existing lists (rows or columns) are taken; the expansion applied by the expansion operator is related to the fact that the existing lists in the object matrix remain, and other lists are added.

Compression. The compression function X/M is simple: the number of the indices in the left argument X must correspond to rows, or columns, number of the target matrix M (in practice $pX = (pM) [2]$ for $/$ and $pX = (pM) [1]$ for $/-$). For instance:

B

7	3	4	_11	_9	_2
_6	4	_5	7	1	12
_1	_9	2	2	9	1
_8	0	_1	5	0	8
_4	3	_5	7	2	10
6	1	4	_11	_7	_1
1	0	1	0	1	0/B

```

7    4    _9
_6   _5    1
_1    2    9
_8   _1    0
_4   _5    2
6    4    _7

```

Matrix B is a 6×6 matrix, and the indices in the left argument (counting both zeroes and ones) are 6. The command simply states: retain columns identified by 1, remove the ones identified by zero. In the example, the first, third and fifth columns are retained and the second, fourth and sixth are removed.

If a positive value is used in place of 1, this means: retain and replicate. See the following examples:

```

1 0 2 0 1 0/B

7    4    4    _9
_6   _5   _5    1
_1    2    2    9
_8   _1   _1    0
_4   _5   _5    2
6    4    4    _7
1 0 2 0 2 0/B

7    4    4    _9    _9
_6   _5   _5    1    1
_1    2    2    9    9
_8   _1   _1    0    0
_4   _5   _5    2    2
6    4    4    _7    _7
2 0 2 0 2 0/B

7    7    4    4    _9    _9
_6   _6   _5   _5    1    1
_1   _1    2    2    9    9
_8   _8   _1   _1    0    0
_4   _4   _5   _5    2    2
6    6    4    4    _7    _7

```

If -1 is used in place of one or zero, this means: add a column of zero in the indicated position; see the example:

```

1 0 _1 0 1 0/B

7    0    _9
_6    0    1
_1    0    9
_8    0    0
_4    0    2
6    0    _7

```

If a negative value is used in place of one, this means: add the indicated number of zero columns to the count in the indicated position; see the example:

```

1 0 _3 0 1 0/B

```

7	0	0	0	_9
_6	0	0	0	1
_1	0	0	0	9
_8	0	0	0	0
_4	0	0	0	2
6	0	0	0	_7

To act on rows rather than columns, the command `/-` must be used. Both commands `/` and `/-` support the axis specification, with the convention that axis 1 acts on rows, and axis 2 acts on columns, so that

`/ ==> /[2] ==> /-[2]`
`/- ==> /-[1] ==> /[1]`

Expansion. The expansion function `X\M` is the converse function of compression, with similar purposes. This time, the number of non-null elements in the left argument `X` must correspond to the dimension of the expanding axis in `M`. See the example:

B

7	3	4	_11	_9	_2
_6	4	_5	7	1	12
_1	_9	2	2	9	1
_8	0	_1	5	0	8
_4	3	_5	7	2	10
6	1	4	_11	_7	_1

1 0 1 1 1 1 0 1\B

7	0	3	4	_11	_9	0	_2
_6	0	4	_5	7	1	0	12
_1	0	_9	2	2	9	0	1
_8	0	0	_1	5	0	0	8
_4	0	3	_5	7	2	0	10
6	0	1	4	_11	_7	0	_1

In the example, the number of non-null elements is 6, which corresponds to the number of columns in B; besides, if zeroes are used in specific places, they mean: add zero columns to the count:

If in place of 1 we use positive values, this means: replicate the correspondent column (or row):

2 1 2 1 1 1\B

7	7	3	4	4	_11	_9	_2
_6	_6	4	_5	_5	7	1	12
_1	_1	_9	2	2	2	9	1
_8	_8	0	_1	_1	5	0	8
_4	_4	3	_5	_5	7	2	10
6	6	1	4	4	_11	_7	_1

In the example, columns first and third are doubled. If in place of zero we use a negative value, this means adding a specific number of nulls:

1 1 1 _3 1 1 1\B

7	3	4	0	0	0	_11	_9	_2
---	---	---	---	---	---	-----	----	----


```

_6      4      _5      0      0      0      7      1      12
_1      _9      2      0      0      0      2      9      1
_8      0      _1      0      0      0      5      0      8
_4      3      _5      0      0      0      7      2      10
6       1       4       0      0      0     _11     _7     _1

```

In the example, -3 means adding 3 empty lines in the place after the first 3 columns.

As for the compression operator, to act on rows rather than columns, the command `\-` must be used. Both commands `\` and `\-` support the axis specification, with the convention that axis 1 acts on rows, and axis 2 acts on columns, so that

```

\  ==>  \[2]  ==>  \-[2]
\- ==>  \-[1] ==>  \[1]

```

5.12. On identity numbers

By confronting the tables 2.3A at page 16 and 2.3B at page 16 and the identity table 2.6 at page 28, one may wonder why not all the primitive scalar dyadic operators have an identity number. As widely known, all dyadic operators which are also commutative should have an identity number which establishes the neutrality of the mathematical operation, for example the 'plus' with zero: $4 + 0 = 0 + 4 = 4$ or the multiplication with 1: $4 \times 1 = 1 \times 4 = 4$; but why 'nand' and 'nor', which are commutative, don't have an identity number? Well, this exclusion is based on precise mathematical concepts.

Consider for example the following truth tables of 'and' and 'nand':

A	B	$A \wedge B$	$A \wedge \sim B$
1	1	1	0
0	1	0	1
1	0	0	1
0	0	0	1

In the previous scheme, the first value of A is retained as result in case 1 is used, both for 0 and 1. So 1 is the identity number of 'and'. But 'nand' has no case where the 0 or 1 retain the value of A as result both for 0 and 1.

Consider now the following truth tables of 'or' and 'nor':

A	B	$A \vee B$	$A \vee \sim B$
1	1	1	0
0	1	1	0
1	0	1	0
0	0	0	1

In the previous scheme, the first value of A is retained as result in case 0 is used, both for 0 and 1. So 0 is the identity number of 'or'. But 'nor' has no case where the 0 or 1 retain the value of A as result both for 0 and 1.

The conclusion is that the primitive scalar operators $\wedge \sim$ and $\vee \sim$ have no identity numbers.

A different consideration must be made for the primitive scalar operators ? (deal), & (general logarithm) and o (circular functions): they do not have identity numbers because they are not commutative functions.

Furthermore, the $\sim$ (not) is monadic, not dyadic, and in this case the identity number loses its significance. And the dyadic version of $\sim$ (remember, this was not in the original APL1130) is not commutative, so there is no identity number for the dyadic 'not'.

6. DIFFERENCES WITH THE ORIGINAL VERSION OF IBM APL\1130

Writing an emulator and making it work in the very same exact way as the original compiler used to behave is quite impossible, and some differences always exist, principally due to the different computer architectures in times and to my non-professional programming knowledge. With APL, in addition to the usual problems related to the conception of the language, I found a further difficulty: the unavailability, for normal computers, to use the APL keyboard, unless using complicated combinations of keys to generate characters that (perhaps) the terminal is not even able to depict. In the following tables all the differences are listed, just to let the programmer know the real extents of **apple**. They are exposed in three fields: limitations, features and enhancements.

IBM APL\1130	apple
Supports the right arrow → for starting interrupted programs from a specific line	Does not support the right arrow → for starting interrupted programs from a specific line; the form →0 is accepted for resetting the program state.
Has fully implemented)SI and)SIV	Implements a simplified form of)SI and)SIV

Table 5.1A: Limitations of **apple** with respect to the IBM APL\1130

IBM APL\1130	apple
Uses specific non-standard keyboard and non-standard characters	Uses the default keyboard and the standard ASCII characters in the range 0..127 for the wider compatibility
Requires two blanks, one before and one after a user function name if used into an expression (unless preceded or followed by the dot or the literal minus sign)	Requires always one blank after a user function name if used into an expression
Prints numbers in matrices aligned following the decimal dot	Prints numbers in matrices aligned to the right, regardless of the position of the decimal dot
Shows one-column matrices as horizontal vectors	Shows one-column matrices as vertical vectors
Forbids catenation of strings and numbers, that generates a domain error	Accepts catenation of strings and numbers; the final type is the one of the last item (the first to be executed)
In program editing, uses the figure [N□M] for editing line N from position M	In program editing, uses the figure [N _ M] for listing the program from line N to line M, and start editing line M+1.

Table 5.1B: Features of **apple** differing from those in the IBM APL\1130

IBM APL\1130	apple
Has 6 characters for names, beginning with an uppercase letter, and followed by uppercase letters or numbers	Has 15 characters for names, beginning with an uppercase letter, and followed by uppercase letters or numbers
Has 256 elements for a vector or a matrix at most, in each direction	Has unlimited elements for a vector or a matrix (the limit is the machine's)
Lacks the following features: - Comments - Ω - Deal - dyadic $?$ - Grade up - monadic $\uparrow$ - Grade down - monadic $\downarrow$ - Take - dyadic $\uparrow$ - Drop - dyadic $\downarrow$ - Nand - dyadic $\wedge$ - Nor - dyadic $\vee$ - Circular functions - $\circ$ - Logarithm - dyadic $\otimes$	Implements these features as: - Comments - $\circ\circ$ - Deal - dyadic $?$ - Grade up - monadic $/^{\wedge}\backslash$ - Grade down - monadic $\backslash\vee/$ - Take - dyadic $/ \backslash$ - Drop - dyadic $\backslash /$ - Nand - dyadic $\wedge\sim$ - Nor - dyadic $\vee\sim$ - Circular functions - $\circ$ - Logarithm - dyadic $\&$
The reduction monadic operators $/$ and $\wedge$ cannot be used with indices	Enables indices for the reduction monadic operators $/$ and $/-$
Uses the dyadic compression operators $/$ and $\wedge$ with boolean (logical) arrays	Uses the dyadic compression operators $/$ and $/-$ also with integer arrays, thus enabling the replication feature
Does not manage the catenation of matrices (as far as is known)	Manages the catenation of matrices
Has no hexadecimal syntax	Can interpret hexadecimal numbers through the symbol $\$$ (the dollar sign)
Accepts only positive integers and returns only integers values for the random number generator	Accepts positive integers for positive integer values, negative integers for negative integer values, and zero for positive real values in the range]0.0, 1.0[for the random number generator
Has no double-quote strings	Implements the double quote strings, following the GNU APL rules

IBM APL\1130	apple
Has no added library (as far as is known)	Implements some extensions - The dyadic grade up and grade down - The lamination operator through the comma - The scan monadic operators $f\backslash$ and $f\backslash-$ - The string execution operator ϵ (ISO ϕ) - The formatted output operator $\bar{\epsilon}$ (ISO $\bar{\phi}$) - The type conversion operator $\subset$ - The match operator $=_$ (ISO $\equiv$) - The unique set $\cup$ (ISO $\cup$) - The unique mask $\#$ - The dyadic $\sim$ operator for the exclusion set - The I/O operators $\wedge$, $\vee$, $>$ and $<$. - The linear algebra extensions

Table 5.1C: Enhancements of **apple** with respect to the IBM APL\1130

7. SAMPLE TERMINAL SESSION

In the following, I report the script of some sessions I ran, as a transcription (with the very same comments) of the APL70 (pages from 110 to 124), to show the complete adherence of **apple** to the features of APL, apart those not implemented in the APL\1130 and in **apple** (like for instance the 3-D matrices).⁵³ The session is formalized in a run-through APL file (See the program `RunThrough.apl` in the package). The symbol @ in the following pages indicates empty lines not appearing in the session output, added here for hosting the comments to the right. This run-through should be run with option `-c`, for starting a with a clean workspace, and the `<` shell operator must be used, like this:

```
$ apple -c < RunThrough.apl
```

```
TONIBIN SIGNED ON
```

```
A (P) P L (E) \ 1 1 3 0
```

@		FUNDAMENTALS
@		
	3x4	Entry automatically indented
12		Response not indented
	X<-3x4	X is assigned the value of
@		the expression
	X	
12		Value of X typed out
	Y<_5	Negative sign for negative
	X+Y	constants
7		
	144E_2	Exponential form of constant
1.44		
	P<-1 2 3 4	Four-element vector
	PxP	Functions apply element by element
1 4	9 16	
	PxY	Scalar applies to all elements
_5 _10 _15 _20		
	Q<-'CATS'	Character constant (4-element
	Q	vector
CATS		
	YZ<-5	Multi-character names
	YZ1<-5	
	YZ+YZ1	
10		
	3+4x5+6	Correction by arrow-up
47		and retyping
	3+4+5+6	
18		
	X<-3	
	Y<-4	
	(XxY)+4	
16		
	XxY+4	Executed from right to left
24		

⁵³ I used the APL70 as base because **apple** implements all the features described in the APL69, but the IBM APL\1130 compiler was explicitly not complete and lacked some of these features, so the APL70, which runs the exact transcription of the 1130 session, using the complete set of commands, is more pertinent.

X Y	Entry of invalid expression
SYNTAX ERROR	Shows type of error committed
X Y	Retypes invalid statement with
^	caret where execution stopped
XY	Multi-character name (not XxY)
VALUE ERROR ⁵⁴	
XY	XY had not been assigned a value
^	
@	SCALAR FUNCTIONS
@	
4x3}5.1	Dyadic maximum
20.4	
(4x3)}5.1	
12	
4x}5.1	Monadic ceiling
24	
X<-i5	Index generator function
X	
1 2 3 4 5	
i0	Empty vector
Y<-5-X	prints as a blank line
Y	All scalar functions extend
4 3 2 1 0	to vectors
X}Y	
4 3 3 4 5	
X<=Y	Relations produce
1 1 0 0 0	logical (0 1) results
)DIGIT 11	
WAS 0	
o1	Greek Pi x 1
3.1415926536	
o%1 2	Greek Pi x 1 2
3.1415926536 1.5707963268	
X<-45 90	
oX%180	Conversion of X to radians
0.7853981634 1.5707963268	
1o1	Sin 1
0.84147098481	
2o1 2	Cos 1 2
0.54030230587 _0.41614683655	
3o1	Tan 1
1.5574077247	
_3o1	Arctan 1
0.7853981634	
3o_3oi7	Tan Arctan 1 2 3 4 5 6 7
1 2 3 4 5 6 7	
Y<-1 2	
4oY	(1+Y*2)*.5
1.4142135624 2.2360679775	
0o%Y	(1-%Y*2)*.5
0 0.86602540378	
7o1 2	Tanh 1 2
0.76159415596 0.96402758008	
_7o7o1 2	Arctanh Tanh 1 2
1 2	
)DIGIT 0	
WAS 11	

⁵⁴ The original APL 69 reports here a SYNTAX ERROR, but in the APL70 (and to me also) this is a VALUE ERROR.

@		DEFINED FUNCTIONS
@		
	\Z<-X F Y	Header (2 args and result)
[1]	Z<-((X*2)+Y*2)*.5	Function body
[2]	\/	Close of definition
	3 F 4	Execution of dyadic function
5		
	P<-7	
	Q<-(P+1)F P-1	Use of F with expressions
	Q	as arguments
10		
	4x3 F 4	
20		
	\B<-G A	G is the signum function
[1]	B<-(A>0)-A<0	A and B are local variables
[2]	\/	
	G 4	
1		
	G _6	
_1		
	X<-_6	
	G X	
_1		
	\H A	Like G but has no explicit result
[1]	P<-(A>0)-A<0	P is a global variable
[2]	\/	
	H _6	
	P	
_1		
	Y<-H _6	H has no explicit result
VALUE	ERROR	and hence produces a value
	Y<-H _6	error when used to the right
	^	of assignment
	\Z<-FAC N;I	FAC is the factorial function
[1]	Z<-1	
[2]	I<-0	
[3]	L1:I<-I+1	L1 becomes 3 at close of def
[4]	->0xiI>N	Branch to 0 (out) or to next
[5]	Z<-ZxI	
[6]	->L1	Branch to L1 (that is, 3)
[7]	\/	
	FAC 3	
6		
	FAC 5	
120		
	T/\FAC<-3 5	Set trace on lines 3 and 5 of FAC
	X<-FAC 3	
FAC[3]	1	Trace of FAC
FAC[5]	1	
FAC[3]	2	
FAC[5]	2	
FAC[3]	3	
FAC[5]	6	
FAC[3]	4	
	T/\FAC<-0	Reset trace control

@	MECHANICS OF FUNCTION DEFINITION
@	
\G<-M GCD N	Greatest common divisor
[1] G<-N	function based on the
[2] M<-M N	Euclidean algorithm
[3] ->4xM#0	
[4] [1]G<-M	Correction of line 1
[2] [4]N<-G	Resume with line 4
[5] [1 _]	Display line 1
[1] G<-N	
[1] [_]	Display entire GCD Function
\G<-M GCD N	
[1] G<-M	
[2] M<-M N	
[3] ->4xM#0	
[4] N<-G	
\	Close of display, not close of def
[5] ->1	Enter line 5
[6] \	Close of definition
36 GCD 44	Use of GCD
4	4 is GCD of 36 and 44
\GCD	Reopen def (use \ and name only)
[6] [4.1]M,N	Insert between 4 and 5
[4.2] [_]	Display entire function
\G<-M GCD N	
[1] G<-M	
[2] M<-M N	
[3] ->4xM#0	
[4] N<-G	
[4.1] M,N	Fraction stays until close of def
[5] ->1	
\	End of display
[6] \	Close of definition
36 GCD 44	
8 36	Iteration printed by
4 8	line 5 (was line 4.1)
4	Final result
\GCD[_]\	Reopen, display, and close GCD
\G<-M GCD N	
[1] G<-M	
[2] M<-M N	
[3] ->4xM#0	
[4] N<-G	
[5] M,N	Line numbers have been
[6] ->1	reassigned as integers
\	Close (Even number of \'s in all)
\GCD	Reopen definition of GCD
[7] [5]	Delete line 5 by linefeed
@ ^	
[5] \	Close definition

<pre> \ / Z <- ABC X [1] Z <- (33 x Q + (R x 5) - 6 [2] [1] Z <- (3 x Q) + (T x 5) - 6 [2] \ / FAC 5 120) ERASE FAC FAC 5 SYNTAX ERROR FAC 5 ^ \ / Z <- BIN N [1] LA: Z <- (Z, 0) + 0, Z [2] -> LA x N >= p Z \ / BIN 3 SYNTAX ERROR BIN[1] LA: Z <- (Z, 0) + 0, Z ^ Z <- -1 -> 1 1 3 3 1 BIN 4 SYNTAX ERROR BIN[1] LA: Z <- (Z, 0) + 0, Z ^) SI BIN [1] * \ / BIN[0.1] Z <- -1 \ / BIN 4 1 4 6 4 1 \ / BIN[_] \ / \ / Z <- BIN N [1] Z <- -1 [2] LA: Z <- (Z, 0) + 0, Z [3] -> LA x N >= p Z \ / S \ BIN <- -2 BIN 3 BIN[2] Z 1 -> 2 BIN[2] -> 2 BIN[2] -> 0 S \ BIN <- 0 </pre>	<pre> A function to show line editing A line to be corrected Re-edit line 2 recalling it with up arrow Close correction FAC still defined Erase function FAC Function FAC no longer exists An (erroneous) function for binomial coefficients Suspended execution Assign value to Z Restart from line 1 Same error (local variable Z does not retain its value) Display state indicator Suspended on line 1 of BIN Insert line to initialize Z Restart execution Display revised function and close definition Set stop on line 2 Execute BIN⁵⁵ Stop due to stop control Display current value of Z Resume execution Stop again on next iteration Resume Stop again on next iteration Branch to 0 (terminate without output) Disable stop control </pre>
--	--

⁵⁵ apple does not admit an assignment here, so I perform the pure execution; after resetting the stop control, the assignment can be safely accomplished.

<pre> \ / MULTDRILL N;Y;X [1] Y<-?N [2] Y [3] X<- _ [4] ->0xiX='S' [5] ->iX=x/Y [6] 'WRONG, TRY AGAIN' [7] ->3\ / MULTDRILL 12 12 3 1 _ : 5 WRONG, TRY AGAIN _ : 3 9 8 _ : 's' \ / Z<-ENTERTEXT [1] Z<-'' [2] D<-pZ [3] Z<-Z, ' [4] ->2xD#pZ [5] \ / Q<-ENTERTEXT THIS IS ALL CHARACTER INPUT @ Q THIS IS ALL CHARACTER INPUT N<-5 'NOTE: i';N;' IS ';iN NOTE: i5 IS 1 2 3 4 5 @ @ P<-2 3 5 7 pP 4 T<- 'OH MY' pT 5 P,P 2 3 5 7 2 3 5 7 T,T OH MYOH MY T,P 79 72 32 77 89 2 3 5 7 </pre>	A multiplication drill⁵⁶ pN random integers Print the random factors Keyboard input Stop if entry is the letter S Repeat if entry is correct product Prints if preceding branch fails Branch to 3 for retry Drill for pairs in range 1 to 12 Indicates that keyboard entry is awaited Entry of letter S stops drill Example of character (') input Make Z an empty vector D is the length of Z Append character keyboard entry Branch to 2 if length increased (i.e., entry was not empty) Keyboard entries Empty input to terminate Display Q Mixed output statement RECTANGULAR ARRAYS Dimension of P Character vector Catenation Characters can be concatenated⁵⁷ with numbers
---	---

⁵⁶ This program MULTIDRILL and the following ENTERTEXT and not contained in the RunThrough.apl file, because of the keyboard-input operators involved |_| and |'|.

⁵⁷ This differs from APL70 because apple permits the catenation of numbers and strings.

M<-2 3p2 3 5 7 11 13	Reshape to produce a 2x3 matrix
M	Display of an array of rank >1
	is preceded by a blank line
2 3 5	
7 11 13	
2 4pT	A 2x4 matrix of characters
OH M	
YOH	
6pM	A matrix reshaped to a vector
2 3 5 7 11 13	
,M	Elements in row-major order
2 3 5 7 11 13	
P<- ,M	
P[3]	Indexing (third element of P)
5	
P[1 3 5]	A vector index
2 5 11	
P[i3]	The first three elements of P
2 3 5	
P[pP]	Last element of P
13	
M[1;2]	Element in row 1, column 2 of M
3	
M[1;]	Row 1 of M
2 3 5	
M[1 1;3 2]	Rows 1 and 1, columns 3 2
5 3	
5 3	
A<- 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'	The alphabet to Q
A[M]	A matrix index produces
	a matrix result
BCE	
GKM	
A[M[1 1;3 2]]	
EC	
EC	
M[1;]<-15 3 12	Respecifying the first row of M
M	
15 3 12	
7 11 13	

Q<-3 1 5 2 4 6	A permutation vector
P[Q]	Permutation of P
5 2 11 3 7 13	
Q[Q]	A new permutation
5 3 4 1 2 6	
P[3]	Present index origin is 1
5	
)ORIGIN 0	Set index origin to 0
WAS 1	
P[3]	
7	
P[0 1 2]	First three elements of P
2 3 5	
i5	Result of index generator
0 1 2 3 4	begins at origin
)ORIGIN 1	
WAS 0	
i5	
1 2 3 4 5	
@	
@	
V<-?3p9	Vector of e random integers ⁵⁸
M<-?3 3p9	Random 3 by 3 matrix
N<-?3 3p9	Random 3 by 3 matrix
V	
1 5 1	
M	
6 1 1	
3 6 1	
1 5 3	
N	
7 8 3	
2 9 6	
9 7 5	
M+N	Sum (element-by-element)
13 9 4	
5 15 7	
10 12 8	

⁵⁸ These results differ from the APL70 because of the nature of random numbers., but if you please you can test using the same results in the text.

M}N	Maximum
7 8 3	
3 9 6	
9 7 5	
M<=N	Comparison
1 1 1	
0 1 1	
1 1 1	
+/V	Sum-reduction of V
7	
x/V	Product-reduction
5	
+/[1]M	Sum over first coordinate
10 12 5	(down columns)
+/[2]M	Sum over second coordinate
@	(over rows)
8 10 9	
+/M	Sum over last coordinate
8 10 9	
/M	Maximum over last coordinate
6 6 5	
)DIGIT 11	Expand digits
WAS 0	
X<-1.5	
+/(1 2oX)*2	Sin squared plus Cos squared
1	
o/1 2,X	Sin Cos X
0.070678224526	
Y<-o/0 2,X	(1-(COS X)*2)*.5
Y	
0.9974949866	
Y=1oX	An identity
1	
)DIGIT 0	Restore default digits
WAS 11	
M+.xN	Ordinary matrix (+.x inner) product
53 64 29	
42 85 50	
44 74 48	
M+.<=N	An inner product
3 3 2	
2 3 3	
2 3 3	
M+.xV	+.x inner product with vector right argument
12	Notice the vertical vector ⁵⁹
34	
29	

⁵⁹ This is a difference with the original APL69 and APL70, because I remain stick to the mathematical final result.

```

V
1 5 1
    V@.xi5                                Outer product (times)

    1 2 3 4 5
    5 10 15 20 25
    1 2 3 4 5
    V@.<=i9                                Outer product

    1 1 1 1 1 1 1 1 1
    0 0 0 0 1 1 1 1 1
    1 1 1 1 1 1 1 1 160
@
@
    Q<-?10p5                                A random 10 elements vector
    Q                                        (range 1 to 5)
2 5 3 4 1 1 1 3 4 5
    +/[1]Q@.=i5                            Ith element of result is number
3 1 2 2 2                                of occurrences of the
@                                        value I in Q
    2 1\oM                                Ordinary transpose of M

    6 3 1
    1 6 5
    1 1 3
    \oM                                    Ordinary transpose of M (monadic)

    6 3 1
    1 6 5
    1 1 3
    1 1\oM61                            Diagonal of M
6 6 3
    X<-o(0,i5)%6
    )DIGIT 4                                Set number of output digits to 4
WAS 0
    )EXP 1                                Set exponential format
WAS 0
    \o1 2 3@.oX

    0.0000E+00  1.0000E+00  0.0000E+00  Table of sines, cosines and
    5.0000E_01  8.6603E_01  5.7735E_01      tangents in interval
    8.6603E_01  5.0000E_01  1.7321E+00      of 30 degrees
    1.0000E+00  0.0000E+00  1.6331E+16
    8.6603E_01  _5.0000E_01  _1.7321E+00
    5.0000E_01  _8.6603E_01  _5.7735E_0162
    )DIGIT 0                                Set default decimals format
WAS 4
    )EXP 0                                Set default decimal output
WAS 1

```

⁶⁰ Matrices of rank 3 are not available in apple, so the examples about this are not contained in RunThrough.apl

⁶¹ Again, rank 3 arrays examples are not present here.

⁶² Differences with the APL70 are due to the algorithm of the trigonometric functions used for apple, deriving from the C-library.

2 5 3 4 1 1 1 3 4 5	
3o Q	Rotate to right by 3 places
4 1 1 1 3 4 5 2 5 3	
_3o Q	Rotate to right by 3 places
3 4 5 2 5 3 4 1 1 1	
0 1 2o [1]M	Rotate columns by different amounts
6 6 3	
3 5 1	
1 1 1	
_2o [2]M	Rotate of rows all by 2 to right
1 1 6	
6 1 3	
5 3 1	
1 2 3o M	Rotation of rows
1 1 6	
1 3 6	
1 5 3	
o Q	Reversal of Q
5 4 3 1 1 1 4 3 5 2	
o [1]M	Reversal of M along first coordinate
1 5 3	
3 6 1	
6 1 1	
o M	Reversal along last coordinate
1 1 6	
1 6 3	
3 5 1	

		U<-Q>4	
		U	
0	1	0 0 0 0 0 0 0 0 1	
		U/Q	Compression of Q by logical
5	5		vector U
		(~U)/Q	Compression by not U
2	3	4 1 1 1 3 4	
		+U/Q	
10			
		1 0 1/[1]M	Compression along first
			coordinate of M
6	1	1	
1	5	3	
		1 0 1/M	Compression along last
			coordinate
6	1		
3	1		
1	3		
		(,M>5)/,M	,M is 6 1 1 3 6 1 1 5 3
6	6		All elements of M which exceed 5
		V<-1 0 1 0 1	
		V\i3	Expansion of iota 3
1	0	2 0 3	
		V\M	Expansion of rows of M
6	0	1 0 1	
3	0	6 0 1	
1	0	5 0 3	
		V\'ABC'	Expansion of literal vector
A B C			inserts spaces
		10_ _1 7 7 6	Base 10 value of vector 1 7 7 6
1776			
		8_ _1 7 7 6	Base 8 value of 1 7 7 6
1022			
		(4p10)\' '1776	4 digit base 10 representation
1	7	7 6	of number 1776
		(3p10)\' '1776	3 digit base 10 representation
7	7	6	of number 1776
		10 10\' '1776	
7	6		
		10\' '1776	
6			
		24 60 60_ _1 3 25	Mixed base value of 1 3 25
3805			(time radix)
		24 60 60\' '3805	Representation of number 3805
1	3	25	in time radix
		2_ _1 0 1 1 0	Base 2 value
22			


```

      M
      6 1 1
      3 6 1
      1 5 3
      )ORIGIN 0
WAS 1
      M[2;0]
      1      (,M) [(pM) _|_2,0]
      1      )ORIGIN 1
WAS 0
      P
      2 3 5 7 11 13
      Pi7
      4      Pi6
      7      Pi4 5 6 7
      7 3 7 4
      Q<-5 1 3 2 4
      R<-QiipQ
      R
      2 4 3 5 1
      Q[R]
      1 2 3 4 5
      A<- 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
      A<-A, 'RSTUVWXYZ'
      A
      ABCDEFGHIJKLMNOPQRSTUVWXYZ
      Ai 'C'
      3
      J<-Ai 'CAT'
      J
      3 1 20
      A[J]
      CAT

```

Indexing of matrix in 0-origin.
Note relation to indexing of
ravel of M

Restore 1-origin

Index of 7 in vector P
7 is 4th element of P
6 does not occur in P, hence
result is 1+pP

A permutation vector
R is the permutation inverse to Q

A is the alphabet

Rank of letter C in alphabet is 3

```

M<-3 5p'THREESHORTWORDS'  A matrix of characters
M
THREE
SHORT
WORDS
J<-AiM                      Ranking of M produces a matrix
J
20  8 18  5  5
19  8 15 18 20
23 15 18  4 19
A[J]                        Indexing by a matrix produces
                             a matrix

THREE
SHORT
WORDS
3?5                          Random choice of 3 out of 5
3 1 5                        without replacement
6?5
DOMAIN ERROR
6?5
^
X<-8?8                      A random permutation vector
X
7 3 8 6 1 2 5 4
/^X                          Grading of X
5 6 2 8 7 4 1 3
X[/^X]                      Arrange in ascending order
1 2 3 4 5 6 7 8
X[\v/X]                    Arrange in descending order
8 7 6 5 4 3 2 1
U<-Ae'NOW IS THE TIME'     Membership
'01'[1+U]
00001001100011100011001000
U/A
EHIMNOSTW
(i8)e3 7 5
0 0 1 0 1 0 1 0

```

ADVANCED ADDED FEATURES

_ <-M<-2 3p'1'	M is a matrix of characters
111	
111	
_ <-H<-3 3p'0'	H is another matrix of characters
000	
000	
000	
M,[1]H	Catenation is possible (expanding rows)
111	
111	
000	
000	
000	
_ <-L<-2 4p'2'	L is a matrix of characters
2222	
2222	
M,L	Catenation is possible (expanding columns)
1112222	
1112222	
M,'+'	Catenation of character '+'
111+	
111+	
M,'34'	Catenation of a vector, whose length conforms to the rows number
1113	
1114	
M,[1]'345'	Catenation of a vector, whose length conforms to the columns number
111	
111	
345	
1 2 3,[.5] 4 5 6	Lamination, vector 1 2 3 is added as row before 4 5 6
1 2 3	
4 5 6	
1 2 3,[1.5]4 5 6	Lamination, vector 1 2 3 is added as column before 4 5 6
1 4	
2 5	
3 6	
1 2 3,[1.5]4	Lamination, vector 1 2 3 is added as column before a column of 4
1 4	
2 4	
3 4	

<pre> (2 0p5), 'A' A A p3, [.5]'' 2 0 </pre>	A vertical vector of characters A vertical vector of characters' dimension
INVERSION FEATURES	
<pre> A<-2 2p0 A 0 0 0 0 % A 0 0 0 0 I_36 NULL SQUARE MATRIX INVERSE A<-2 2p1 0 A 1 0 1 0 A1<- % A I_36 PSEUDO-INVERSE A1 0.5 0.5 0 0 A+.xA1+.xA 1 0 1 0 A1+.xA+.xA1 0.5 0.5 0 0 A<-2 2p1 0 _1 0 A 1 0 _1 0 % A 0.5 _0.5 0 0 I_36 PSEUDO-INVERSE </pre>	Inversion of the null matrix The description of I_36 Inversion of a special matrix Demonstration of Moore-Penrose laws about pseudo-inverses Another special matrix inversion

```

A<-2 2p1 0 2 0
A
1 0
2 0
| % | A

0.2 0.4
0 0
I_36
PSEUDO-INVERSE
A<-2 2p1
A
1 1
1 1
| % | A

0.25 0.25
0.25 0.25
I_36
PSEUDO-INVERSE
A<-3 2p 1 0 0 1 0 1
A
1 0
0 1
0 1
A1<-| % | A
I_36
LEFT PSEUDO-INVERSE
A1

1 0 0
0 0.5 0.5
A1+.xA

1 0
0 1
M<-3 3pi9
M

1 2 3
4 5 6
7 8 9
| % | M

_0.638889 _0.166667 0.305556
_0.0555556 0 0.0555556
0.527778 0.166667 _0.194444
I_36
PSEUDO-INVERSE

```

Another special matrix inversion

Inversion of the 1-matrix

Ben-Israel algorithm

Classic non-invertible matrix

```

M<-3 3p 1 2 3 0 2 2 1 1 0   Invertible matrix
M

1  2  3
0  2  2
1  1  0
| % | M

0.5  _0.75  0.5
_0.5  0.75  0.5
0.5  _0.25  _0.5
I_36
PROPER INVERSE
M<-3 3p1 2 _3 0 2 2 1 1 0   Invertible matrix
M

1  2  _3
0  2  2
1  1  0
| % | M

_0.25  _0.375  1.25
0.25  0.375  _0.25
_0.25  0.125  0.25
I_36
PROPER INVERSE
M<-4 4p1 1 0 0 0 1 0 0 0 0 1 1 1 1 0 1
M                                     M is a 4x4 invertible
                                     rather sparse

1  1  0  0
0  1  0  0
0  0  1  1
1  1  0  1
| % | M

1  _1  0  0
0  1  0  0
1  0  1  _1
_1  0  0  1
I_36
PROPER INVERSE
M<-4 4p2 1 0 0 0 1 0 0 0 0 1 1 1 1 0 1
M

2  1  0  0
0  1  0  0
0  0  1  1
1  1  0  1
| % | M

0.5  _0.5  0  0
0  1  0  0
0.5  0.5  1  _1
_0.5  _0.5  0  1
I_36
PROPER INVERSE
M<-4 4p2 1 0 0 0 1 0 0 0 0 2 1 1 1 0 1
M                                     M is another rather sparse
                                     4x4 invertible

```

```

2  1  0  0
0  1  0  0
0  0  2  1
1  1  0  1
| % | M

0.5  _0.5  0  0
0  1  0  0
0.25 0.25  0.5 _0.5
_0.5 _0.5  0  1
I_36
PROPER INVERSE
M<-4 4p2 1 1 1 1 2 1 1 1 1 2 1 1 1 1 2
M M is a full matrix, invertible

2  1  1  1
1  2  1  1
1  1  2  1
1  1  1  2
| % | M

0.8  _0.2  _0.2  _0.2
_0.2  0.8  _0.2  _0.2
_0.2  _0.2  0.8  _0.2
_0.2  _0.2  _0.2  0.8
I_36
PROPER INVERSE
M<-3 3p4 5 _6 1 2 0 1 1 1
M M is another full matrix, invertible

4  5  _6
1  2  0
1  1  1
M1<-| % | M
M1

0.222222  _1.22222  1.33333
_0.111111  1.11111  _0.666667
_0.111111  0.11111  0.333333
I_36
PROPER INVERSE
dM Determinant of M
_9
M1x9 M1 can be multiplied by 9
to obtain an integer matrix

2  _11  12
_1  10  _6
_1  1  3

```

```

      V<-1 2 3 4 5
      +\V
1 3 6 10 15
      M<-3 4pi12
      +\M

      1 3 6 10
      5 11 18 26
      9 19 30 42
      +\[1]M

      1 2 3 4
      6 8 10 12
      15 18 21 24
      'an'/^\'Banana'
2 4 6 3 5 1
      'an'\v/'Banana'
1 3 5 2 4 6
      3 1 4 1 5~5 1
3 4
      (i10)~5 6 7
1 2 3 4 8 9 10
      6 2f3.125 0.002
3.12 0.00
      M<-2 3pi6
      6 2fM

      1.00 2.00 3.00
      4.00 5.00 6.00
      6 2 6 3 8 4fM

      1.00 2.000 3.0000
      4.00 5.000 6.0000
      6 2f1234
*****
      M%2.4

      0.416667 0.833333 1.25
      1.66667 2.08333 2.5
      3fM%2.4

      0 0 1
      1 2 2
      e'1+1'
2
      V<-1 2 3
      e'V'
1 2 3
      e"T<-'string' "
      T
string

```

Scanning vector V

Scanning matrix M along rows

Scanning matrix M along columns

Dyadic grade up, set collating
using 'an'

Dyadic grade down, setting inverse
collating sequence

Exclude elements

Formatted output

Format for M applies to all elements

Format for M is specific for
each column

Format is too short for fit

Integer format in 3 spaces

Execute the expression 1+1

Show vector V

Execute an assignment
Is is retained? Yes!

1	V=_1 2 3	Do V and 1 2 3 match?
		Yes
1	V=_1 2	Do V and 1 2 match?
0		No
	V=1 2 3	The equal sign works
1 1 1		element-by-element, and when
	V=1 2	length does not match,
LENGTH ERROR		the error appears
	V=1 2	
	^	
	'b' "e" 'x' =_ 'bex'	Match =_ works also for chars
1		
	'b' "e" 'x' =_ 'bey'	char characters need not be coherent
0		
	1=_1 1	Match =_ applied on numeric items
0		
	\$45 \$67	The hexadecimal format
69 103		
	\$10+16	Mixed sum decimal/hex
32		
	#'Banana'	The first values in string
1 1	1 0 0 0	in positions
	#'Mississippi'	
1 1	1 0 0 0 0 0 1 0 0	
	u'Banana'	The first values in string
Ban		visualized
	u'Mississippi'	
Misp		
	u2 3 2 4 2 5 2 6	Returned unique values
2 3	4 5 6	
	r[2]M	Linear dependence of 2x3 M
2		
	rM	Echelon version of 2x3 M
1 2 3		
0 1 2		
	MM<-4 4p1 2 4 2 2 4 8 4 3 4 5 6 1 1 1 1	
	MM	MM is a 4x4 matrix
1 2 4 2		
2 4 8 4		
3 4 5 6		
1 1 1 1		
	rMM	Visualization under echelon form
1 2 4 2		
0 1 3.5 0		
0 0 1 -2		The fourth row is null, this
0 0 0 0		reduces rank by 1 (4-1 = 3)
	r[2]MM	Linear dependence of 4x4 MM
3		
	r[3]MM	Linear dependent rows
3		
	r[4]MM	Linear dependent columns
4		

TONIBIN SIGNED OFF

8. THE LIBRARY

In this chapter I will comment some programs that I report here, written by others or by me, that may reveal useful both for usage and for learning how APL works.

I tell every one of you that I'm not a great programmer and that it needs a lot of professional skill to conceive the neat programs that are reported in the APL77, which show the necessary mastering; a skill that I haven't got, yet. And, please, note that the programs here discussed are not meant for great numbers volumes, wide matrices or the like: they were created to show the mastering of APL, not for serious number-crunching techniques. So, I'll be enormously happy if you tell me "*Hey! I've been able to process a 100×100 matrix!*", but please don't tell me "*Eech! Your programs couldn't even treat a 20×20 matrix!*", because this is not their duty. Not at least at the APL programming level that I can master today.

Much of the matrix programs have a proper APL extended counterpart, that you can read about in the *apple Extended Features Manual* in the package.

Anyway, here they are, all made by me (except where indicated):

- FACTOR, a program for factorizing an integer (and finding if it's prime); this program was written by Ian Jones.
- PRIMES, a fast program for finding all primes into an interval (commented in detail); this program was in the APL68, and it supposedly was written by one of its authors.
- PLOT, a program to plot data contained into a matrix; the data may be anything suitable to be shown into a specific grid; this program was made by me, trying to mimic the program (to me unknown) of the APL11 plotting program output.
- FFPLOT, a program to plot a function, with X as the abscissa value and Y as the ordinate value (as in the Cartesian world), and the result plotted in a specified abscissa-range; this program sizes the dimensions automatically, and so it's suitable for functions that have a limited range in the ordinates.
- FFPLOTL, a program to plot a function, with X as the abscissa value and Y as the ordinate value (as in the Cartesian world), and the result plotted in a specified both in the abscissa-range and in the ordinate-range; this program plots functions more realistically than FFPLOT, because plots are in the range specified by the user.
- INV and INVP, two programs to find the inverse matrix (only for invertible matrices with a non-null determinant), the second using pivoting; these wonderful programs were in the APL77, and were supposedly written by one or more of its authors.
- QRD, a program to perform the QR Decomposition of a square matrix A so that $A = Q \cdot R$, where Q is an orthonormal matrix and R an upper triangular matrix. This algorithm is useful for solving the least square problem (see also the `q` operator).
- HESS, a program to perform the Hessenberg Decomposition of a square matrix A so that $A = Q \cdot H \cdot Q^T$, where H is the Hessenberg matrix with zeroes under the main sub-diagonal, and Q is an orthonormal matrix; this reduction maintains the eigenvectors and eigenvalues of the original matrix, but since it's almost triangular, the computation should be lighter (see also the `h` operator).
- LUD, a program to perform the LU Decomposition of the argument matrix A so that $A = L \cdot U$, where L is a lower triangular matrix and U is an upper triangular matrix. This decomposition can be performed only on invertible matrices, that is matrices with non-null determinant. This decomposition is useful for solving systems of linear equations in matrix form.
- JORD, a program to perform the Jordan Decomposition of a square matrix M so that $M = P \cdot J \cdot P^{-1}$, where P is the normalized eigenvectors matrix and J the eigenvalues matrix, using the QR algorithm and the solving of the main equation.
- SCHUR, a program to perform the Schur Decomposition of a square matrix A so that $A = U \cdot Q$, where U is an upper triangular matrix and Q is an orthonormal matrix, using the non-normalized version of the QR algorithm.

- EIGEN, a simple program to perform the Rayleigh iteration to find the normalized eigenvectors of a given matrix A (right argument) so $A = P \cdot J \cdot P^{-1}$, where P is the eigenvectors matrix and J the eigenvalues matrix, the same output of JORD. This program sometimes solves the irregular eigenvectors matrix P returned by JORD.
- SVD, a program to perform the Singular Value Decomposition of a matrix A so that $A = U \cdot \Sigma \cdot V^T$, where U and V are orthonormal matrices and Σ is a diagonal matrix. This program uses the non-normalized version of the QR algorithm (see also the s operator).
- POL, a program to perform the Polar Decomposition of a square matrix A so that $A = U \cdot P$, where U is an orthogonal matrix, and P is a positive semi-definite symmetric matrix. This program uses the SVD algorithm.
- KON, a program to calculate the condition number of the matrix argument (see also the k operator).
- INVMAT, a general program to find the inverse of any matrix, using INV for invertible matrices (those with non-null determinant) and the Ben-Israel algorithm for non-invertible matrices, to find the pseudo-inverse.

8.1. Mathematical paraphernalia

I present here some tools that will be used in the next programs; they are simple aids in calculation, normal stuff in linear algebra. For the technicalities, please refer to the specific listings.

The mathematical paraphernalia are:

- τ EIGTEST and EIGRES, simple dyadic test programs that show if the collected eigenvalues in the diagonal matrix result (left argument) are precise, and to what extent, giving the original matrix as the right argument. The program EIGTEST returns a zero (for failure) or a one (for success). The program EIGRES, on the other hand, returns a 2×2 matrix with the traces of the first line and the determinants on the second. They may be influenced by the tools) DIGIT,) FUZZ or) COMP.
- τ EYE, a monadic function that returns the identity matrix with the dimensions specified by the scalar given as argument; the name is inspired by the Python libraries.
- τ DIFF, a dyadic function to calculate the difference between two matrices; the two matrices must concord in rows and column numbers.
- τ IDEN, a monadic function to produce the identity matrix equal in dimension to the argument, which must be a square matrix. For non-square matrices, the returned dimension is the number of columns of the argument matrix.
- τ NORM, a monadic function to calculate the norm of a horizontal vector (or simply vector) and of a matrix column.
- τ NULL, a monadic function that return a zero matrix with the same dimensions of the matrix used as argument, which may have any dimension.
- τ TRACE, a monadic function that returns the trace of any matrix or vector given as argument.
- τ ZERO, a dyadic function that returns a zero matrix with the dimensions specified by the arguments, so that $M \text{ ZERO } N$ constructs a zero matrix with M rows and N columns.

8.2. Factoring an integer

The integer factorization is normal in integer calculus, and APL cannot avoid dealing with it.

In the library, there is the file FACTOR.apl program; when you) IMPORT this program, the function will be found under the FACTOR name.

When executed, it returns a vector containing the factors of the argument, with their multiplicity:

```

      FACTOR 10
2 5
      FACTOR 24
2 2 2 3

```

```
        FACTOR 9901
9901
        FACTOR 86750
2  5  5  5  347
```

As you can see in the listing, the program is a pure transcription of a Pascal version of the same algorithm, and it may sound naive, but nonetheless shows how APL treats things, by simply appending a result to the previous list.

This algorithm can also find primes: when the output is a single element, this means there are no other factors, so it's prime (as 9901).

Thanks, Ian for this program.

8.3. A commented program: finding primes

The APL68 shows, at page 132 the following program (translated here to ASCII chars suitable for apple):

```
\ / PRIMES
P<-1+T<-1
TEST:->(END<=P,P) /PRINT
ADD:->(v/0=P | T<-T+2) /ADD
P<-P,T
->TEST
PRINT: P
\ /
```

The program is contained in the file `PRIMES.apl` in the library. In the following, I will try to explain how this works, to show the power of the APL reasoning.

Let's begin.

The program header⁶³ is plain, no arguments, no return value. This means that the program is executed by simply calling its name, and the result must be supplied in the code itself.

```
\ / PRIMES
```

The first instruction sets T=1 and P=2 (that is 1+T); P contains the first prime, which is 2; T contains the first item to try (which is 1), but the algorithm adds 2 before starting, so that the first effective first item to try is 3. See ahead.

```
P<-1+T<-1
```

The second instruction sets for quitting if P is longer than END (END must be assigned before the start of the program) If true, before quit, it jumps to label PRINT, which prints the value of P, which is an array (we'll see why).

```
TEST:->(END<=P,P) /PRINT
```

The third instruction sets T=T+2, then obtains the P | T sequence list of remainders (by dividing all items in P by T and storing the remainders), checks for the equality to zero, then compresses it with 'or', that is, if at least one element returns a non-null remainder, this becomes 1, otherwise it's 0; the result is then compressed to ADD; it returns ADD (which is equal to 3) if the former 'or' compression gave 1, or it's the empty array, in which case it skips to the next line. This instruction is repeated, because the scope is finding the next prime, which is not divisible by any of the primes already in memory, returning zero in the 'or' compression of the zero equality. The key of the program is that P, which contains only primes, is used to test all the following numbers with a single instruction P | T, which returns the list of remainders for the number under test, with the rule that a prime has no null remainders.

```
ADD:->(v/0=P | T<-T+2) /ADD
```

The fourth instruction adds T (which is prime, since the 0=P | T in the last run gave all zeroes) to P (through

⁶³ Notice that the original name in the APL68 was PR, but I changed it to PRIMES which is clearer.

comma), and the list increases by one item.

```
P<-P,T
```

Now, jump back to TEST to begin a new sequence. ->TEST

This line simply prints the list of primes and exits.

```
PRINT:P
```

```
V
```

The simplicity of this algorithm does not reflect the complexity of its design. The reasoning behind is tough and based on thinking in terms of pure APL, while all of us are still anchored to the succession of steps related to the sieve algorithm concept studied at school.

Here is an example of execution under **apple**, looking for the first 50 primes (in the experiment, the screen length was reduced to 70 characters, to make it fit this manual page):

```
      )IMPORT primes
PRIMES IMPORTED
      )LIST PRIMES
      )LIST PRIMES
      \/\ PRIMES
[1]   P<-1+T<-1
[2]   TEST:->(END<=p,P)/PRINT
[3]   ADD:->(v/0=P|T<-T+2)/ADD
[4]   P<-P,T
[5]   ->TEST
[6]   PRINT: P
      \/\
      END<-50
      PRIMES
2  3  5  7  11  13  17  19  23  29  31  37  41  43  47  53  59  61
    67  71  73  79  83  89  97  101  103  107  109  113  127  131
    137  139  149  151  157  163  167  173  179  181  191  193
    197  199  211  223  227  229
```

The following last example shows instead how to run the same program from console (file `primes.apl`) and how to give it a starting END value:

```
$ apple --apl="END<-50" primes.apl
2  3  5  7  11  13  17  19  23  29  31  37  41  43  47  53  59  61
    67  71  73  79  83  89  97  101  103  107  109  113  127  131
    137  139  149  151  157  163  167  173  179  181  191  193
    197  199  211  223  227  229
```

The algorithm is incredibly efficient; on my modest PC, to find the first 1000 primes it takes 1.20÷1.25 seconds, and on your PC this result could be even better.

To obtain this result, I ran the following command in the directory where I develop **apple**; the program is the same of the previous example, with END fixed this time to 1000, and run with option `-t` (the extension `.apl` is omitted because non necessary, here):

```
$ ./apple -t --apl="END<-1000" primes
2  3  5  7  11  13  17  19  23  29  31  37  41  43  47  53  59  61
    67  71  73  79  83  89  97  101  103  107  109  113  127  131
```

137	139	149	151	157	163	167	173	179	181	191	193
197	199	211	223	227	229	233	239	241	251	257	263
269	271	277	281	283	293	307	311	313	317	331	337
347	349	353	359	367	373	379	383	389	397	401	409
419	421	431	433	439	443	449	457	461	463	467	479
487	491	499	503	509	521	523	541	547	557	563	569
571	577	587	593	599	601	607	613	617	619	631	641
643	647	653	659	661	673	677	683	691	701	709	719
727	733	739	743	751	757	761	769	773	787	797	809
811	821	823	827	829	839	853	857	859	863	877	881
883	887	907	911	919	929	937	941	947	953	967	971
977	983	991	997	1009	1013	1019	1021	1031	1033	1039	
1049	1051	1061	1063	1069	1087	1091	1093	1097	1103		
1109	1117	1123	1129	1151	1153	1163	1171	1181	1187		
1193	1201	1213	1217	1223	1229	1231	1237	1249	1259		
1277	1279	1283	1289	1291	1297	1301	1303	1307	1319		
1321	1327	1361	1367	1373	1381	1399	1409	1423	1427		
1429	1433	1439	1447	1451	1453	1459	1471	1481	1483		
1487	1489	1493	1499	1511	1523	1531	1543	1549	1553		
1559	1567	1571	1579	1583	1597	1601	1607	1609	1613		
1619	1621	1627	1637	1657	1663	1667	1669	1693	1697		
1699	1709	1721	1723	1733	1741	1747	1753	1759	1777		
1783	1787	1789	1801	1811	1823	1831	1847	1861	1867		
1871	1873	1877	1879	1889	1901	1907	1913	1931	1933		
1949	1951	1973	1979	1987	1993	1997	1999	2003	2011		
2017	2027	2029	2039	2053	2063	2069	2081	2083	2087		
2089	2099	2111	2113	2129	2131	2137	2141	2143	2153		
2161	2179	2203	2207	2213	2221	2237	2239	2243	2251		
2267	2269	2273	2281	2287	2293	2297	2309	2311	2333		
2339	2341	2347	2351	2357	2371	2377	2381	2383	2389		
2393	2399	2411	2417	2423	2437	2441	2447	2459	2467		
2473	2477	2503	2521	2531	2539	2543	2549	2551	2557		
2579	2591	2593	2609	2617	2621	2633	2647	2657	2659		
2663	2671	2677	2683	2687	2689	2693	2699	2707	2711		
2713	2719	2729	2731	2741	2749	2753	2767	2777	2789		
2791	2797	2801	2803	2819	2833	2837	2843	2851	2857		
2861	2879	2887	2897	2903	2909	2917	2927	2939	2953		
2957	2963	2969	2971	2999	3001	3011	3019	3023	3037		
3041	3049	3061	3067	3079	3083	3089	3109	3119	3121		
3137	3163	3167	3169	3181	3187	3191	3203	3209	3217		
3221	3229	3251	3253	3257	3259	3271	3299	3301	3307		
3313	3319	3323	3329	3331	3343	3347	3359	3361	3371		
3373	3389	3391	3407	3413	3433	3449	3457	3461	3463		
3467	3469	3491	3499	3511	3517	3527	3529	3533	3539		
3541	3547	3557	3559	3571	3581	3583	3593	3607	3613		
3617	3623	3631	3637	3643	3659	3671	3673	3677	3691		
3697	3701	3709	3719	3727	3733	3739	3761	3767	3769		
3779	3793	3797	3803	3821	3823	3833	3847	3851	3853		
3863	3877	3881	3889	3907	3911	3917	3919	3923	3929		
3931	3943	3947	3967	3989	4001	4003	4007	4013	4019		
4021	4027	4049	4051	4057	4073	4079	4091	4093	4099		
4111	4127	4129	4133	4139	4153	4157	4159	4177	4201		
4211	4217	4219	4229	4231	4241	4243	4253	4259	4261		
4271	4273	4283	4289	4297	4327	4337	4339	4349	4357		

4363	4373	4391	4397	4409	4421	4423	4441	4447	4451
4457	4463	4481	4483	4493	4507	4513	4517	4519	4523
4547	4549	4561	4567	4583	4591	4597	4603	4621	4637
4639	4643	4649	4651	4657	4663	4673	4679	4691	4703
4721	4723	4729	4733	4751	4759	4783	4787	4789	4793
4799	4801	4813	4817	4831	4861	4871	4877	4889	4903
4909	4919	4931	4933	4937	4943	4951	4957	4967	4969
4973	4987	4993	4999	5003	5009	5011	5021	5023	5039
5051	5059	5077	5081	5087	5099	5101	5107	5113	5119
5147	5153	5167	5171	5179	5189	5197	5209	5227	5231
5233	5237	5261	5273	5279	5281	5297	5303	5309	5323
5333	5347	5351	5381	5387	5393	5399	5407	5413	5417
5419	5431	5437	5441	5443	5449	5471	5477	5479	5483
5501	5503	5507	5519	5521	5527	5531	5557	5563	5569
5573	5581	5591	5623	5639	5641	5647	5651	5653	5657
5659	5669	5683	5689	5693	5701	5711	5717	5737	5741
5743	5749	5779	5783	5791	5801	5807	5813	5821	5827
5839	5843	5849	5851	5857	5861	5867	5869	5879	5881
5897	5903	5923	5927	5939	5953	5981	5987	6007	6011
6029	6037	6043	6047	6053	6067	6073	6079	6089	6091
6101	6113	6121	6131	6133	6143	6151	6163	6173	6197
6199	6203	6211	6217	6221	6229	6247	6257	6263	6269
6271	6277	6287	6299	6301	6311	6317	6323	6329	6337
6343	6353	6359	6361	6367	6373	6379	6389	6397	6421
6427	6449	6451	6469	6473	6481	6491	6521	6529	6547
6551	6553	6563	6569	6571	6577	6581	6599	6607	6619
6637	6653	6659	6661	6673	6679	6689	6691	6701	6703
6709	6719	6733	6737	6761	6763	6779	6781	6791	6793
6803	6823	6827	6829	6833	6841	6857	6863	6869	6871
6883	6899	6907	6911	6917	6947	6949	6959	6961	6967
6971	6977	6983	6991	6997	7001	7013	7019	7027	7039
7043	7057	7069	7079	7103	7109	7121	7127	7129	7151
7159	7177	7187	7193	7207	7211	7213	7219	7229	7237
7243	7247	7253	7283	7297	7307	7309	7321	7331	7333
7349	7351	7369	7393	7411	7417	7433	7451	7457	7459
7477	7481	7487	7489	7499	7507	7517	7523	7529	7537
7541	7547	7549	7559	7561	7573	7577	7583	7589	7591
7603	7607	7621	7639	7643	7649	7669	7673	7681	7687
7691	7699	7703	7717	7723	7727	7741	7753	7757	7759
7789	7793	7817	7823	7829	7841	7853	7867	7873	7877
7879	7883	7901	7907	7919					

Time: 1.22 s

To find the first 2000 primes (up to the value 17389) it takes only 4.14 seconds on my computer. Amazing, astonishing and astounding⁶⁴.

8.4. Plotting data

The APL11 manual reports the output of a graphic program by Michael Cain, a program that is not in the sources. So I decided to write a program (a naive program, I admit) to plot data elements, contained in the `PLOT.apl` file.

This program plots the data contained in the right argument, with abscissa values contained in column one; the

⁶⁴ ...because the programs I created until now are kind of slow. This version of APL may be not the fastest on earth, but this result makes me proud of apple!

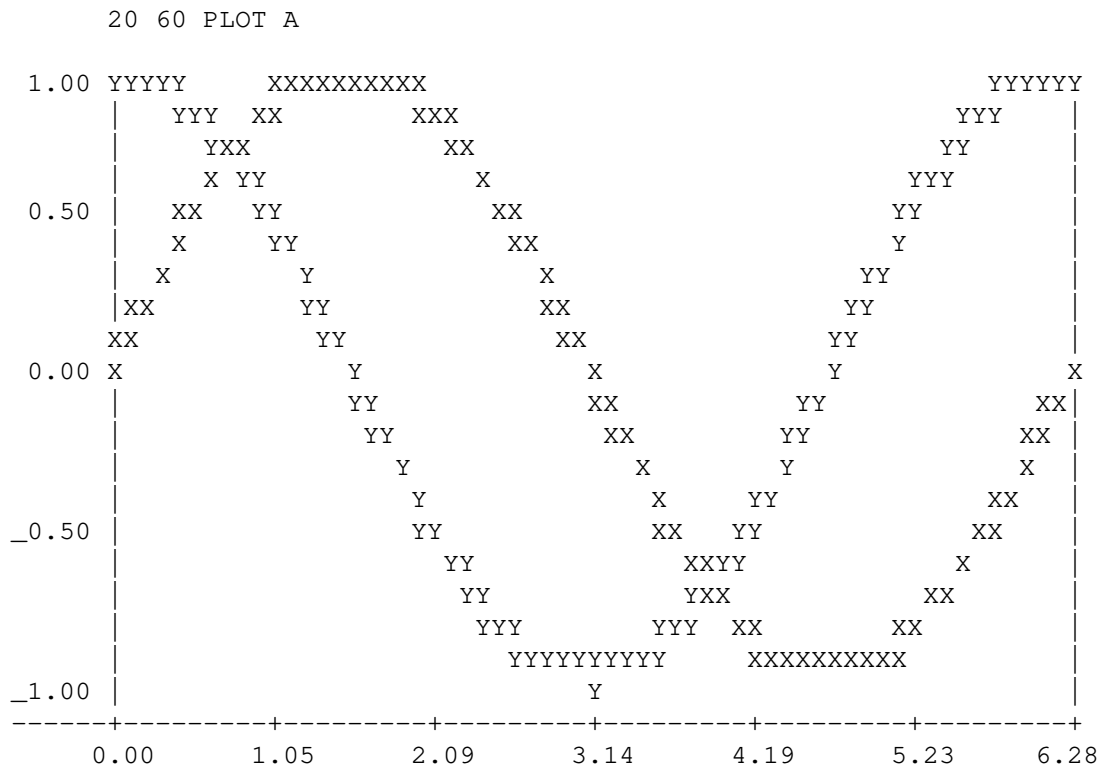
following columns, up to 12 (that is the argument may have any number of lines and up to 13 columns in total) contain the ordinate values, one series for each column. Each column will be represented with a different letter symbol. The left argument is a vector with two elements, the first being the ordinate output dimension and the second the abscissa output dimension of the graphic. The data in the columns from the second on must refer to the same x -values, because the dimension of the output depends on this.

To use it, I do quite the same that Mr. Cain did, that is creating a one column matrix, map it to the interval $[0, 2\pi]$ and then composing a 3 column matrix like this:

```
A<-101 1pi101
A<-A-1
A<-Ax0.0628
A<-A, (1oA), 2oA
```

This sequence creates a 101×3 matrix, with the x -values in the first column, the sinus of x in the second, and the cosine of x in the third; as said, x was mapped to the interval $[0, 6.28]$, which is about $[0, 2\pi]$.

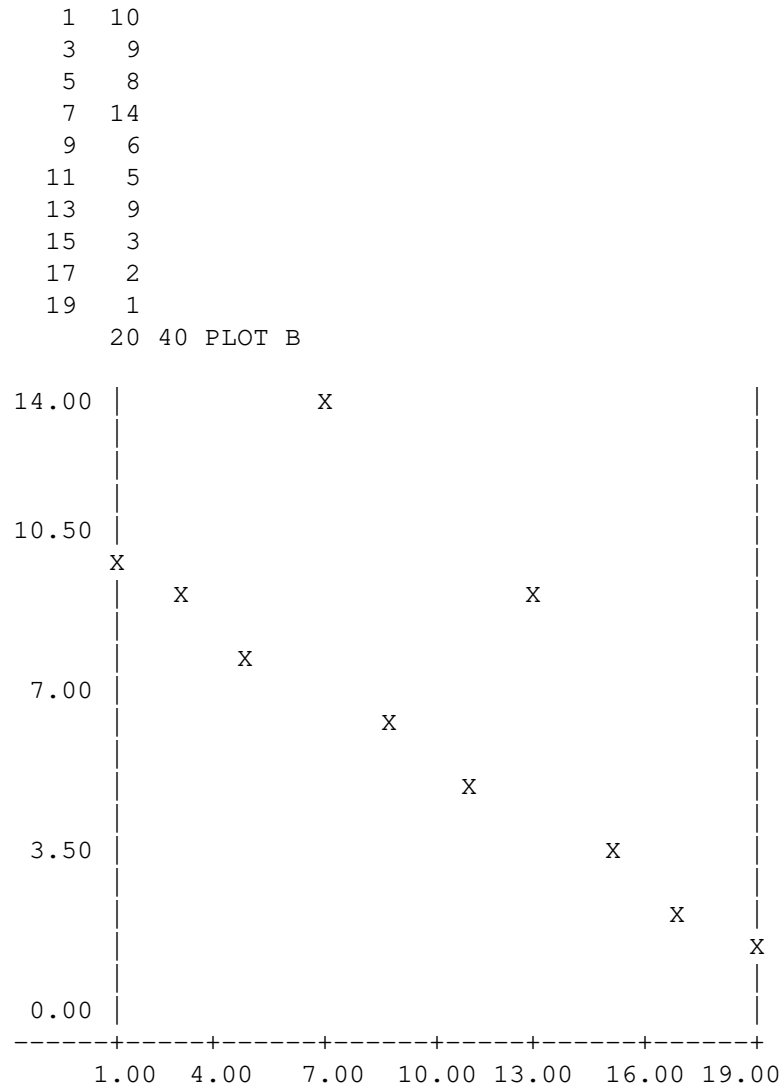
This is the output of the program, using the same command of Mr. Cain:



The X symbol shows the sinus flow, the Y symbol the cosine flow. The numbers on the left side show the ordinate space, the values on the final row the abscissa space. As you can see in the listing, PLOT uses the operator f for the formatting output. The output is returned as a string matrix, that can be stored in a variable, and used elsewhere.

This program can be used with any kind of data; for instance, see the following data, where the column 1 identifies the year (1 is 2001, 3 is 2003 and so forth), and the second column is some data about those years (for example the number of times I fell to the floor that year):

B



This is a plain column output of data, where the X shows the top level of the column; this is a simple output, that can be bettered, of course. But this shows also the power of this function.

Of course, you should choose intervals that fit the plot, since the characters are not dots, and the continuity may gets compromises. The following example shows why.

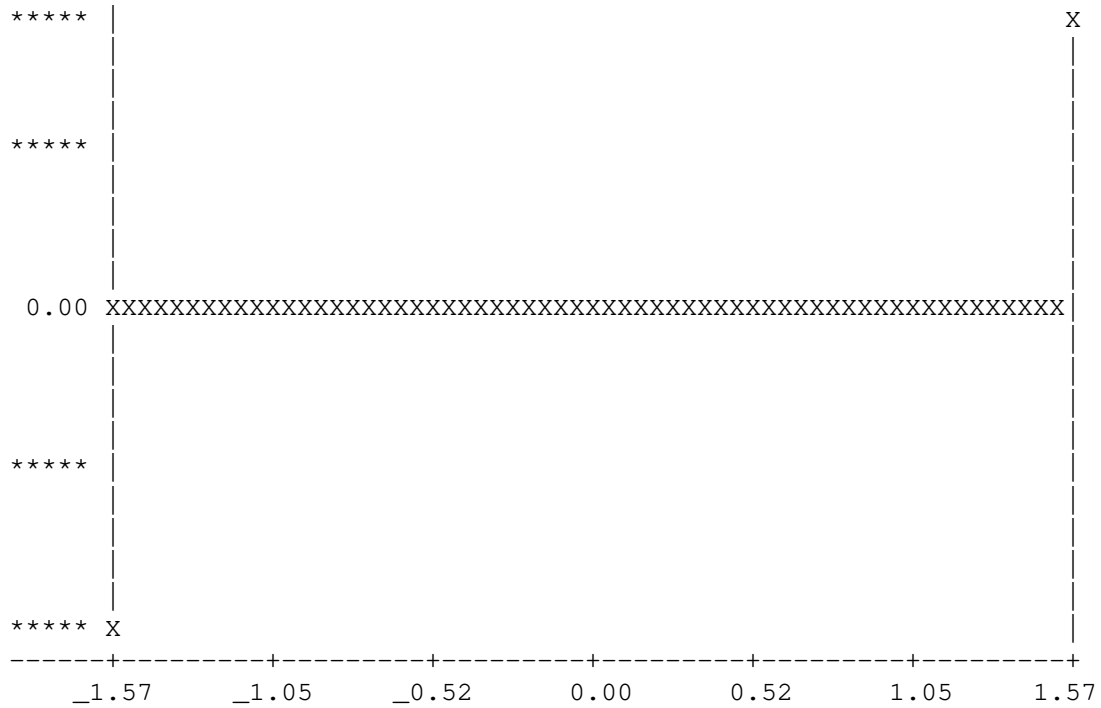
Let's take the tangent function, and let's build the matrix in the interval $[-\pi, \pi]$.

```

A<-101 1pi101
A<-A-51
A<-Ax0.0314
B<-A, (3oA)
20 60 PLOT B

```

The result is the following:

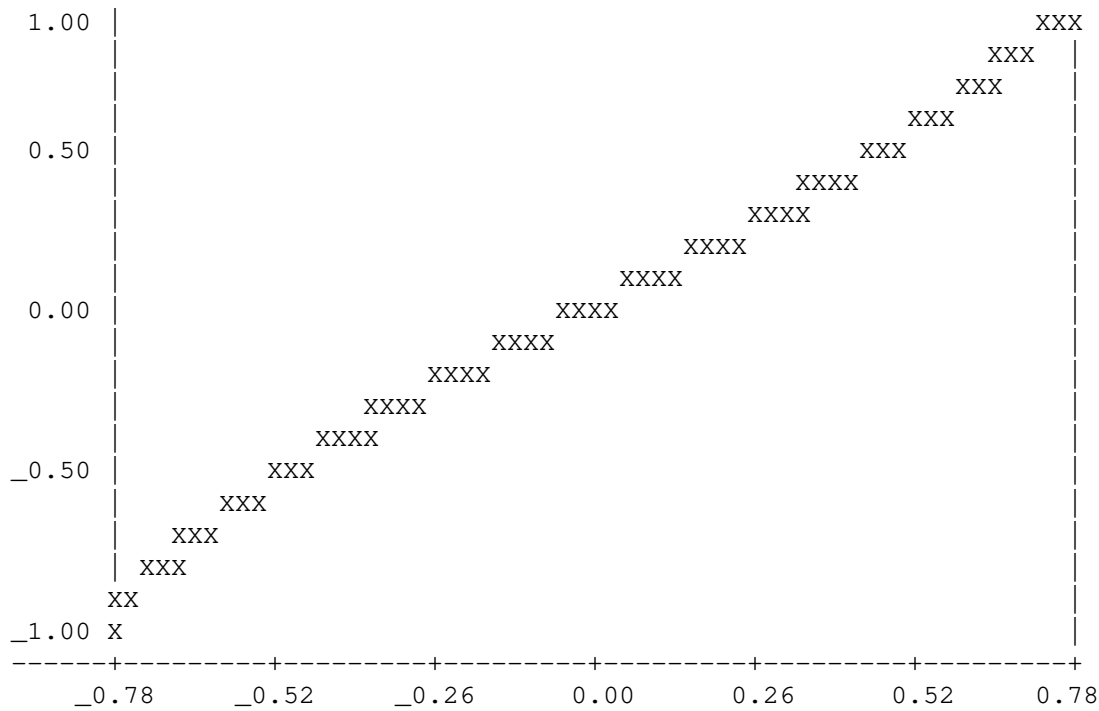


As you can see, the extreme values are too big for the 20 characters required for plot, so that all the remaining values gets compressed in the line 0.00 (see for this the FPLLOT and FPOLTL programs).

Now let's use a tight interval:

```
A<-101 1pi101
A<-A-51
A<-Ax0.0157
B<-A, (3oA)
20 60 PLOT B
```

This time we choose to work in the interval $[-\frac{\pi}{2}, \frac{\pi}{2}]$: the result is:



As you can see, the light wave that is shown depicts the tangent, but the approximation is low.

A final example tries to replicate the famous curve that appears on the plotting routines in "*APL functions for interactive data analysis - Graphics and labels*", by Evans, Neideffer and Gage (1980); the data are contained in the file GRAPHITE.txt" (to save you from typing all those numbers by hand), that you can use this way:

```
Z<-1|v|'GRAPHITE'
R<-1|<|1
Z<-1|v|0
```

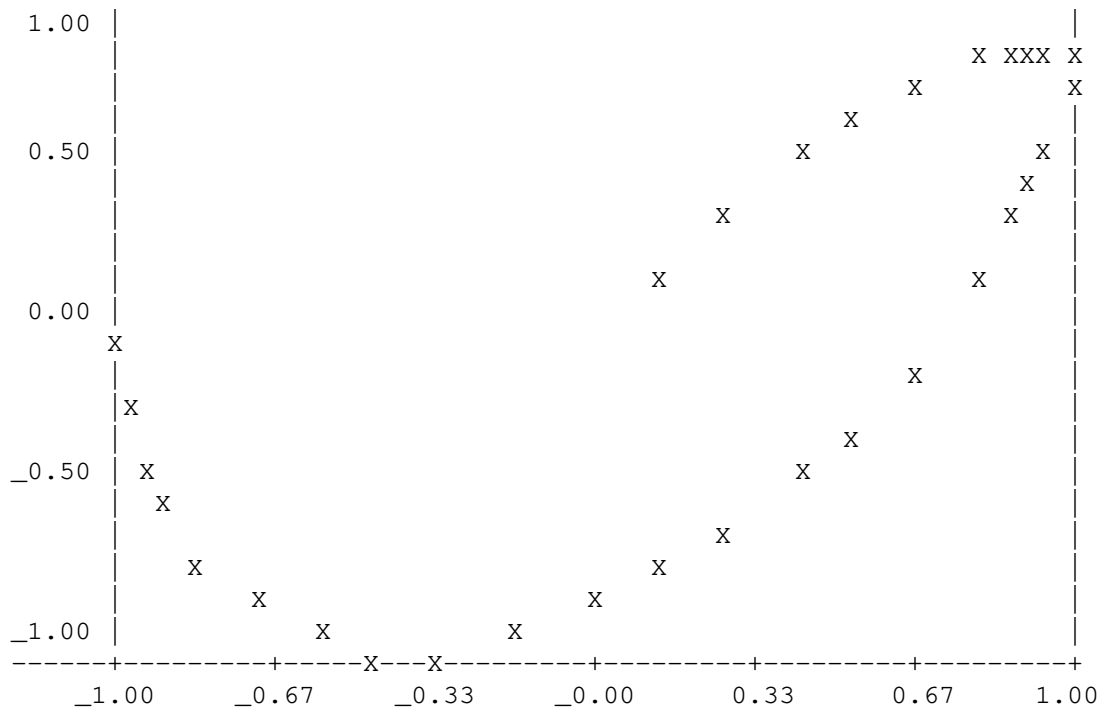
This operation will load the 31x2 matrix from the file GRAPHITE, copied here for you convenience:

```
R
  _1      0
 _0.96  _0.2
 _0.92  _0.43
 _0.88  _0.5
  _0.8   _0.76
 _0.68  _0.84
 _0.56  _0.92
 _0.44   _1
 _0.32   _1
 _0.16  _0.94
   0    _0.84
  0.16  _0.76
  0.28  _0.6
  0.44  _0.43
  0.56  _0.3
```

0.68	_0.12
0.8	0.18
0.88	0.38
0.92	0.52
0.96	0.63
1	0.83
1	0.91
0.96	0.96
0.92	1
0.88	0.96
0.8	0.91
0.68	0.83
0.56	0.63
0.44	0.58
0.28	0.38
0.16	0.2

The plotting is easily done through:

```
20 60 PLOT R
```

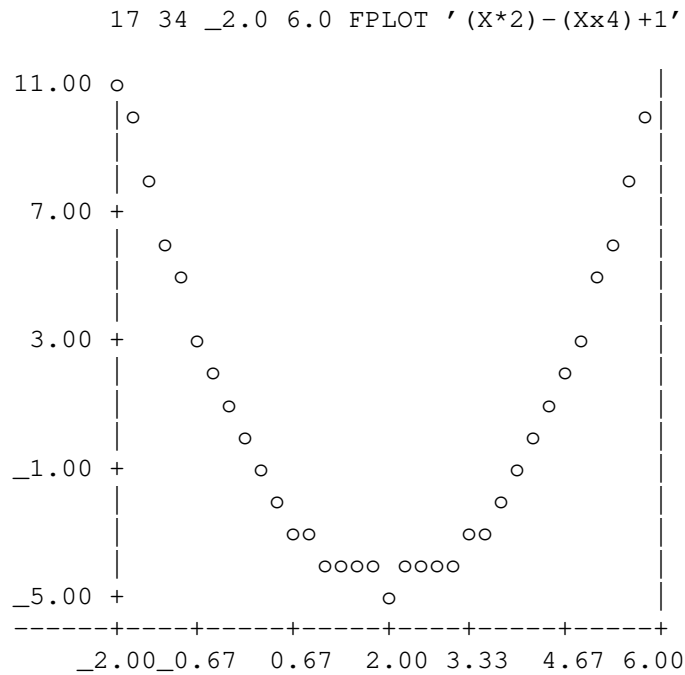


8.5. Plotting a function

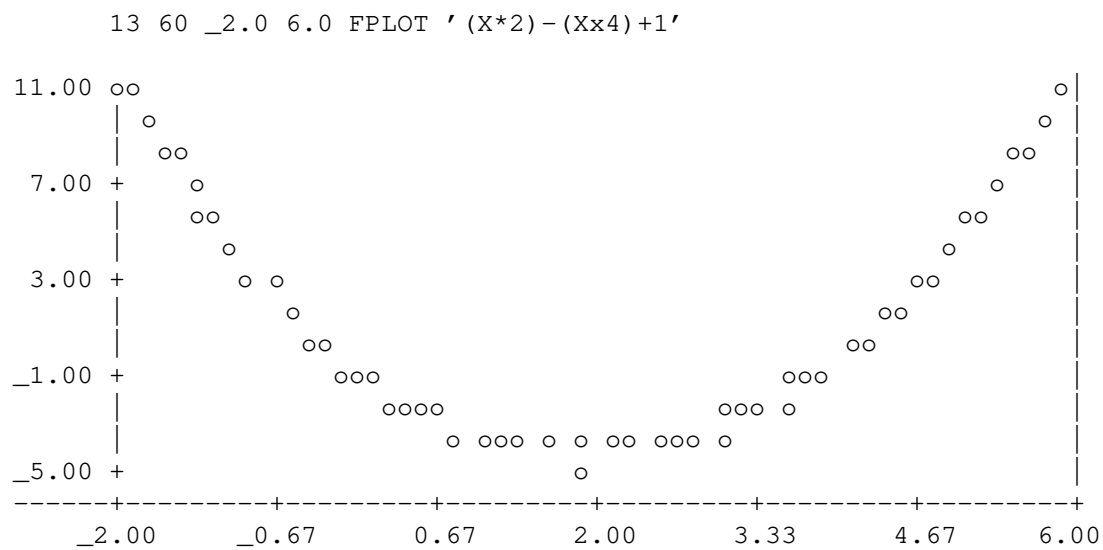
Extending the graphic program by Michael Cain, I developed two programs to print functions. The listing of the first is contained in the `FPLLOT.apl` file.

This program plots a function, contained into a string given in the right argument; the formula must be a real APL formula, that will be evaluated by means of the operator `⍷`. The left argument is a vector with four elements, the first being the ordinate output dimension and the second the abscissa output dimension of the graphic, the third the lower x -dimension and the fourth the higher x -dimension (the function output domain). The ordinate range is calculated automatically, so this program is suitable for functions that have limited values in the ordinate range.

The execution does not require any matrix, because the matrix is built inside the program. Here is an example of execution:

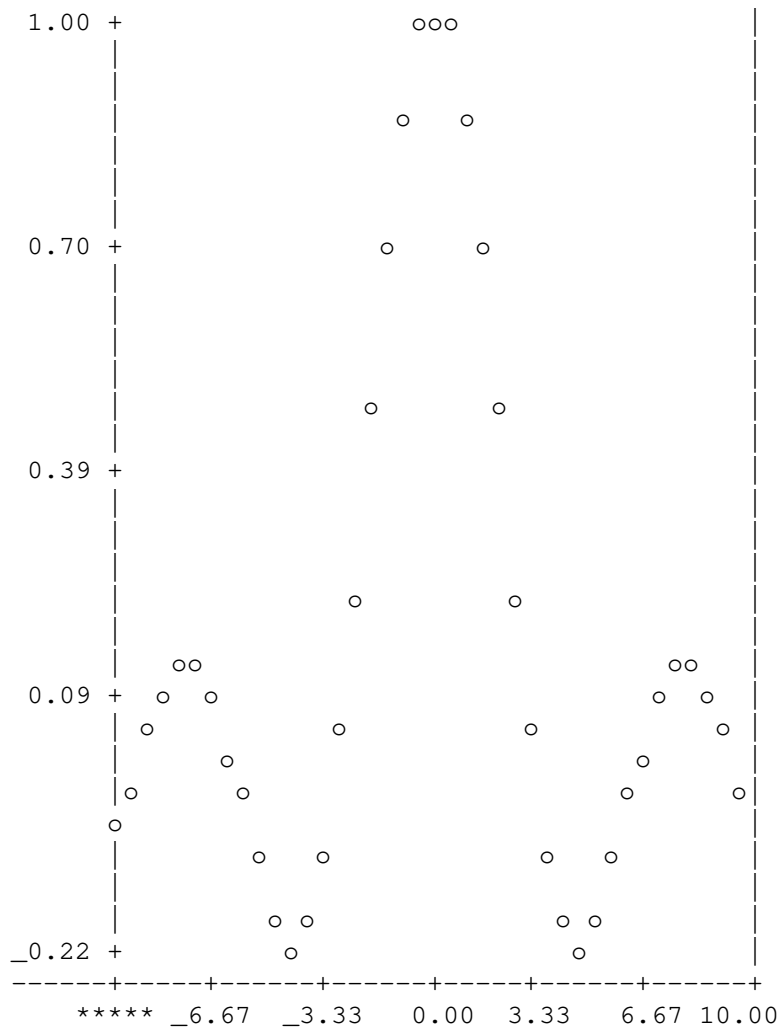


Since, in the terminal, the character's height is twice as its width, I invoked the command with 17 34, to obtain a proportional dimension in x and y (not well rendered in this page), but you can size it as you like:



Another examples uses the famous $\frac{\sin(x)}{x}$ ratio, choosing the interval so as to appreciate the first side waves:

```
30 40 _10.0 10.0 FPLOT '(1/X)%X'
```



The ***** sequence is a simple result of the fact that five digits are printed in the formatter operator, for the abscissa and the ordinate markers, but _10.00 needs six characters...

Please note also that this result is achieved because in APL, the division $0\%0$ returns 1, which is appalling, confronting to the usual behaviour of the common and modern languages; this rule has a mathematical support, of course, demonstrated by the fact that $\lim_{x \rightarrow 0} \frac{\sin(x)}{x} = 1.0!$

To demonstrate it, take a value ε small as needed; if it's sufficiently small, we can use the Taylor series to demonstrate that $\sin \varepsilon \approx \varepsilon$, so that, again if ε is sufficiently small, the ratio becomes:

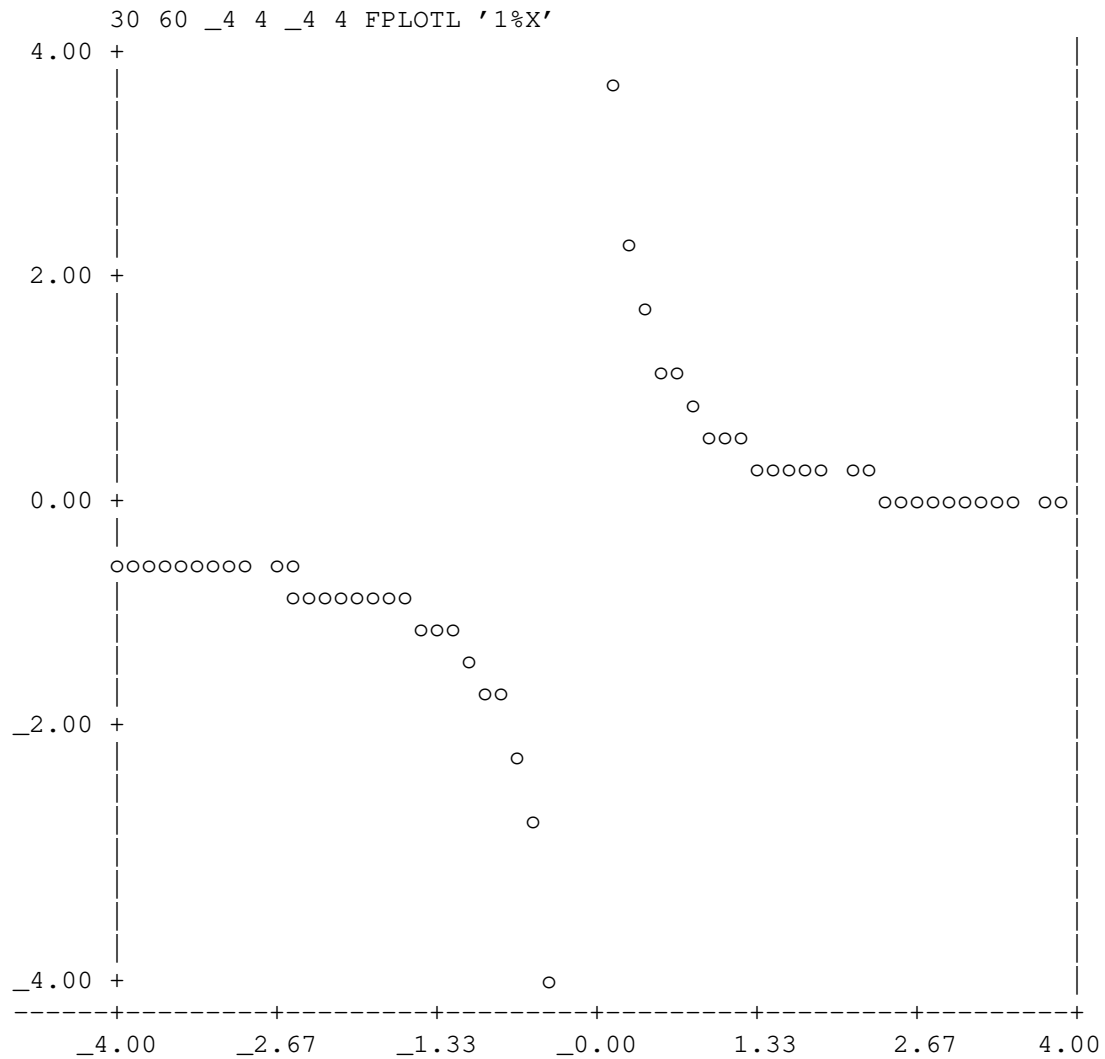
$$\lim_{x \rightarrow \varepsilon} \frac{\sin x}{x} \approx \frac{\varepsilon}{\varepsilon} = 1.$$

When ε is lesser than the mathematical resolution of the machine, it's like saying that x has been reduced to almost zero; extending the result to the limit, when $x = 0$, then $\sin x = 0$ and the result is still 1. The magic of the limits.

8.6. Plotting a function giving limits

The F_{PLOT} program, based on P_{LOT}, automatically calculates the ordinate range, but the asymptotic functions produce the result that, in the scale range of the drawing, high values "crush" all other values downwards, and the graph gets distorted. F_{PLOT}L, on the other hand, also limits the field of ordinates, and allows only the values that fall within the specified range to be evaluated in the drawing, so that values are not "crushed".

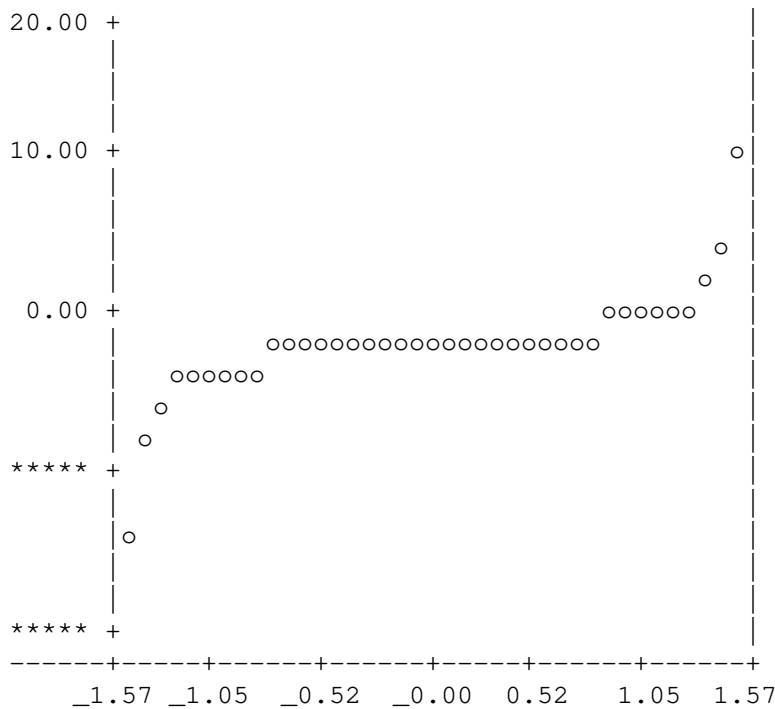
The FPLOTL program uses two further inputs, which are the minimum and maximum dimension of representation of the ordinates. Here is an example:



The asymptote in $x = 0.0$ is well represented, even in this poor graphics.

Another example with the tangent, that failed to be represented in the previous PLOT and FPLOTL cases:

20 40 _1.57 1.57 _20 20 FPLOTL '3oX'



8.7. Inverting matrices

The APL77 manual reports the program INV (in the file INV.apl) which inverts a matrix using the Gauss-Jordan algorithm, in a version not employing pivoting, and that can be used only for regular invertible matrices. There is also another program, INVP (in the file INVP.apl), which has the same scope and uses pivoting, and which is not discussed here.

The listing of INV is the following astonishing and astounding four-lines-program, which I simply adore:

```
\ / Z<-INV M;I;J
M<-\o(1 0 +pM)p(,\oM),~J<-1<iI<-1/\pM
M<-1o|(J,1)o|[1]M-(JxM[;1])@.xM[1;]<-M[1;]%M[1;1]
->2xi0#I<-I-1
Z<-M[;i1]/\pM
\ /
```

Basing on the description in the APL77, "*The first line appends the unit vector $1 \geq iN$ as the last column of the argument and the second line [...] performs at each iteration one of the N complete inversions required. See exercise 1.40 of Iverson, "A Programming Language", Wiley, 1962.*"

I don't fully understand this program, and I'll probably keep looking at it as the primitive men looked at the lightning, wondering what the hell was it; so, take for granted that it works, and look at the examples:

```
H<-3 3pi9
H[3;3]<-_9
H
1 2 3
4 5 6
7 8 _9
INV H
```



```

_1.72222      0.777778  _0.0555556
 1.44444    _0.555556   0.111111
_0.0555556   0.111111  _0.0555556

```

The matrix H was built with `⍲9`; in this guise, H is not invertible, but changing the sign of the element in position [3;3] turns H to an invertible matrix; by contrast, see the result of the inversion operator `⍲`:

```

⍲ H
_1.72222      0.777778  _0.0555556
 1.44444    _0.555556   0.111111
_0.0555556   0.111111  _0.0555556

```

This is the very same output.

Another example, using a symmetric Hilbert matrix K:

```

K
      1      0.5  0.333333
    0.5  0.333333  0.25
0.333333  0.25  0.2
INV K
 9  _36  30
_36  192 _180
30 _180  180

```

Again, see the result of the inversion operator `⍲`:

```

⍲ K
 9  _36  30
_36  192 _180
30 _180  180

```

Now, hold your breath:

```

K+.xINV K
      1      0  _3.55271E_15
_8.88178E_16      1  _1.77636E_15
 0  _1.77636E_15      1
K+.x⍲ K
      1      0  _3.19744E_14
1.01252E_13      1  _2.66454E_14
7.37188E_14  _1.06581E_14      1

```

These last lines of output show absolutely how incontrovertibly, inevitably and inescapably, **the results of INV developed in APL are better than mine developed in C!** *Ubi maior, minor cessat...*

The INV program is also the core of the first half of INVMAT (in the file `INVMAT.apl`), a more general program to find the inverse of any matrix; if the argument is invertible (with non-null determinant), the program INVMAT uses

the very same algorithm in INV (without pivoting) to find the inverse. In case of non-invertible argument (that is also the case of non-square matrices), the pseudo-inverse is returned, using the Ben-Israel algorithm (the theory behind can be found on the *apple Extended Features Manual*).

Some examples involving non-invertible matrices (the unseen case so far):

```

M
1 2 3
4 5 6
7 8 9
INVMAT M
    _0.638889    _0.166667    0.305556
_0.0555556      0      0.0555556
    0.527778     0.166667    _0.194444

```

Here is the result of the inversion operator:

```

| % | M
    _0.638889    _0.166667    0.305556
_0.0555556      0      0.0555556
    0.527778     0.166667    _0.194444

```

In case of non square matrices, for instance:

```

F
1 2
3 4
5 6
7 8
INVMAT F
    _1          _0.5  2.13718E_15    0.5
    0.85        0.45          0.05    _0.35

```

This pseudo inverse of this case satisfies the two properties $F = F \cdot F^+ \cdot F$ and $F^+ = F^+ \cdot F \cdot F^+$; in APL:

```

FP<-INVMAT F
F+.xFP+.xF
1 2
3 4
5 6
7 8
FP+.xFP+.xFP
    _1          _0.5  2.10942E_15    0.5
    0.85        0.45          0.05    _0.35

```

It works also in case of one dimensional matrices:

```

      U
1
2
3
4
      INVMAT U
0.0333333  0.0666667  0.1  0.133333

```

As an example, this is the result of the inversion operator $| \% |$:

```

      | \% | U
0.0333333  0.0666667  0.1  0.133333

```

8.8. QR Decomposition

The QR Decomposition is a process that decomposes any matrix in one orthogonal matrix and one triangular matrix, so the inner multiplication of the two return the original matrix. The rebuild is:

$$A = Q \cdot R$$

The program `QRD`⁶⁵ does not return any value; it rather creates two global variables `Q` (the orthogonal matrix) and `R` (the upper triangular matrix) which will hold the results of computation. It's contained in the file `QRD.apl`.

The theory behind the QR Decomposition can be found in the *apple Extended Features Manual*; here will follow some examples for the usage of `QRD`:

```

      M
1  2  3
4  5  6
7  8  _9
      QRD M
      Q
0.123091  0.904534  _0.408248
0.492366  0.301511  0.816497
0.86164  _0.301511  _0.408248
      R
8.12404  9.60114  _4.43129
      0  0.904534  7.23627
      0  0  7.34847
      Q+.xR
1  2  3
4  5  6
7  8  _9

```

As already said for the matrix division operator $| \% |$ and the QR Decomposition operator `q`, the QR Decomposition is mainly used to solve the least square problem. The `QRD` program can thus be used to retrieve the solutions manually; running the same example used in the matrix division operator example, we have:

⁶⁵ This program is based on the algorithms available (and converted to apple APL by me) at http://www.math.iit.edu/~fass/477577_Chapter_4.pdf.

```

M
3  5  9
2  1 _3
3 _5  2
2  4 _1
_1  2  8
4  1  1
Y

0.691473
2.48101
0.404958
0.311734
0.608437
1.47789
QRD M
X<-(| % |R)+.x(\oQ)+.xY
X

0.322974
0.0725512
_0.0276467

```

The very same results are obtained.

8.9. Hessenberg Decomposition

A square matrix H is said to be in upper Hessenberg form if it has non-zeros only in the upper triangle and the first subdiagonal. The Hessenberg Decomposition transforms a square matrix A into two special matrices H (the Hessenberg matrix) and Q (the orthogonal matrix); the recombination is possible through:

$$A = Q \cdot H \cdot Q^T$$

The program HESS (in the file HESS.apl) uses the argument square matrix to produce the two results H and Q , that can be used to reconstruct the argument.

The theory behind the Hessenberg Decomposition can be found in the *apple Extended Features Manual*; here will follow some examples for the usage of HESS:

```

M
1  2  3  4
2  3  4  5
3  4  5  6
4  5  6  7
HESS M
H

1      _5.38516  4.15513E_17  2.18122E_16
_5.38516  15.2069  _1.6893  _1.66533E_16
0      _1.6893  _0.206897  6.03764E_17
0      0      1.45717E_16  1.68622E_16

Q

```

```

1      0      0      0
0  _0.371391  0.833908  0.408248
0  _0.557086  0.15162  _0.816497
0  _0.742781  _0.530669  0.408248

```

The prove Q is orthogonal is this:

```

\oQ
1      0      0      0
0  _0.371391  _0.557086  _0.742781
0   0.833908   0.15162  _0.530669
0   0.408248  _0.816497   0.408248
| % |Q
1      0      0      0
0  _0.371391  _0.557086  _0.742781
0   0.833908   0.15162  _0.530669
0   0.408248  _0.816497   0.408248

```

The reconstruction of the original matrix M is this:

```

Q+ .xH+ .x\oQ
1  2  3  4
2  3  4  5
3  4  5  6
4  5  6  7

```

The importance of the Hessenberg reduction is that the matrix retains the same eigenvalues of the original matrix, but being quasi-upper triangular, the process of finding the eigenvalues (and the eigenvectors), if possible, is faster⁶⁶.

8.10. LU Decomposition

The LU Decomposition transforms a matrix A into two special matrices L (lower triangular) and U (upper triangular); the recombination is possible through:

$$A = L \cdot U$$

The program LUD (in the file LUD.apl) uses the argument square matrix to produce the two results L and U , that can be used to reconstruct the argument. The limit imposed is 10000 cycles (that seem sufficient).

The theory behind the LU Decomposition can be found in the *apple Extended Features Manual*; here will follow some examples for the usage of LUD:

```

MM
16      2      3      4      5
6      7      8      9     10
11     12     13     14     15
16     17     18    _16     20
21     22     23     24    _25
LUD MM
L

```

⁶⁶ It's more like of order $O(n^{N-1})$ than $O(n^N)$, if N is the dimension of the square matrix.

```

      1      0      0      0      0
0.375      1      0      0      0
0.6875     1.7      1      0      0
      1     2.4      2      1      0
1.3125     3.1      3      0      1
      U

```

```

      16      2      3      4      5
      0     6.25    6.875    7.5    8.125
      0      0    _0.75    _1.5    _2.25
      0      0      0    _35      0
      0      0      0      0    _50
      L+.xU

```

```

      16      2      3      4      5
      6      7      8      9     10
     11     12     13     14     15
     16     17     18    _16     20
     21     22     23     24    _25

```

The recombination returned the original matrix MM.

The second example uses a non-invertible matrix:

```

      M
      1      2      3      4      5
      6      7      8      9     10
     11     12     13     14      1
      2      3      4      5      6
      7      8      9     10     11
      LUD M
      L
      1      0      0      0      0
      6      1      0      0      0
     11      2      1      0      0
      2    0.2      1      1      0
      7    1.2      1      1      1
      U
      1      2      3      4      5
      0     _5    _10    _15    _20
      0      0      0      0    _14
      0      0      0      0     14
      0      0      0      0      0
      L+.xU

```

```

      1      2      3      4      5
      6      7      8      9     10
     11     12     13     14      1
      2      3      4      5      6
      7      8      9     10     11

```

Again, the recombination returned the original matrix.

Special cases. In case of sparse matrices, that typically occur with some nulls on the diagonal, the algorithm in LUD, requiring some divisions, may stop with a domain error. The solution is to enable the)DIV 1 division flag, to remove the error condition in the division by zero. Here is a session excerpt that shows how to proceed:

```

R
0 0 0 0 0
0 0 1 0 0
0 1 1 1 0
0 1 1 0 0
0 0 0 0 0
dR
0
LUD R
DOMAIN ERROR
LUD[11] L[J;I]<-U[J;I]%U[I;I]
^
)DIV 1
WAS 0
LUD R
L
1 0 0 0 0
1 1 0 0 0
1 0 1 0 0
1 0 1 1 0
1 1 -1 -1 1
U
0 0 0 0 0
0 0 1 0 0
0 1 1 1 0
0 0 0 -1 0
0 1 0 0 0
L+.xU
0 0 0 0 0
0 0 1 0 0
0 1 1 1 0
0 1 1 0 0
0 0 0 0 0

```

The first LUD invocation ended with a domain error, because R has some nulls on the main diagonal. Also note that R is not invertible, because its determinant is null. By setting)DIV 1 the correct solution was achieved without any error, because the returned matrices L and U reconstructed the original matrix R.

Another special case is the zero-diagonal case:

```

H
0 2 3 4 5
6 0 8 9 10

```

```

11 12 0 14 1
2 3 4 0 6
7 8 9 10 0
    LUD H
    L
        1      0      0      0      0
        0      1      0      0      0
0.333333      1.5      1      0      0
1.833333      6      10.3158      1      0
1.16667      4      3.89474      0.279651      1
    U
        6      0      8      9      10
        0      2      3      4      5
        0      0      _3.16667      _9      _4.83333
        0      0      0      66.3421      2.52632
        0      0      0      0      _13.5486
    xL+.xU

0 2 3 4 5
6 0 8 9 10
11 12 0 14 1
2 3 4 0 6
7 8 9 10 0

```

Of course, also the last case was calculated with) DIV 1, because in case of DIV 0):

```

)DIV 0
WAS 1
    LUD H
DOMAIN ERROR
LUD[11] L[J;I]<-U[J;I]%U[I;I]
    ^

```

I suggest, for calculating the LU Decomposition, to enable DIV 1 in any case, since it's transparent in case there are no nulls on the diagonal, but essential in the opposite case.

Let's see an example of system solution through the LU decomposition. The matrix MM is invertible; let's prove it:

```

MM
16 2 3 4 5
6 7 8 9 10
11 12 13 14 15
16 17 18 _16 20
21 22 23 24 _25
dMM
131250

```

The determinant is not null, so MM is invertible. Now, let's use the two symmetric cases for the knowns vector T printed as follows:

T

1
2
3
4
5

The first is the matrix division operator that belongs to the *apple* operators set:

```
T | % | MM
1.38778E_17
_0.4
0.6
6.93889E_18
_1.38778E_17
```

The second is calculated through the LU Decomposition (see the last formula):

```
( | % | U) + .x ( | % | L) + .xT
0
_0.4
0.6
_1.26883E_17
_2.66454E_17
```

The two solutions (apart from the noise of the quasi-zero numbers) are the same. Let's prove they are the right solutions:

```
X<-( | % | U) + .x ( | % | L) + .xT
MM+.xX
1
2
3
4
5
```

The vector T was retrieved entirely right, and this proves that the solutions are correct. This simple procedure shows that the inversion of M and the inversions of L and U are the same process, but the inversion of M may be a more challenging task than the inversion of two triangular matrices, and probably if the linear systems is made of hundreds or thousands equations, the second calculation is faster and less error-prone.

8.11. Jordan and Schur Decompositions

The Jordan Decomposition is a process that decomposes a square matrix *A* into a diagonal matrix *J* and an eigenvectors matrix *P* so that the rebuild is:

$$A = P \cdot J \cdot P^{-1}$$

The theory behind the QR Algorithm and the eigen-decomposition can be found in the *apple Extended Features Manual*.

Jordan decomposition. The program JORD (in the file JORD.apl) uses the QR algorithm to determine the eigenvalues and eigenvectors of a given matrix (called the *eigenspace* of the matrix); it returns the number of iterations of

the cycle and stores the results in the variables J and P.

Here are some examples; the first is a symmetric matrix (a symmetric matrix has all real eigenvalues and eigenvectors, and the QR algorithm works at its best with this kind of matrices):

```

BI
280  _160  _131  _131  _154  247
_160  271   _5  184   232  _148
_131   _5  172   24    9  _93
_131  184   24  154   152  _115
_154  232    9  152   203  _142
247  _148  _93  _115  _142  224
JORD BI
40
J
920.035      0      0      0      0      0
      0  291.478      0      0      0      0
      0      0  68.1309      0      0      0
      0      0      0  23.8808      0      0
      0      0      0      0  0.398488      0
      0      0      0      0      0  0.0770449
P
1.12005      1.35928      0.548568      1.03272      _1.11136      _0.925785
_1.07121      1.52519      0.354933      3.35304      0.668171      _1.50618
_0.350533      _2.1032      1.42526      3.1543      _0.209971      _0.33757
_0.800284      0.736817      0.70973      _12.0258      _0.296774      0.38357
_0.959236      1.08029      0.173716      7.38429      _0.677021      1.44684
      1      1      1      1      1      1
P+.xJ+.x|%/P
280  _160  _131  _131  _154  247
_160  271   _5  184   232  _148
_131   _5  172   24    9  _93
_131  184   24  154   152  _115
_154  232    9  152   203  _142
247  _148  _93  _115  _142  224

```

The second example is a simple 3×3 matrix:

```

R1
1  1  1
_2  4  _2
_1  0  1
JORD R1
10001
J
3.5874      0      0
      0  1.46614      0
      0      0  0.946458

```

```

P
_2.5874 _3.35168 0.308227
_7.69464 1.257 0.026316
1 1 1
P+.xJ+.x| % | P
1.63335 0.627409 _0.228229
0.627409 3.3431 _0.256455
_0.228229 _0.256455 1.02355

```

Rayleigh iteration. As it can be seen in the last examples, sometimes this algorithm fails. Why?

This is due to many reasons:

- The matrix has (some) complex eigenvalues, two-by-two conjugated
- The matrix has (some) null eigenvalues (it's defective)
- The QR algorithm does not converge
- The matrix has (some) multiple eigenvalues (i.e. some eigenvalues with multiplicity greater than 1) that the algorithm does not calculate equal (so they differ where they should not)

So I conceived another program, called EIGEN (in the file EIGEN.apl) that, using the Rayleigh iteration, recalculates the eigenvectors in P (and also the eigenvalues in J , as output by JORD); in doing this, the Rayleigh iteration sometimes succeeds where JORD fails, following these steps:

$$b_{i+1} = \frac{(A - \mu_i I)^{-1} b_i}{\|(A - \mu_i I)^{-1} b_i\|}$$

$$\mu_{i+1} = \frac{b_{i+1}^* A b_{i+1}}{b_{i+1}^* b_{i+1}}$$

In few iterations the eigenvalue μ_{i+1} and the eigenvector b_{i+1} are refined; the initial value of b_0 is taken from the correspondent column of $A - \mu_i I$ and the starting value of μ_0 is the correspondent eigenvalue; this demonstrates why sometimes it needs only one iteration to complete the algorithm.

An example is thousand words. This is the output of JORD applied to a simple invertible 3×3 matrix:

```

M
1 2 3
4 5 6
7 8 _9
JORD M
175
J
10.0593 0 0
0 _0.424882 0
0 0 _12.6344
P
1.11896 22.1115 _0.175977
2.67405 _9.62658 _0.300327

```

1 1 1

This is not the right eigenvectors matrix; while the eigenvalues are correct, the eigenvectors are not. The program EIGEN is executed as follows, feeding it with the previous result of J (which is correct) and the given matrix:

```
J EIGEN M
1001 1 2
```

The returned values are the limits of the Rayleigh iterations for each individual eigenvalue of the matrix in the left argument (in this case, of order 3). The calculation limit was fixed to 1000 because the algorithm should return the value in few iterations, and 1000 seems sufficient. Note that in most cases the cycle is completed in one or two iterations only.

The returned values are again J and P (the same returned by JORD), so if you want to retain the J matrix returned by JORD, save it under another name.

```
J
10.0593      0      0
      0 _0.424882      0
      0      0 _12.6344

P
0.718345    15.8213 _0.175977
1.75386    _12.7717 _0.300327
      1      1      1
```

The result in J is equal to the previous, but the point is that P is different, and it's the correct one. Here is the proof:

```
P+.xJ+.x|P
1 2 3
4 5 6
7 8 _9
```

Another example:

```
A1
1 2 3 4
5 6 7 8
9 10 11 _12
13 14 _15 _16
JORD A1
107
J
19.4747      0      0      0
      0 11.3876      0      0
      0      0 _0.457186      0
      0      0      0 _28.4051

P
_0.840019  0.730929  35.6273 _0.158406
```

```

_2.2466      1.80633    _14.5582  _0.294003
_4.31854    _0.850311    0.875052  0.415319
      1          1          1          1
P+.xJ+.x|P

0.696698    3.30011    2.94474    4.35714
3.30011     8.62288    6.97457    8.51242
2.94474     6.97457    11.9732   _14.2529
4.35714     8.51242   _14.2529   _19.2928

```

Again, the eigenvectors are not good. Let's apply EIGEN:

```

J EIGEN A1
3 1 4 2
J

19.4747      0          0          0
      0    11.3876      0          0
      0          0   _0.457186      0
      0          0          0   _28.4051
P

_1.09592     0.342594    19.2659  _0.158406
_2.97969     0.863647   _16.4702  _0.294003
_6.09582    _0.722855    0.288774  0.415319
      1          1          1          1
P+.xJ+.x|P

1      2      3      4
5      6      7      8
9     10     11   _12
13    14   _15   _16

```

In some other cases, the algorithm does not work at all. See the next example, again with some multiplicity involved; it's an almost diagonal matrix:

```

Q<-4 4p1 1 0 0 0 1 0 0 0 0 1 1 1 1 0 1
Q

1 1 0 0
0 1 0 0
0 0 1 1
1 1 0 1
JORD Q
10001
J

1.0002      0          0          0
      0    1.0002      0          0
      0          0   0.999801      0
      0          0          0   0.999798
P

0.000400886    9.65068E_05    _10362.1    _2494.48

```

```

6.05243E_08 _1.74094E_09 5.21118E+07 _0.496012
_0.000198845 5029.05 1.8045E_05 _0.000150976
1 1 1 1
P+.xJ+.x|%/P

0.999798 _6.56479E_10 _2.45281E_11 1.62047E_07
_6.55382E_10 0.999801 2.08102E_12 2.45275E_11
_2.45277E_11 _4.83893E_15 1.0002 _6.47489E_10
1.62047E_07 2.45286E_11 _6.48527E_10 1.0002

```

The eigenvectors, printed with)FUZZ 0, are badly approximated, not really wrong, but the rebuild fails in practice (the iterations reach the maximum value). If we try to refine them with EIGEN, here's what happens:

```

J EIGEN Q
1001 1001 1 1001
J

1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
P

3.965E_07 2.01698E_07 1 0
7.86846E_14 0 1 0
1.68305E+06 2.48143E+06 0 _4.95743E+06
1 1 1 1
P+.xJ+.x|%/P

1.00068 _0.000474211 9.04306E_17 _0.00121347
0.000303732 0.999902 _9.84844E_21 _0.00121347
_0.000976562 _1.00488 1.00026 1.00103
1.18377 _1.18394 4.53661E_10 0.999162

```

As can be easily seen, the eigenvalues are bettered (they are four ones as it should) but the eigenvectors are not well approximated; Wolfram Alpha returns:

$$S = \begin{Bmatrix} 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{Bmatrix}$$

This demonstrates that the Rayleigh iteration sometimes fails in the determination of the correct eigenvalues and eigenvectors of a matrix, even if given the correct eigenvalues matrix, and in these cases some other starting matrix should be given as the left argument of EIGEN (a very difficult task) or devised some other method than the Rayleigh iteration.

Schur Decomposition. The third program, SCHUR (in the file SCHUR.apl), uses the same QR algorithm of JORD, but doesn't make any assumptions on the eigenvalues and eigenvectors, nor any reordering, returning the number of iterations of the QR algorithm, and storing the final triangular matrix (called *Schur matrix*) in S and the relative orthogonal matrix in T; for the two examples above, the corresponding results are:

```
SCHUR M
```

```

S
    _12.6344      3.51036      _2.10616
1.85337E_16      10.0593      _2.69099
      0          0          _0.424882
T
    _0.166196      0.364914      _0.916088
    _0.283635      0.872058      0.398832
    0.944421      0.326118      _0.0414303
SCHUR Q
1001
S
    1.002      0.999996      _0.00198601      0.00199999
    _2.00199E_06      1.002      1.002      1.00198
    3.99199E_09      _3.98003E_06      0.998002      0.996006
    3.992E_12      _3.98004E_09      _1.998E_06      0.998002
T
    1.99799E_06      0.00399      0.99999      _0.00199999
    _3.45532E_14      5.98201E_06      0.00199999      0.999998
    0.999998      _0.00199999      5.98201E_06      3.46332E_14
    0.00199999      0.99999      _0.00399      1.99799E_06

```

8.11.1. Eigenspace calculating procedure

The procedure that I suggest here for the calculation of the eigenvectors and the eigenvalues of a square matrix with **apple** library programs is the following:

- 1 First of all, load and call **JORD** and apply it on your matrix. The program will produce the two matrices **J** and **P**.
- 2 If the program returns an error, the matrix is not suitable for analysis, so stop here.
- 3 Try the rebuild $P + .xJ + .x | \% | P$: if the result is your original matrix, you're done, so stop here..
- 4 If no error is returned, but the rebuild fails, load and call **SCHUR** on the same matrix. The program will produce the two matrices **S** and **T**. Check **S** as first; if it's not triangular, your matrix has some complex eigenvalues, so stop here. Eventually, derive the real and imaginary parts of these complex eigenvalues, if needed, using the method suggested in the **apple** Extended Features Manual.
- 5 If **S** is perfectly triangular, the eigenvalues are all real; to check if they are the right ones, use the program **EIGTEST** or proceed manually.
- 6 Load and call **EIGEN**, by setting the **J** matrix that you derived from **JORD** as its left argument and your original matrix as its right argument. **EIGEN** will produce a new matrix **J** (with some possible refinements derived from the Rayleigh iteration, if any), and a new matrix **P**, hopefully correct.
- 7 If the program returns an error, again, the matrix is not suitable for analysis, so stop here.
- 8 Using the same techniques of the previous step 5, and only in case some eigenvalues changed, verify if the eigenvalues are correct. If not so, jump to step 10.
- 9 Check the rebuild of your original matrix through the same expression as before, $P + .xJ + .x | \% | P$; if the matrix is completely rebuilt (again, within acceptable approximation), then the eigenvectors and the eigenvalues are correct. Stop here.
- 10 If even the second rebuild fails, or if the eigenvalues that are produced by **EIGEN** are not correct, stop here. There are no possible solutions using the two methods adopted, the QR algorithm and the Rayleigh iteration. You should write another program, with another algorithm, for solving such matrices: you can bet I'll add it to the library, if you send it to me!

To tell the truth, and according also to Wikipedia⁶⁷, the Rayleigh iteration could be used alone for the generation of the eigenvalues and eigenvectors of any square matrix. Simply start with an acceptable left diagonal matrix, and J will be produced with the eigenvalues in the diagonal, and the hopefully correct eigenvectors in P . So why, you may ask, do you suggest to use JORD instead?

Well, it's because the QR algorithm derives hopefully correct values without any other user intervention. The Rayleigh iteration, instead, requires starting values for the eigenvalues, and if they are badly chosen, the search may return unsuitable or plainly wrong values. And my experience says that choosing the right starting vectors is a very tough task!

8.12. SV Decomposition

The SV Decomposition (SVD) decomposes matrix A (with any dimensions) in two square matrices U and V and in one diagonal matrix Σ containing the singular values of the original matrix, to enable the rebuild of the original matrix according to this equation:

$$A = U \cdot \Sigma \cdot V^T$$

The theory behind the SV Decomposition can be found in the *apple Extended Features Manual*. The program SVD (in the file `SVD.apl`) returns the value representing the QR iterations accomplished; it produces three results, being the matrices U and V , plus the matrix $SIGMA$ that stands for Σ .

Here are some examples:

```

A1
1      2      3      4
5      6      7      8
9      10     11     _12
13     14     _15     _16
SVD A1
40
U
_0.0595794    0.199605    _0.309054    _0.927951
_0.0489776    0.519722    _0.76813    0.370764
0.448697     0.757918    0.472957    _0.0232964
0.89035     _0.340011    _0.301285    _0.0299597
V
0.484506     0.266281    _0.353328    0.754657
0.523451     0.324524    _0.437307    _0.655321
_0.282986     0.905172     0.316968     0.0106955
_0.641228     0.0666484    _0.763841     0.0305366
SIGMA
31.5958      0          0          0
0      19.5257      0          0
0          0      10.7826      0
0          0          0      0.432945
U+.xSIGMA+.x\oV
1      2      3      4
5      6      7      8

```

⁶⁷ See here: https://en.wikipedia.org/wiki/Rayleigh_quotient_iteration

9 10 11 _12
13 14 _15 _16

The fact U and V are orthogonal, with) FUZZ set to zero, is proved by:

```

U+.x| % |U
      1  1.64799E_17  _2.77556E_17  3.46945E_17
6.76542E_17      1  _1.8735E_16  2.08167E_17
_1.38778E_17  7.97973E_17      1  _1.11022E_16
_1.11022E_16  _7.63278E_17  5.55112E_17      1
(| % |U)+.xU
      1  8.50015E_17  8.32667E_17  9.71445E_17
_1.07553E_16      1  _2.28983E_16  8.32667E_17
_1.73472E_17  7.63278E_17      1  _5.55112E_17
_3.46945E_17      0  _1.11022E_16
V+.x| % |V
      1      0  2.22045E_16  _1.66533E_16
_5.55112E_17      1  1.11022E_16  _1.66533E_16
1.11022E_16  _8.32667E_17      1  0
1.66533E_16  _1.66533E_16  _2.77556E_17      1
(| % |V)+.xV
      1  1.94289E_16  1.94289E_16  _1.66533E_16
2.77556E_17      1  1.249E_16  1.38778E_16
2.77556E_17  _5.55112E_17      1  _1.11022E_16
5.55112E_17  8.32667E_17      0  1

```

Another example with a non-square matrix:

```

R3
1 1 2
1 2 1
2 3 3
3 3 3
SVD R3
47
U
0.302976  _0.707107  0.273628
0.302976  0.707107  0.273628
0.605952  _1.12924E_15  0.547255
0.670248  _1.47181E_15  _0.742137
V
0.496149  _3.42574E_16  _0.868238
0.613937  0.707107  0.35083
0.613937  _0.707107  0.35083
SIGMA
7.71664  0  0

```

```

      0      1      0
      0      0  0.67337
U+.xSIGMA+.x

1  1  2
1  2  1
2  3  3
3  3  3

```

The rebuild is perfect, but what about the orthogonality of U and V?

```

(|%|U)+.xU

      1  2.77556E_17  2.77556E_17
_2.77556E_17      1      0
1.11022E_16  2.77556E_17      1
U+.x|%|U

0.666667  _0.333333  0.333333  1.38778E_16
_0.333333  0.666667  0.333333  1.38778E_16
0.333333  0.333333  0.666667  _5.55112E_17
8.32667E_17  8.32667E_17  _1.66533E_16      1

```

Well, the previous result shows that U is left-orthogonal. And V?

```

(|%|V)+.xV

      1  _5.89455E_17  _5.55112E_17
_5.59387E_17      1  _1.3127E_17
1.66533E_16  _9.11422E_17      1
V+.x|%|V

      1  _1.11022E_16  _2.22045E_16
_5.55112E_17      1      0
5.55112E_17  _5.55112E_17      1

```

V is instead fully orthogonal.

8.13. Polar Decomposition

The Polar Decomposition factorizes any matrix A into two matrices called U and P , where U is an orthogonal matrix, and P is a positive semi-definite symmetric matrix, so that:

$$A = U[md]P$$

The theory behind the Polar Decomposition can be found in the *apple Extended Features Manual*. The program POL (in the file POL.apl) calculates this decomposition, and returns the number of iterations of the QR decomposition in the SVD cycle, and stores the results in the matrices U and P; here's an example with an invertible matrix:

```

M

_4  2  3  4
5  6  7  8
9  10 _8  12
13 14 15 16

```

```

POL M
22
U
_0.842563    0.312916    0.130392    0.418532
0.209971 _0.556939    0.298245    0.746178
0.288206    0.400717 _0.710266    0.501882
0.403655    0.656756    0.624156    0.127136
P
12.2615  8.10793  2.69129  8.22647
8.10793  10.486   3.68578  12.1129
2.69129  3.68578  17.5234  4.37083
8.22647  12.1129  4.37083  15.7003

```

The fact that U is orthogonal is here:

```

U+.x| % |U
1  0  0  0
0  1  0  0
0  0  1  0
0  0  0  1
U+.x
1      2.0095E_14 _1.88377E_13 _4.4742E_13
2.0095E_14      1      9.7708E_13 1.52728E_12
_1.88377E_13 9.7708E_13      1 _4.3196E_13
_4.4742E_13 1.52728E_12 _4.3196E_13      1

```

The last result is an identity matrix if we consider that all the values out of the diagonal are all practically zero (the error is lesser than 10^{-11}). The rebuild of the original matrix follows:

```

U+.xP
_4  2  3  4
5  6  7  8
9 10 _8 12
13 14 15 16

```

In case of not invertible matrices, POL works the same, even if it takes some minutes:

```

M
1  2  3  4
5  6  7  8
9 10 11 12
13 14 15 16
POL M
10001
U
_0.535706 _0.162186 0.211333 0.584853
_0.162186 0.0442326 0.250651 0.45707

```

```

0.211333    0.250651    0.289969    0.329287
0.584853    0.45707     0.329287    0.201504
P

```

```

8.15845    8.25675    8.35504    8.45334
8.25675    8.84652    9.43629    10.0261
8.35504    9.43629    10.5175    11.5988
8.45334    10.0261    11.5988    13.1715
U+.xP

```

```

1   2   3   4
5   6   7   8
9  10  11  12
13 14  15  16

```

This algorithm required 10001 iterations, but the result is good, for the rebuild, with the difference that matrix U is not fully orthogonal, because the matrix M is not invertible:

```

U+.x| % | U
_2.7298     4.20898     0.519184    _0.754286
_1.50857     2.94857     1.08735     0.205714
_0.287347    1.68816     1.65551     1.16571
0.933878    0.427755     2.22367     2.12571
U+.x\oU

```

```

0.7   0.4   0.1  _0.2
0.4   0.3   0.2   0.1
0.1   0.2   0.3   0.4
_0.2   0.1   0.4   0.7
(| % | U) +.xU

```

```

0.486531    0.891429     1.29633     1.70122
0.809796    0.803265     0.796735     0.790204
0.777143    0.607347     0.437551     0.267755
1.15592     1.52816     1.90041     2.27265
(*)

```

```

0.7   0.4   0.1  _0.2
0.4   0.3   0.2   0.1
0.1   0.2   0.3   0.4
_0.2   0.1   0.4   0.7

```

In other words, U is neither orthogonal for the case $U^T \cdot U$ (orthonormal columns basis), nor for the case $U \cdot U^T$ (orthonormal rows basis).

In case of non-square matrices, the algorithm works thanks to the fact that the program SVD respects the rank-reduction case (when only the meaningful r columns in U and P are retained, and Σ is reduced to an $r \times r$ matrix):

```

M
97  56  24
17  39  60
50  26  58

```

```

90  9  53
85  22 58
POL M
71
U
0.59516  0.721188 _0.343545
_0.289238 0.5413 0.640608
0.0982945 0.124016 0.504326
0.585754 _0.398633 0.306357
0.457555 _0.112222 0.351281
P
149.338 39.9424 60.2138
39.9424 58.6651 29.3431
60.2138 29.3431 96.0535
U+.xP
97 56 24
17 39 60
50 26 58
90  9  53
85  22 58

```

In this case the columns of U form an orthonormal basis, but not the rows; this is confirmed by the following cases:

```

U+.x
0.992351 _0.00184145 _0.0253189 _0.0441192 0.0707046
_0.00184145 0.787043 0.361775 _0.188948 0.0319461
_0.0253189 0.361775 0.279387 0.162643 0.208218
_0.0441192 _0.188948 0.162643 0.59587 0.420367
0.0707046 0.0319461 0.208218 0.420367 0.345349
U+.x|U
0.992351 _0.00184145 _0.0253189 _0.0441192 0.0707046
_0.00184145 0.787043 0.361775 _0.188948 0.0319461
_0.0253189 0.361775 0.279387 0.162643 0.208218
_0.0441192 _0.188948 0.162643 0.59587 0.420367
0.0707046 0.0319461 0.208218 0.420367 0.345349
(|U)+.xU
1 5.55112E_17 0
5.55112E_17 1 _5.55112E_17
2.77556E_17 0 1
(\oU)+.xU
1 4.32987E_15 _2.08167E_15
4.32987E_15 1 3.9968E_15
_2.08167E_15 3.9968E_15

```

In other words, U is orthogonal only in case $U^T \cdot U$ is calculated (orthonormal columns basis), and not orthogonal in case $U \cdot U^T$ is calculated (orthonormal rows basis).

8.14. The condition number

KOND calculates the condition number as the ratio of the largest to smallest singular value in the Singular Value Decomposition of a matrix (the program is in the file `KOND.apl`); in case the matrix is singular, it returns the infinite value.

Let's see a practical hand-made example:

```
M
1  2  3
4  5  6
7  8  9
SVD M
47 62
SIGMA
13.9876      0      0
      0 9.44355      0
      0      0 0.408805
```

The largest singular value is 13.9876, the smallest is 0.408805; their ratio is 34.2158, which is the condition number calculated for the matrix M.

The program KOND in the library does this for you using the right decimals:

```
KOND M
34.2158
```

The logarithm in base 10 of this number is:

```
10&KOND A
1.53423
```

This value defines the loss of precision up to 1-2 digits of accuracy on top of what would be lost to the numerical method due to loss of precision from arithmetic method; in our case, assuming that the limit of computation is 10^{-15} , we expect exact results up to 10^{-13} at worst.

The next example features the calculation of the condition number for the identity matrix:

```
KOND I
1
```

This is the perfect condition number.

In case of singular matrices, the result is infinite, as shown in the next example:

```
M
1  2  3
4  5  6
7  8  9
KOND M
INF
```

9. apple WORKFLOW

Any instruction sent to **apple** by the user as a statement from the console, or any programming line sent to **apple** by the program executor, is processed by **apple** along the following steps:

- a) First of all, two different types of requests are searched:
 1. The request to edit a program; in this case, **apple** activates a purposed section (called **progMode**) to process the editing; this closes the request.
 2. The request to trace (**T/**) or set stops (**S/**) in a particular program; in this case, **apple** activates the purposed instances (called respectively **do_tracing** and **do_stopping**) to fix in memory the necessary data for the program executor for the tracing or the stopping; the output of this request is printed only in case of an error message; this closes the request.
- b) If **apple** does not find requests like those previously seen, it calls the line analyser, which deals with other different types of expressions:
 1. Go-to, using the command **->**; the go-to command behaves differently if invoked by the console (whose instance is called **parse**) or called by the program executor (whose instance is **execProg**); in the first case, the console processes the reprise of the program if no numbers are specified (naked form **->**), otherwise it accepts the **->0** command to signal the program termination and the resetting of the initial state; in the second case, the runtime process performs the jump to a different line number, provided the specified address is in the program range.
 2. Console-system commands, like **)DIGIT**, that begin with the closed parenthesis **)**; in this case, the system commands evaluator is invoked (the name of this process is **executeSystemCommand**); the output of these commands depends on the commands themselves, and in general it aims to inform about a system change, or prints some user information; this closes the request. Note that there is a subclass of system commands that don't work in runtime (namely, **)CLEAR**, **)COPY**, **)SAVE**, **)LOAD**, **)PCOPY** and **)RENAME**);
 3. APL expressions; in this case, the APL expression engine is invoked (called **analyse**). The expression engine steps are:
 - Parenthesis and square brackets balance check (this is called **checkBal**).
 - Procession of all the assignments starting from the right (this is called **do_assign**).
 - Decomposition, from the left, of the whole expression, depurated from the assignments, in a sort of metalanguage, by creating a collection of stacks for operators and values. In this phase, all the two- or three-character operators are turned into one-character tokens (this process is called **expression**).
 - Execution of the metalanguage, performing all the calculations from the right, the classic APL algebraic model (this process is called **do_calc**).
 - If, at the end of the expression, a semicolon is found, the whole process iterates starting from step *a).1* for **parse** and from *b).1* for **execProg**.
 - All the expressions (even the argument-less programs) are instructed to return an object, anyway the output is printed only in case the APL expression is not a single or composed assignment (that is only in the case of a pure mathematical expression), or in case the **|_| <-** APL command is used.

Please note that the **parse** console instance traverses all the steps in *a)* and *b)*, while the **execProg** runtime instance traverse only *b)*; this means that the tracing, the stopping and the program editing cannot be used into programs, and actually they are meta-programming instructions. Moreover, all the requests in *a)* and *b)* but the editing can be sequenced using the semi-colon; the editing request, instead must be put on its own.

10. IN THE END...

Dr. Edsger W. Dijkstra, who was a well known Algol supporter and a very well-known any-other-language hater, once wrote in a 1972 essay titled *"The Humble Programmer"*⁶⁸:

"The competent programmer is fully aware of the strictly limited size of his own skull; therefore he approaches the programming task in full humility, and among other things he avoids clever tricks like the plague. In the case of a well-known conversational programming language I have been told from various sides that as soon as a programming community is equipped with a terminal for it, a specific phenomenon occurs that even has a well-established name: it's called 'the one-liners'. It takes one of two different forms: one programmer places a one-line program on the desk of another and either he proudly tells what it does and adds the question 'Can you code this in less symbols?' - as if this were of any conceptual relevance! - or he just asks 'Guess what it does!'. From this observation we must conclude that this language as a tool is an open invitation for clever tricks; and while exactly this may be the explanation for some of its appeal, viz. to those who like to show how clever they are, I am sorry, but I must regard this as one of the most damning things that can be said about a programming language."

In another 1975 essay *"How do we tell truths that might hurt?"*⁶⁹ he expresses his ideas about some programming languages:

"The tools we use have a profound (and devious!) influence on our thinking habits, and, therefore, on our thinking abilities."

FORTRAN - "the infantile disorder" -, by now nearly 20 years old, is hopelessly inadequate for whatever computer application you have in mind today: it's now too clumsy, too risky, and too expensive to use.

PL/I - "the fatal disease" - belongs more to the problem set than to the solution set.

It's practically impossible to teach good programming to students that have had a prior exposure to BASIC: as potential programmers they are mentally mutilated beyond hope of regeneration.

The use of COBOL cripples the mind; its teaching should, therefore, be regarded as a criminal offence.

APL is a mistake, carried through to perfection. It is the language of the future for the programming techniques of the past: it creates a new generation of coding bums. [...]

PS. If the conjecture "You would rather that I had not disturbed you by sending you this." is correct, you may add it to the list of uncomfortable truths."

In both cases, Dijkstra shows his reluctant inclination to support APL; I'm afraid the 'one-liners' in his quote mostly came out of the APL world and, as for my situation, I'd been exposed to BASIC - the one of the Commodore 64 - because it's been my first programming language, so probably I'm sentenced; but, since even the hopeless and unfortunate condemned bloke has the right to one last wish, let this dude express his: I want to tell you here, before the firing squad, that I support APL!

This program was designed, written, and implemented just for fun. I'm not a professional programmer, you know. But I like programming a lot, so here it's. For the final testing phase, I benefited from the help of Ian Jones from Bristol (U.K.), who, as usual, has helped me a lot.

This manual was written using gvim and was type-processed using groff. If you find errors or if you don't like something, it's entirely my fault.

⁶⁸ available here: <https://www.cs.utexas.edu/~EWD/transcriptions/EWD03xx/EWD340.html>

⁶⁹ available here: <https://www.cs.utexas.edu/users/EWD/transcriptions/EWD04xx/EWD498.html> as one of the essays of the book "Selected Writings on Computing: A Personal Perspective", 1982.

If you find any bugs, which are undoubtedly all results of my inexperience and are due to me and to me only, write to <ing dot antonio dot maschio at gmail dot com>; don't hesitate! I will try to fix the bugs as soon as possible.

Two things are still to be said:

The first is that I hope you like apple.

The second is: it's GPL, enjoy! ♥

Appendix 1: CONCEPTS INDEX

The concept index reports all the relevant arguments in the manual.

- About this manual, 4
- Algebraic principle, The, 6
- APL language, The, 6
- APL library, 146
- Array output, 26
- ASCII rendition, 9
- Base-value function, The, 40
- Ben-Israel Algorithm, 59
- Branch command, The, 84
- Branching, 93
- CTRL-C interrupt, 84
- CTRL-D interrupt, 84
- Catenation, 19, 117
- Characters Constant, 11
- Commented program on primes,
- Comments, 50
- Compress, 37
- Concept of Workspace, The, 8
- Console execution, 100
- Constant matrix, 12
- Conversion to/from string,
- Data plot (APL library), 151
- Dash, The usage of the, 9
- Deal, 44
- Decode, 40
- Determinant,
- Differences between `apple` and the IBM APL1130, 122
- Dimension of a matrix, 21
- Dimension of a vector, 19
- Double-quoted strings, properties, 11
- Drop, 42
- Dyadic functions, 17
- Editing of the header, 90
- Editor, The, 85, 88
- Element-by-element calculus, 27
- Empty arrays, 22
- Encode, 41
- Error reports, 13
- Execution from console, 100
- Execution interrupt command, 84
- Execution of a string-statement,
- Expansion, 39
- Exponential format,
- Factoring (APL library), 147
- Form of a jump condition, The, 99
- Formatted numeric display, 46
- Function plot (APL library), 156
- Functions on arrays, 27
- Generalized inverse,
- Global and local variables, 90
- Grade down, 43
- Grade up, 43
- Header editing, 90
- Hex numbers, 10
- History, 6
- I-Beam system variables, 68
- Identity numbers, 28
- Index generator, 19
- Index of, 41
- Indexing on the left, 25
- Indexing, 23
- Inner product, 29
- Input interrupt command, 84
- Internal Editor, The, 85
- Interrupt Input, 84
- Interrupt execution, 84
- Item Names, 12
- Jump condition form, The, 99
- Labels, 94
- Lamination, 20, 117
- Language fundamentals, 10
- Library of APL programs, 146
- Local and global variables, 90
- Matrices dimension, 21
- Matrices division, 67
- Matrix inversion, 58
- Matrix inversion (APL library), 160
- Membership, 42
- Memory access, 50
- Mixed functions, 31
- Monadic and dyadic functions, 17
- Monadic functions, 17
- Moore-Penrose inverse,
- Names of Items, 12
- Names of primitive functions, 15
- Negative numbers, 10
- Non-numeric vectors, 11
- Notes, 184
- Number to string conversion,
- Numeric comparison, 51
- Outer product, 30
- Permutations, 25
- Primes (APL library), 148
- Primitive functions names, 15
- Program editing, 88
- Program numbering, 90
- Programming branches, 93
- Programming in APL, 85
- Programming labels, 94
- Properties of double-quoted strings, 11
- Properties of single-quoted strings, 11
- Pseudo inverse, , 59

- Quad variable, 70
- Quad-char variable, 72
- Ravel, 21
- Rectangular arrays, 19
- Recursion,
- Reduction, 27
- Replication, 37
- Representation function, 41
- Reshape, 22
- Reverse, 36
- Rotate, 35
- Scalar and vector constants, 10
- Scalar functions, 16
- Scalar inversion, 58
- Separation symbols, 57
- Session run-through, 125
- Single-quoted strings, properties, 11
- Special 'square' variables, 70
- Special commands, 83
- Specification, 32
- String comparison, 52
- String reversing, 53
- String to number conversion,
- String-statement execution,
- Sub-array assignment, 25
- System commands, 76
- Take, 42
- Terminal System, The, 8
- Tracing, 94
- Transpose, 33
- Type conversion, 54
- Undeclared variables, 93
- Use of spacing, 12
- Vector inversion, 58
- Vectors of non-numeric items, 11
- Vectors, 18
- Workspace, The concept of, 8

Appendix 2: LIST OF LANGUAGE AND SYSTEM ELEMENTS

In the following, f and g are scalar operators.

Operators

$_ _$ dyadic decode	$_ _$ quad,
$-$ dyadic subtraction	$\% \mid$ dyadic matrices division
$-$ monadic set negative	$\% \mid$ monadic item inverse
$\rightarrow$ go-to (in programs)	$\mid ' \mid$ quad-char,
$,$ (comma) dyadic catenate/laminate	$\mid ^ \mid$ file info,
$,$ (comma) monadic ravel	$\mid < \mid$ input from file,
$!$ dyadic binomial	$\mid > \mid$ write to file,
$!$ monadic factorial	$\mid \vee \mid$ file open/close,
$?$ dyadic random selection	$\sim$ dyadic excluding set
$?$ monadic random generator	$\sim$ monadic logical 'not'
$\{$ dyadic minimum value	$\circ \circ$ comment
$\{$ monadic floor	$\circ$ monadic type conversion
$\}$ dyadic maximum value	d monadic determinant
$\}$ monadic ceiling	e dyadic membership
$@ .f$ outer product	e monadic execute APL string
$*$ dyadic power	f dyadic print formatted
$*$ monadic exponential	$f . g$ inner product
$/$ dyadic compression over columns	$f /$ monadic reduction over columns
$/ -$ dyadic compression over rows	$f / -$ monadic reduction over rows
$/ ^ \backslash$ dyadic relative grade up	$f \backslash$ monadic scan over columns
$/ ^ \backslash$ monadic absolute grade up	$f \backslash -$ monadic scan over rows
$/ \mid \backslash$ dyadic take	i (iota) dyadic items location
$\backslash$ dyadic expansion over columns	i (iota) monadic indices list
$\backslash -$ dyadic expansion over rows	m monadic memory access
$\backslash \mid /$ dyadic drop	$\circ$ dyadic circular operations
$\backslash \circ$ dyadic generalized transpose	$\circ$ monadic π times
$\backslash \circ$ monadic transpose	$\circ -$ dyadic rotation
$\backslash \vee /$ dyadic relative grade down	$\circ -$ monadic column reversal
$\backslash \vee /$ monadic absolute grade down	$\circ \mid$ dyadic rotation
$\&$ dyadic general log	$\circ \mid$ monadic row reversal
$\&$ monadic natural log	ρ (rho) dyadic restructure
$\#$ dyadic not equal	ρ (rho) monadic shape
$\#$ monadic unique mask	r monadic rank and echelon reduction
$\%$ dyadic division	u dyadic unique set
$\%$ monadic reciprocal	$\vee$ dyadic gcd/logical 'or'
$' \mid ' \backslash$ dyadic encode	$\vee \sim$ dyadic logical 'nor'
$^$ dyadic lcm/logical 'and'	$\times$ dyadic multiplication
$^ \sim$ dyadic logical 'nand'	$\times$ monadic signum
$+$ dyadic addition	
$+$ monadic set positive (optional)	
$<$ dyadic lesser than	
$< -$ dyadic assignment	
$< =$ dyadic lesser than or equal	
$=$ dyadic equal	
$= _$ dyadic match	
$>$ dyadic greater than	
$> =$ dyadic greater than or equal	
$\mid$ dyadic residue	
$\mid$ monadic absolute value	

Symbols

" . . " double quote string
\$ token for hexadecimal integer
' . . ' single quote string
(. .) expression sectioning
· (dot) separator for inner and outer product
· (dot) token for setting the decimal part
: label definer
; separator of expressions
; separator of indices
I_ token for I-Beam variables identification
[. .] indexing group
\/ 'del' character (program editing)
_ token for negative numbers

System commands

) CLEAR, 76
) COMP, 76
) CONTINUE, 76
) COPY, 77
) DATE, 77
) DEBUG, 77
) DESCRIBE, 77
) DIGIT, 77
) DISPOSE, 77
) DIV, 77
) DROP, 77
) EDIT, 77
) ERASE, 77
) EXP, 78
) EXPORT, 78
) EXTENDED, 78
) FILES, 78
) FNS, 78
) FUZZ, 78
) HELP, 78
) IBEAM, 78
) IMPORT, 78
) LIB, 79
) LIST, 79
) LOAD, 79
) MEMORY, 79
) OFF, 79
) OPS, 79
) ORIGIN, 79
) PCOPY, 79
) PROG, 79
) PRWS, 79
) PWD, 79
) READ, 78
) RENAME 79
) SAVE, 80

) SCRIPT, 80
) SEED, 80
) SHELL, 80
) SI, 80
) SIV, 80
) SPACES, 80
) STATE, 80
) TIME, 81
) TYPE, 81
) VARS, 81
) WIDTH, 81
) WRITE, 78
) WSID, 81
S/\ set/reset stop control
T/\ set/reset tracing
-> restart program in console

Appendix 3: INDEX OF ILLUSTRATIONS

Here is the list of all the tables in the text.

Table 2.1:	ASCII rendition, 9
Table 2.2A:	Default Error reports, 14
Table 2.2B:	Console messages, 15
Table 2.3A:	Monadic scalar operators, 16
Table 2.3B:	Dyadic scalar operators, 16
Table 2.3C:	Circular functions, 17
Table 2.4:	Dimension and rank vectors, 21
Table 2.5:	Indexing cases for arrays with [], 24
Table 2.6:	Identity numbers for scalar dyadic functions, 28
Table 2.7:	Conformability requirements for scalar functions f and g , 30
Table 2.8:	Outer products for scalar dyadic function g , 31
Table 2.9A:	Mixed monadic functions, 31
Table 2.9B:	Mixed dyadic functions, 32
Table 2.9C:	Added monadic functions, 32
Table 2.9D:	Added mixed dyadic functions, 32
Table 2.9E:	Added circular functions, 32
Table 2.9F:	Linear Algebra extended operators, (on the Extended Features Manual)
Table 2.10:	I_Beam variables, 69
Table 2.11:	File error reports, 75
Table 3.1:	Special commands, 83
Table 4.1:	Form of defined functions, 85
Table 5.1A:	Limitations of apple with respect to the IBM APL\1130, 122
Table 5.1B:	Features of apple differing from those in the IBM APL\1130, 122
Table 5.1C:	Enhancements of apple with respect to the IBM APL\1130, 123

Table of Contents

1.	Introduction	1
1.1.	A necessary preamble	1
1.2.	What is this manual?	1
1.3.	Sparse bibliography	4
2.	The APL language	6
2.1.	A bit of history	6
2.2.	The algebraic principle	6
2.3.	The concept of workspace	8
2.4.	The Terminal System	8
2.5.	Language fundamentals	10
2.6.	Scalar functions	16
2.7.	Rectangular arrays	19
2.8.	Functions on arrays	27
2.9.	Mixed functions	31
2.10.	Matrix Inversion	58
2.10.1.	The Feddeev-LeVerrier algorithm	58
2.10.2.	The Ben Israel-Cohen algorithm	59
2.10.3.	Inversion rules	60
2.11.	Matrix Division	66
2.11.1.	Exact solutions	66
2.11.2.	Least squares analysis	67
2.12.	I-Beam system variables	68
2.13.	Special 'square' variables	70
2.14.	File Input/Output	72
2.14.1.	File error messages	74
3.	System commands	76
3.1.	Console commands	76
3.2.	Special commands	83
4.	Programming in APL	85
4.1.	The internal editor	85
4.2.	The programming elements	85
4.3.	Program editing	88
4.4.	Local and global variables	90
4.5.	Usage of undeclared variables	93
4.6.	Branching	93
4.7.	Labels	94
4.8.	Tracing	94
4.9.	Stop control	95
4.10.	Scripting	96
4.11.	Style guidelines	96
5.	Programming techniques	98
5.1.	On the mathematical precision	98
5.2.	On the form of the jump condition	99
5.3.	On console execution	100
5.3.1.	Argument-less execution	100
5.3.2.	Execution with arguments	100
5.3.3.	Indirect execution	101
5.4.	On program abrupt stops	101
5.5.	On bitwise calculus	102
5.6.	On roots	104

5.7.	On the matrix diagonal	107
5.8.	On recursion	109
5.8.1.	The direct recursion	110
5.8.2.	The Ackermann recursion	113
5.9.	On the axis commands	114
5.10.	On catenation and lamination	117
5.11.	On compression and expansion	118
5.12.	On identity numbers	121
6.	Differences with the original version of IBM APL\1130	122
7.	Sample terminal session	125
8.	The library	146
8.1.	Mathematical paraphernalia	147
8.2.	Factoring an integer	147
8.3.	A commented program: finding primes	148
8.4.	Plotting data	151
8.5.	Plotting a function	156
8.6.	Plotting a function giving limits	158
8.7.	Inverting matrices	160
8.8.	QR Decomposition	163
8.9.	Hessenberg Decomposition	164
8.10.	LU Decomposition	165
8.11.	Jordan and Schur Decompositions	169
8.11.1.	Eigenspace calculating procedure	175
8.12.	SV Decomposition	176
8.13.	Polar Decomposition	178
8.14.	The condition number	182
9.	apple workflow	183
10.	In the end...	184
App. 1	Concepts index	186
App. 2	List of language and system elements	188
App. 3	Index of illustrations	190